



UNC

Universidad
Nacional
de Córdoba



Facultad de Matemática,
Astronomía, Física y
Computación

RHCD-2023-172-UNC-DEC#FAMAF

Efecto de la dilución aleatoria en el aprendizaje automático profundo con redes neuronales feed-forward

por

Gamalerio Joaquín Ignacio

Presentado ante la **FACULTAD DE MATEMÁTICA, ASTRONOMÍA, FÍSICA Y COMPUTACIÓN**

como parte de los requerimientos para la obtención del grado de
Licenciado en Física de la

UNIVERSIDAD NACIONAL DE CÓRDOBA

Mayo, 2026

Director: **Dr. Francisco A. Tamarit**



Este trabajo se distribuye bajo una licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Agradecimientos

A mis padres y a mis hermanos, por haberme dado la oportunidad de estudiar la carrera que elegí y por sostenerme con su apoyo a lo largo de este camino. Sin ellos, este logro no habría sido posible.

A mis abuelos, por haberme querido siempre.

A mi novia, Sofía, por haberme dado el impulso y el coraje necesarios para atravesar la etapa final de esta carrera, incluso en los momentos más difíciles.

A mis amigos de toda la vida, que me acompañan desde antes de comenzar este recorrido, en especial a Igna, Fran y Joma.

A mis compañeros de la facultad, que con el tiempo se convirtieron también en amigos y con quienes compartí desde el inicio las aulas y los pasillos de FAMAF, en especial a Álvaro y Pau.

A mi director, Pancho A. Tamarit, por haber confiado en mí cuando más importaba.

Y a la universidad pública, gratuita y de calidad; a FAMAF y a la UNC, por haberme formado no solo en lo profesional, sino también en lo humano, y por representar un motivo permanente de orgullo.

Resumen

Las redes neuronales artificiales ocupan hoy un lugar central en el desarrollo de la inteligencia artificial moderna. Su capacidad para aprender patrones complejos a partir de datos ha permitido avances notables en problemas de clasificación, reconocimiento de imágenes, procesamiento del lenguaje natural y modelado de sistemas de alta dimensión. Sin embargo, esa misma capacidad expresiva trae consigo una dificultad fundamental: cuando el número de parámetros es grande en relación con la información disponible, una red puede aprender no solo las regularidades relevantes del problema, sino también fluctuaciones particulares del conjunto de entrenamiento. Este fenómeno, conocido como sobreajuste u *overfitting*, limita la capacidad de generalización del modelo y vuelve necesario introducir mecanismos de regularización.

Entre las técnicas de regularización más influyentes en aprendizaje profundo se encuentra *Dropout*, que consiste en suprimir aleatoriamente activaciones neuronales durante el entrenamiento. Este procedimiento reduce la dependencia excesiva entre unidades, favorece representaciones más robustas y puede interpretarse como una forma eficiente de entrenar implícitamente muchas subredes que comparten parámetros. Una variante relacionada es *DropConnect*, donde la supresión aleatoria no actúa sobre las activaciones sino sobre las conexiones individuales de la red. Ambos métodos comparten una idea común: perturbar estocásticamente la propagación de la información para desalentar soluciones frágiles y mejorar el desempeño sobre datos no vistos.

En esta tesis se propone y estudia un nuevo mecanismo de regularización, denominado *BoostDropout*, inspirado en esa familia de métodos pero distinto en su acción. A diferencia de *Dropout*, que apaga neuronas, y de *DropConnect*, que elimina conexiones de manera aleatoria, *BoostDropout* introduce una modulación multiplicativa no binaria sobre las activaciones. En lugar de decidir únicamente entre conservar o anular una señal, el método permite amplificar o modificar estocásticamente su magnitud mediante parámetros controlables. De este modo, se explora una región más amplia de perturbaciones posibles, manteniendo el espíritu regularizador de los métodos clásicos pero incorporando una dinámica más flexible.

El trabajo se desarrolla sobre redes neuronales *feed-forward* densas aplicadas a un problema de clasificación sobre un subconjunto controlado de MNIST. El diseño experimental compara una arquitectura base sin regularización estocástica con variantes regularizadas mediante *Dropout*, *DropConnect* y *BoostDropout*. Para sostener una comparación justa, las arquitecturas comparten la misma topología base y difieren únicamente en el mecanismo de regularización utilizado. La evaluación se organiza en dos etapas: una búsqueda de hiperparámetros con múltiples semillas y una evaluación final multisemilla, manteniendo una separación estricta entre entrenamiento, validación y test.

Los resultados obtenidos muestran que, dentro del protocolo experimental considerado, *BoostDropout* no solo mejora significativamente el comportamiento de la arquitectura base, sino que también supera de manera consistente a *Dropout* y *DropConnect* en las métricas principales analizadas. En particular, la estrategia propuesta logra mitigar el sobreajuste y mejorar la generalización en el problema estudiado. No obstante, las conclusiones deben interpretarse con cautela: la comparación realizada es parcial y se limita a una arquitectura, un conjunto de datos y un régimen experimental específico. Por ello, resulta natural plantear como trabajo futuro su evaluación en otras arquitecturas, otros conjuntos de datos y escenarios de mayor complejidad.

Esta tesis busca, entonces, aportar una idea original dentro del estudio de la regularización estocástica en redes neuronales. Más que presentar una solución universal, propone una dirección concreta: explorar mecanismos inspirados en *Dropout*, pero capaces de actuar de manera más rica que la simple supresión binaria de unidades o conexiones. En ese sentido, *BoostDropout* constituye una contribución inicial al diseño de estrategias de regularización más flexibles, motivadas tanto por la práctica del aprendizaje automático como por la tradición de la física estadística en el estudio de sistemas complejos.

Índice

Capítulo 1. Introducción	9
Capítulo 2. Fundamentos de redes neuronales	12
2.1 Redes neuronales artificiales	12
2.2 Perceptrón simple y perceptrón multicapa	12
2.2.1 Perceptrón simple	13
2.2.1.1 Alcances y limitaciones	14
2.2.2 Perceptrón multicapa	14
2.2.2.1 Formulación matemática	15
2.3 Funciones de activación	17
2.3.1 Funciones sigmoides.	17
2.3.2 Unidades rectificadas.	18
2.3.3 Funciones modernas.	19
2.4 Salida de la red y tipo de predicción	19
2.4.1 Predicción en aprendizaje supervisado	19
2.4.2 Clasificación	20
2.4.3 Regresión	21
2.4.4 Partición de datos: entrenamiento, validación y test	21
2.5 Entrenamiento con retropropagación	22
2.5.1 Gradientes por regla de la cadena.	23
2.5.2 Desvanecimiento y explosión del gradiente	23
2.6 Funciones de pérdida	23
2.6.1 Entropía cruzada	24
2.6.1.1 Gradientes	25
2.6.1.2 Calibración posterior mediante <i>temperature scaling</i>	25
2.6.2 Error cuadrático medio (Mean Squared Error, MSE)	26
2.7 Algoritmos de optimización	26
2.7.1 Descenso de gradiente clásico	27
2.7.2 Descenso de gradiente estocástico (SGD)	27
2.7.3 Métodos adaptativos	27
2.7.3.1 AdaGrad	27
2.7.3.2 RMSProp	28
2.7.3.3 Adam	28
2.7.4 Entrenamiento desde inicialización aleatoria y uso de <i>learning-rate schedules</i>	29
2.8 Regularización en aprendizaje profundo	29
2.8.1 El papel de la regularización en redes neuronales profundas	29

2.8.2	Sobreajuste (overfitting), generalización y compromiso sesgo-varianza	30
2.8.3	Sparsity en representaciones y en parámetros	32
2.8.4	Criterio de selección de los métodos de regularización considerados	32
2.8.5	Dropout	33
2.8.5.1	Detalles de implementación: Dropout	34
2.8.5.2	Escalado en entrenamiento e inferencia.	35
2.8.5.3	Interpretación y efecto regularizador.	36
2.8.5.4	Observaciones.	36
2.8.6	DropConnect	36
2.8.6.1	Formulación matemática	37
2.8.6.2	Intuición: diferencia con Dropout	37
2.8.6.3	Interpretación como ensamble implícito	37
2.8.6.4	Inferencia	38
2.8.6.5	Efecto regularizador	39
2.8.6.6	Comparación conceptual con Dropout	39
2.8.6.7	Evidencia empírica y comparación con Dropout	39
2.9	Modelo propuesto: <i>BoostDropout</i>	40
2.9.1	Definición formal de la capa <i>BoostDropout</i>	41
2.9.1.1	Interpretación conceptual	42
2.10	Autoencoders	43
2.10.1	Definición general	43
2.10.2	Componentes fundamentales	44
2.10.3	Usos principales	45
Capítulo 3. Detalles de implementación y diseño experimental		46
3.1	Diseño experimental y criterios de implementación	46
3.1.1	Conjunto de datos, subset de entrenamiento y partición	46
3.2	Protocolo experimental principal: redes <i>feed-forward</i> para clasificación	48
3.2.1	Arquitecturas comparadas y criterios de comparación	48
3.2.2	Hiperparámetros fijos	49
3.2.3	Entrenamiento individual, selección de modelos y trazabilidad	50
3.2.4	Métricas de evaluación	51
3.2.5	Búsqueda de hiperparámetros	51
3.2.6	Evaluación final multiseed	52
3.2.7	Reproducibilidad y separación entre fuentes de aleatoriedad	53
3.3	Protocolo complementario: autoencoders para análisis de representaciones	54
3.3.1	Motivación y objetivo del análisis cualitativo	54
3.3.2	Arquitectura del autoencoder de referencia	54
3.3.3	Variantes regularizadas analizadas	55

3.3.4	Entrenamiento y evaluación del autoencoder	56
3.3.5	Criterios de análisis de representaciones	56
3.3.6	Construcción de las visualizaciones representacionales	57
3.4	Alcances, limitaciones y sentido de los resultados	57
Capítulo 4.	Resultados experimentales	59
4.1	Resultados experimentales en clasificación con redes neuronales	
	<i>feed-forward</i>	59
4.1.1	Resultados del <i>grid search</i>	59
4.1.1.1	Configuraciones seleccionadas para <i>Dropout</i>	59
4.1.1.2	Configuraciones seleccionadas para <i>DropConnect</i>	59
4.1.1.3	Configuraciones seleccionadas para <i>BoostDropoutNet</i>	60
4.1.2	Resultados del entrenamiento principal	60
4.1.2.1	Criterio: MMV	60
4.1.2.2	Criterio: <i>early stopping</i>	62
4.1.2.3	Distribución de la mejor época	64
4.1.2.4	Curvas representativas de validación	66
4.1.3	Dependencia de las métricas respecto del parámetro p	68
4.1.3.1	Dependencia respecto de p para el criterio MMV	68
4.1.3.2	Dependencia respecto de p para el criterio de <i>early stopping</i>	69
4.1.4	Dependencia de las métricas respecto de λ	70
4.1.4.1	Criterio: mejor modelo de validación	70
4.1.4.2	Criterio: <i>early stopping</i>	72
4.1.5	Conclusiones del protocolo principal	73
4.2	Visualización cualitativa con autoencoder	75
4.2.1	Visualización de coadaptaciones	76
4.2.2	Decodificaciones a partir de neuronas latentes individuales	78
4.2.3	Esparsidad de las representaciones	80
4.2.4	Esparsidad paramétrica en la primera capa	82
4.2.5	Conclusiones del análisis cualitativo con autoencoder	83
Capítulo 5.	Conclusiones generales y próximos pasos	85
Anexo A:	Nomenclatura	87

Capítulo 1. Introducción

La inteligencia artificial contemporánea atraviesa una etapa de expansión extraordinaria. Sistemas capaces de reconocer imágenes, traducir textos, asistir en tareas científicas, generar lenguaje natural o resolver problemas complejos forman parte de una transformación tecnológica que hace apenas algunas décadas parecía lejana. En el centro de muchos de estos avances se encuentran las redes neuronales artificiales: modelos computacionales que aprenden a partir de ejemplos y que, al combinar una gran cantidad de parámetros ajustables con algoritmos de optimización eficientes, pueden capturar regularidades difíciles de describir mediante reglas explícitas.

Aunque su desarrollo actual suele asociarse a la informática y a la disponibilidad masiva de datos, las redes neuronales tienen también una raíz profunda en la física. En particular, la física estadística aportó herramientas conceptuales decisivas para pensar sistemas compuestos por muchos elementos interactuantes, donde el comportamiento colectivo no se reduce de manera simple al de cada componente individual. Desde los modelos de Hopfield hasta las formulaciones modernas del aprendizaje profundo, existe una conexión fértil entre redes neuronales, sistemas complejos, dinámica colectiva y optimización en espacios de alta dimensión. Esta tesis se inscribe dentro de esa tradición: estudiar una red neuronal no solo como una herramienta computacional, sino también como un sistema cuyos mecanismos internos pueden ser modificados, perturbados y analizados.

Una red neuronal artificial puede pensarse, en términos generales, como una función parametrizada que transforma una entrada en una salida. Durante el entrenamiento, sus parámetros se ajustan para que las predicciones se aproximen a los valores correctos de un conjunto de datos. En arquitecturas *feed-forward*, como las consideradas en este trabajo, la información fluye capa por capa desde la entrada hasta la salida. Cada capa aplica una transformación sobre la representación anterior, y la composición de muchas transformaciones permite construir descripciones internas cada vez más elaboradas. Esta capacidad de representación explica buena parte del éxito de las redes neuronales modernas.

Sin embargo, el mismo poder expresivo que vuelve atractivos a estos modelos introduce una dificultad fundamental. Una red con muchos parámetros puede aprender regularidades verdaderamente relevantes, pero también puede memorizar particularidades accidentales del conjunto de entrenamiento. Cuando esto ocurre, el modelo obtiene buenos resultados sobre los datos que vio durante el aprendizaje, pero falla al enfrentarse a ejemplos nuevos. Este fenómeno se conoce como sobreajuste, u *overfitting*, y constituye uno de los problemas centrales del aprendizaje automático. En términos prácticos, no alcanza con que una red aprenda: debe aprender de un modo que le permita generalizar.

La regularización aparece precisamente como respuesta a este problema. Bajo ese nombre se agrupa una familia amplia de técnicas cuyo objetivo es mejorar el desempeño del modelo sobre datos no vistos, aun cuando ello implique limitar su libertad durante el entrenamiento. Algunas formas de regularización actúan directamente sobre los parámetros; otras modifican la dinámica de optimización; otras introducen ruido o perturbaciones controladas en la propagación de la información. Entre estas últimas se encuentran dos métodos especialmente importantes para esta tesis: *Dropout* y *DropConnect*.

Dropout consiste en suprimir aleatoriamente una fracción de las activaciones neuronales durante el entrenamiento [1]. De este modo, la red no puede depender rígidamente de un

conjunto fijo de neuronas y se ve forzada a distribuir la información de manera más robusta. Una intuición útil es pensar que, en cada paso de entrenamiento, se entrena una subred distinta formada por las neuronas que permanecen activas. Aunque estas subredes no son modelos independientes en sentido estricto, ya que comparten parámetros, el efecto global se asemeja a una forma eficiente de ensamble implícito y reduce coadaptaciones frágiles entre unidades.

DropConnect, por su parte, traslada una idea similar al nivel de las conexiones [2]. En lugar de apagar neuronas completas, enmascara aleatoriamente pesos individuales de la red. La perturbación ya no actúa sobre qué unidades participan, sino sobre qué enlaces específicos quedan disponibles durante una pasada de entrenamiento. Así, mientras *Dropout* modifica la representación intermedia de la red, *DropConnect* altera su conectividad efectiva. Ambos métodos comparten una filosofía común: introducir una dilución estocástica durante el aprendizaje para favorecer soluciones más robustas.

El punto de partida de esta tesis es preguntarse si esa familia de ideas puede extenderse. En los métodos anteriores, la perturbación es esencialmente binaria: una neurona o conexión se conserva, o se anula. Pero nada obliga, en principio, a que una estrategia de regularización deba limitarse a esa decisión extrema. Podría imaginarse una perturbación más flexible, que no apague necesariamente la señal, sino que module su amplitud. Esta es la idea que motiva el método propuesto en este trabajo.

En esta tesis se introduce y estudia *BoostDropout*, un mecanismo de regularización estocástica inspirado en *Dropout*, pero basado en una modulación multiplicativa no binaria de las activaciones. En lugar de decidir únicamente si una señal se conserva o se suprime, *BoostDropout* permite modificar su magnitud mediante una máscara aleatoria controlada por hiperparámetros. Desde esta perspectiva, el método puede entenderse como una extensión natural de la lógica de enmascaramiento multiplicativo: conserva la idea de perturbar la red durante el entrenamiento, pero reemplaza la supresión estricta por una transformación más rica de la señal propagada.

El objetivo de este trabajo no es presentar a *BoostDropout* como una solución universal al problema de la regularización. Más bien, se busca evaluar de manera controlada si este tipo de perturbación puede mejorar la generalización en un escenario concreto y comparable. Para ello, se estudian redes neuronales *feed-forward* densas aplicadas a una tarea de clasificación sobre un subconjunto de MNIST. La elección de un subconjunto controlado permite inducir un régimen donde el sobreajuste resulta visible, y por lo tanto ofrece un marco adecuado para comparar el efecto de distintas estrategias de regularización.

La comparación experimental considera cuatro arquitecturas: una red base sin regularización estocástica, una variante con *Dropout*, una variante con *DropConnect* y la variante propuesta con *BoostDropout*. Todas comparten la misma topología base, de modo que las diferencias observadas puedan atribuirse, en la medida de lo posible, al mecanismo de regularización empleado. El protocolo incluye una búsqueda de hiperparámetros con múltiples semillas y una evaluación final multisemilla, manteniendo separadas las etapas de entrenamiento, validación y test. Esta separación es esencial para evitar conclusiones sesgadas y para evaluar de manera honesta la capacidad de generalización de cada modelo.

Los resultados obtenidos muestran que *BoostDropout* no solo mejora significativamente el desempeño de la arquitectura base en el problema estudiado, sino que también supera de manera consistente a *Dropout* y *DropConnect* bajo el protocolo experimental adoptado.

Además de mitigar el sobreajuste, esta comparación permite ubicarlo dentro de una familia de métodos de regularización estocástica, destacando tanto sus similitudes conceptuales como su diferencia principal: la sustitución de una máscara binaria por una modulación multiplicativa más flexible. En este sentido, el método propuesto constituye una contribución original de esta tesis, al menos hasta donde conocemos, y abre la posibilidad de explorar nuevas formas de regularización inspiradas en la dilución, pero no restringidas a la supresión.

La tesis se organiza de la siguiente manera. En el Capítulo 2 se presentan los fundamentos teóricos necesarios para comprender el trabajo: redes neuronales artificiales, arquitecturas *feed-forward*, entrenamiento mediante retropropagación, funciones de pérdida, algoritmos de optimización y técnicas de regularización. Allí se introducen en particular *Dropout*, *Drop-Connect* y la formulación del método propuesto, *BoostDropout*. En el Capítulo 3 se describe el diseño experimental y numérico utilizado para comparar las arquitecturas consideradas, incluyendo la construcción del conjunto de datos, la partición en entrenamiento, validación y test, la búsqueda de hiperparámetros, el control de semillas y los criterios de evaluación. En el Capítulo 4 se presentan y discuten los resultados experimentales, tanto en la tarea principal de clasificación como en los análisis complementarios de representaciones internas. Finalmente, se resumen las conclusiones principales y se proponen posibles líneas de trabajo futuro.

En conjunto, esta tesis busca mostrar que incluso en modelos relativamente simples, como las redes *feed-forward* densas, todavía existen preguntas interesantes sobre cómo regularizar, cómo perturbar la dinámica de aprendizaje y cómo diseñar mecanismos que mejoren la generalización. *BoostDropout* nace de esa pregunta. Su valor no reside únicamente en los resultados obtenidos para un problema particular, sino en señalar una dirección posible: pensar la regularización no solo como supresión aleatoria, sino como una intervención más general sobre la intensidad de las señales internas de una red neuronal.

Capítulo 2. Fundamentos de redes neuronales

2.1 Redes neuronales artificiales

Las redes neuronales artificiales constituyen una familia de modelos computacionales inspirados, de manera abstracta, en la organización y el procesamiento distribuido de los sistemas neuronales biológicos. Su desarrollo combina ideas provenientes de distintas disciplinas—entre ellas la neurociencia, la matemática aplicada, la informática y la física estadística—y ha dado lugar a una amplia variedad de arquitecturas capaces de aprender relaciones complejas a partir de datos.

Desde un punto de vista general, una red neuronal puede entenderse como una función parametrizada

$$f(\mathbf{x}; \boldsymbol{\theta}),$$

donde $\mathbf{x}$ representa la entrada y $\boldsymbol{\theta}$ reúne los parámetros ajustables del modelo, principalmente pesos y sesgos. El objetivo del aprendizaje consiste en encontrar valores adecuados de esos parámetros para que la red aproxime de la mejor manera posible la relación entre entradas y salidas presente en un conjunto de datos.

La idea fundamental detrás de estos modelos es simple: cada unidad de procesamiento recibe señales de entrada, calcula una combinación de ellas y aplica luego una transformación no lineal. Al organizar muchas de estas unidades en capas sucesivas, la red puede construir representaciones internas cada vez más elaboradas de los datos. Esta capacidad de composición es la que explica gran parte de la flexibilidad y del poder expresivo de las arquitecturas neuronales modernas.

En esta tesis nos centraremos en redes *feed-forward*, es decir, arquitecturas en las que la información fluye desde la entrada hacia la salida sin ciclos de realimentación. Dentro de esta familia, el caso más elemental es el perceptrón, que constituye el punto de partida histórico y conceptual de las redes neuronales entrenables. El perceptrón será introducido aquí como base para comprender su generalización natural a redes multicapa, que son las que permiten abordar problemas no lineales y que sirven de marco para estudiar los mecanismos de regularización analizados más adelante.

Por esta razón, en las secciones siguientes se presentarán de manera progresiva los elementos fundamentales de una red neuronal moderna. A partir de este recorrido, se introducirán las nociones teóricas necesarias para comprender tanto el funcionamiento de estas arquitecturas como las estrategias de entrenamiento y regularización sobre las que se apoya la propuesta desarrollada en esta tesis.

2.2 Perceptrón simple y perceptrón multicapa

El perceptrón constituye uno de los primeros intentos formales de modelar el proceso de decisión de una neurona artificial. Sus raíces se encuentran en los trabajos de McCulloch et al. [3], quienes introdujeron un modelo lógico de neuronas basado en funciones booleanas. Posteriormente, Rosenblatt [4] formuló el *perceptrón*, definiéndolo como una máquina capaz

de aprender a clasificar patrones linealmente separables mediante un algoritmo de aprendizaje supervisado.

2.2.1 Perceptrón simple

El perceptrón simple representa la forma más elemental de neurona artificial entrenable. Formalmente, recibe un vector de entrada $\mathbf{x} = (x_1, x_2, \dots, x_n)$, que es ponderado por un conjunto de parámetros $\mathbf{w} = (w_1, w_2, \dots, w_n)$ y un sesgo b . La salida y se define como:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (2.2.1)$$

donde $f(\cdot)$ es una función escalón, definida como:

$$f(z) = \begin{cases} 1 & \text{si } z > 0, \\ 0 & \text{si } z \leq 0. \end{cases} \quad (2.2.2)$$

La estructura general de este modelo puede visualizarse en la Figura 1, donde se identifican explícitamente las entradas, los pesos, el sesgo y la salida de la neurona.

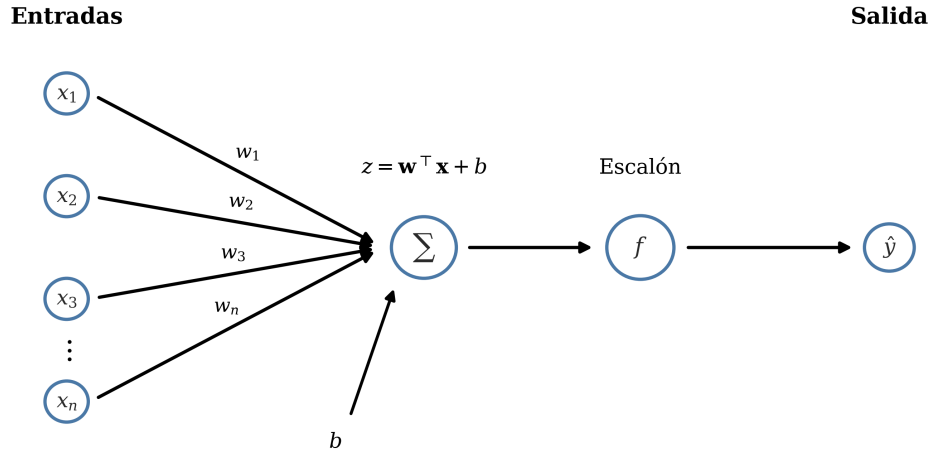


Figura 1: Esquema de un perceptrón simple. Se muestran las entradas, la combinación lineal ponderada por los pesos y la salida después de la función de activación.

Desde un punto de vista geométrico, este modelo implementa una frontera de decisión lineal en el espacio de entrada. En consecuencia, resulta adecuado para problemas de clasificación binaria cuyas clases pueden separarse mediante un hiperplano.

La Figura 2 ilustra este caso en un problema linealmente separable, donde la frontera aprendida por el perceptrón corresponde a una recta en el espacio de características.

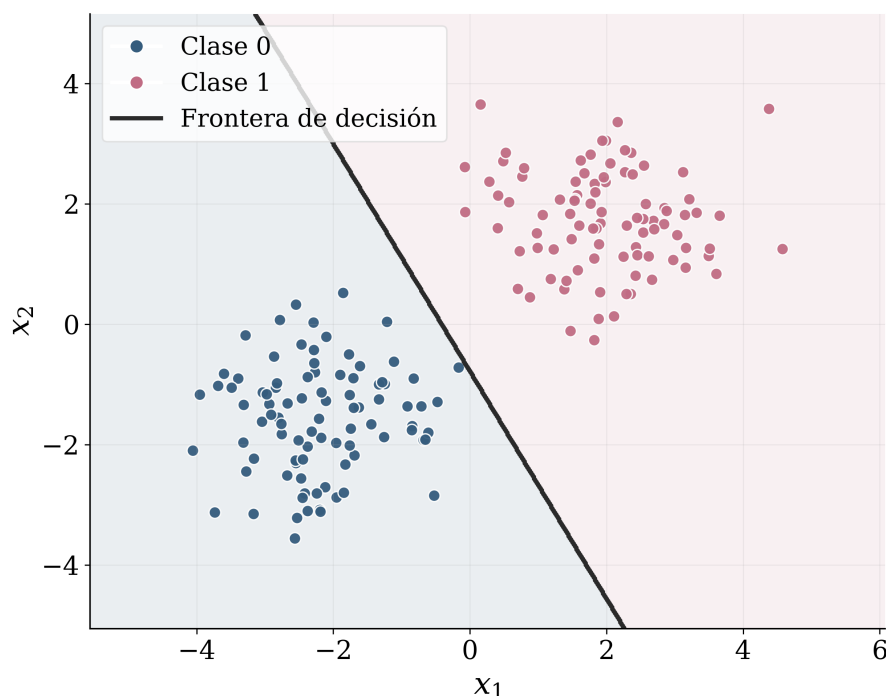


Figura 2: Problema linealmente separable y frontera de decisión lineal aprendida por un perceptrón simple.

2.2.1.1 Alcances y limitaciones

El perceptrón simple constituyó un avance fundamental en la historia de las redes neuronales, ya que introdujo de manera explícita los conceptos de pesos ajustables, sesgo, activación y aprendizaje supervisado. Sin embargo, su capacidad expresiva es limitada: solo puede resolver problemas linealmente separables.

Esta restricción fue analizada rigurosamente por Minsky et al. [5], quienes demostraron que el perceptrón es incapaz de representar funciones no lineales, siendo el caso paradigmático el de la función XOR. Este resultado condujo a un periodo de escepticismo en torno a las redes neuronales, frecuentemente denominado “invierno de las redes neuronales”.

A pesar de estas limitaciones, el perceptrón simple conserva un valor conceptual central: establece los elementos básicos de una neurona artificial que luego se generalizan en arquitecturas más profundas y expresivas.

2.2.2 Perceptrón multicapa

La principal limitación del perceptrón simple puede superarse mediante la introducción del *perceptrón multicapa* (*Multi-Layer Perceptron*, MLP), formalizado por Rumelhart et al. [6], que incorpora una o más *capas ocultas* (*hidden layers*) entre la capa de entrada y la capa de salida. Cada una de estas capas está compuesta por un conjunto de *unidades ocultas* (*hidden units*), también denominadas neuronas, que aplican transformaciones no lineales a sus entradas.

La arquitectura general de este tipo de modelo se presenta en la Figura 3, que muestra el flujo de información desde la entrada hacia la salida a través de capas ocultas intermedias.

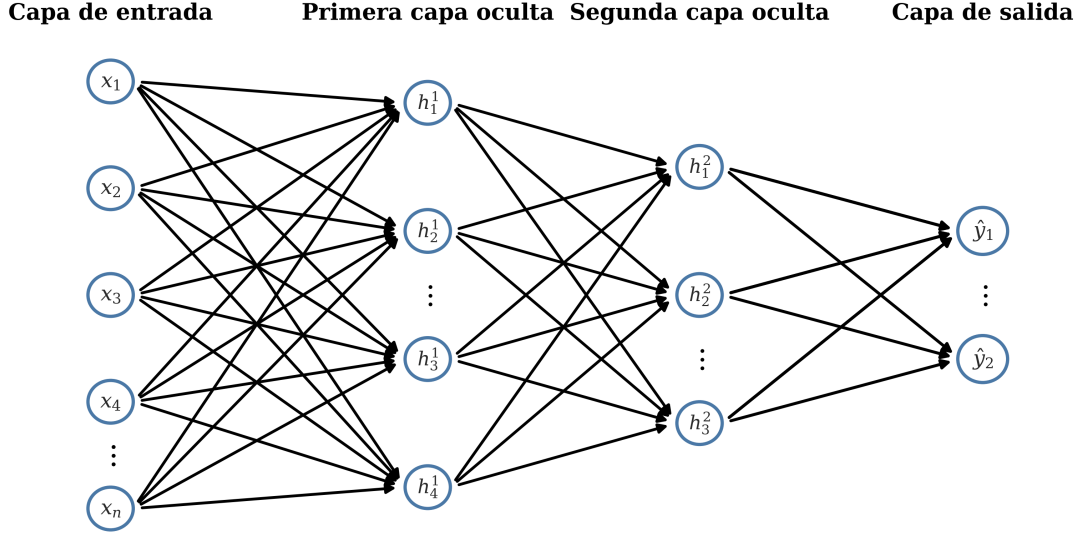


Figura 3: Esquema de un perceptrón multicapa (MLP) con capa de entrada, capa oculta y capa de salida en una arquitectura *feed-forward*.

Una *hidden unit* es una neurona artificial que pertenece a una capa oculta de la red. Su función es aprender representaciones intermedias de los datos, permitiendo que la red modele relaciones no lineales complejas. Gracias a estas capas ocultas, el MLP extiende de manera sustancial la capacidad expresiva del perceptrón simple.

2.2.2.1 Formulación matemática

Desde un punto de vista matemático, un MLP puede describirse de forma compacta utilizando notación vectorial y matricial, donde cada ecuación representa el comportamiento de una capa completa. Sea $\mathbf{x} \in \mathbb{R}^n$ el vector de entrada. La salida de la primera capa oculta se define como:

$$\mathbf{h}^{(1)} = \vec{f}^{(1)}(W^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}) \quad (2.2.3)$$

donde:

- $\mathbf{h}^{(1)} \in \mathbb{R}^{m_1}$ es el vector de activaciones de la primera capa oculta,
- $W^{(1)} \in \mathbb{R}^{n \times m_1}$ es la matriz de pesos, cuyos elementos w_{ij} conectan la entrada i con la unidad oculta j ,
- $\mathbf{b}^{(1)} \in \mathbb{R}^{m_1}$ es el vector de sesgos,

- $\vec{f}^{(1)}(\cdot)$ es la función vectorial obtenida al aplicar componente a componente la activación escalar $f^{(1)}(\cdot)$.

Cada componente $h_j^{(1)}$ del vector $\mathbf{h}^{(1)}$ corresponde a la salida de una unidad oculta individual y puede escribirse explícitamente como:

$$h_j^{(1)} = f^{(1)}\left(\sum_{i=1}^n w_{ij}^{(1)} x_i + b_j^{(1)}\right) \quad (2.2.4)$$

Las capas siguientes se construyen de manera análoga, componiendo sucesivamente transformaciones afines y funciones de activación no lineales. En términos generales, para una red de L capas, la activación de la capa ℓ puede expresarse como:

$$\mathbf{h}^{(\ell)} = \vec{f}^{(\ell)}(W^{(\ell)\top} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, 2, \dots, L, \quad (2.2.5)$$

donde se toma $\mathbf{h}^{(0)} = \mathbf{x}$. La salida final de la red resulta entonces de la composición de todas estas transformaciones.

La diferencia esencial respecto del perceptrón simple no reside únicamente en la presencia de más neuronas, sino en la posibilidad de construir representaciones internas jerárquicas. Gracias a la combinación de múltiples capas y activaciones no lineales, un perceptrón multicapa puede aproximar funciones considerablemente más complejas que aquellas representables mediante una sola frontera lineal.

Este punto se aprecia en la Figura 4, donde un problema no linealmente separable puede resolverse mediante una frontera de decisión curvada aprendida por una red *feed-forward* con una capa oculta.

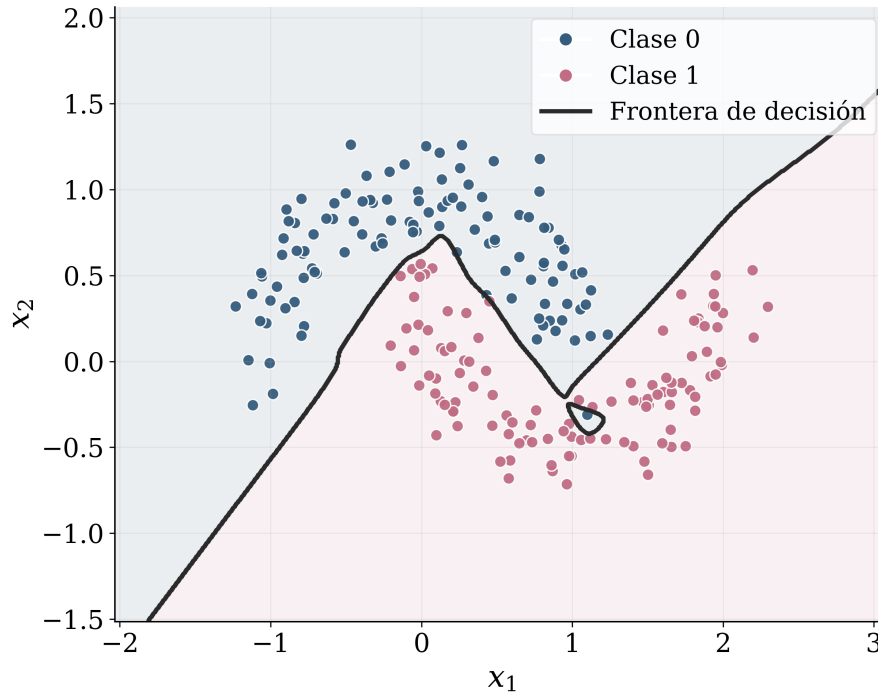


Figura 4: Problema no linealmente separable y frontera de decisión no lineal aprendida por una red *feed-forward* con una sola capa oculta.

2.3 Funciones de activación

Como se mencionó en la sección anterior, las funciones de activación introducen la **no linealidad** en las redes neuronales. Esta propiedad es indispensable para que un MLP pueda modelar relaciones complejas: sin activaciones no lineales, la composición de capas se reduce a una única transformación afín, de modo que una red profunda sería esencialmente equivalente a un modelo lineal.

Un resultado teórico central que justifica el poder expresivo de estas arquitecturas es el *teorema de aproximación universal*, demostrado por Cybenko [7] y extendido por Hornik et al. [8]. En términos generales, este teorema establece que un MLP con una sola capa oculta y un número suficiente de neuronas puede aproximar con precisión arbitraria cualquier función continua definida sobre un dominio compacto.

2.3.1 Funciones sigmoides.

Entre las primeras funciones utilizadas se encuentra la **sigmoide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (2.3.1)$$

la cual comprime la entrada en el intervalo $(0, 1)$, lo que la hizo atractiva en tareas de clasificación binaria. Una función emparentada es la **tangente hiperbólica**, $\tanh(z)$, cuyo rango es $(-1, 1)$ y presenta simetría alrededor de cero.

Las formas características de ambas funciones se muestran en la Figura 5, con detalle en las subfiguras 5a y 5b.

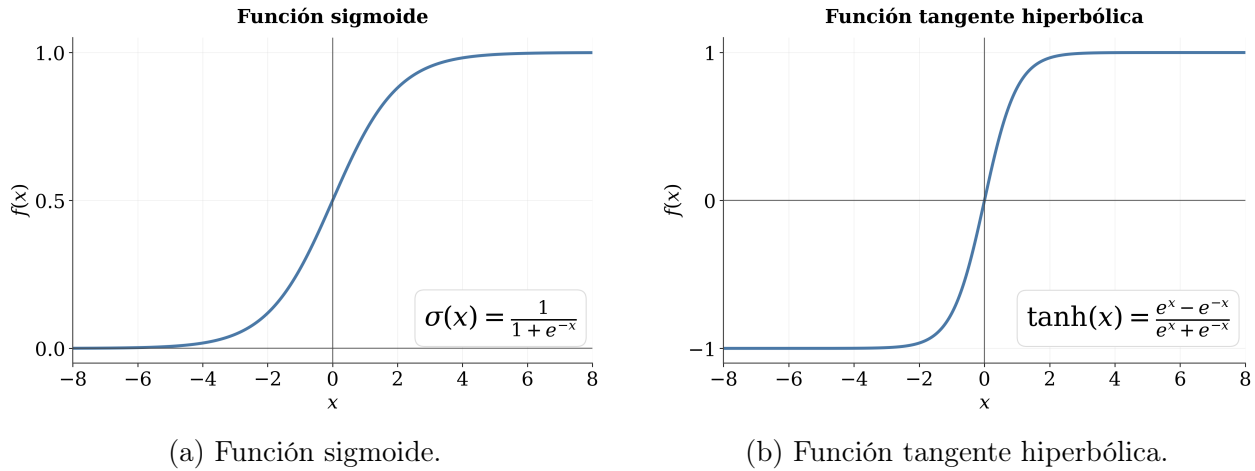


Figura 5: Funciones de activación sigmoides utilizadas históricamente en redes neuronales: (a) sigmoide logística y (b) tangente hiperbólica.

Ambas funciones comparten una característica relevante: presentan **saturación** para valores grandes de $|z|$, donde la pendiente se vuelve muy pequeña. Este comportamiento será importante al discutir el entrenamiento mediante métodos basados en gradientes (Sección 2.5), ya que en redes profundas puede dificultar la actualización eficiente de parámetros [9].

2.3.2 Unidades rectificadas.

Para mitigar los inconvenientes de las activaciones saturantes, Nair et al. [10] introdujeron la *Rectified Linear Unit* (ReLU), definida por:

$$f_{\text{ReLU}}(z) = \max(0, z) = \begin{cases} z, & \text{si } z \geq 0, \\ 0, & \text{si } z < 0. \end{cases} \quad (2.3.2)$$

La ReLU se destaca por su simplicidad computacional y por inducir **activaciones esparsas** (*sparsity*), ya que anula exactamente una fracción de neuronas para entradas negativas. Además, en el semieje positivo conserva una pendiente constante, lo cual suele favorecer el entrenamiento eficiente de redes profundas. Estas propiedades contribuyeron a que se convirtiera en una elección estándar en aprendizaje profundo [11].

La forma funcional de ReLU se ilustra en la Figura 6.

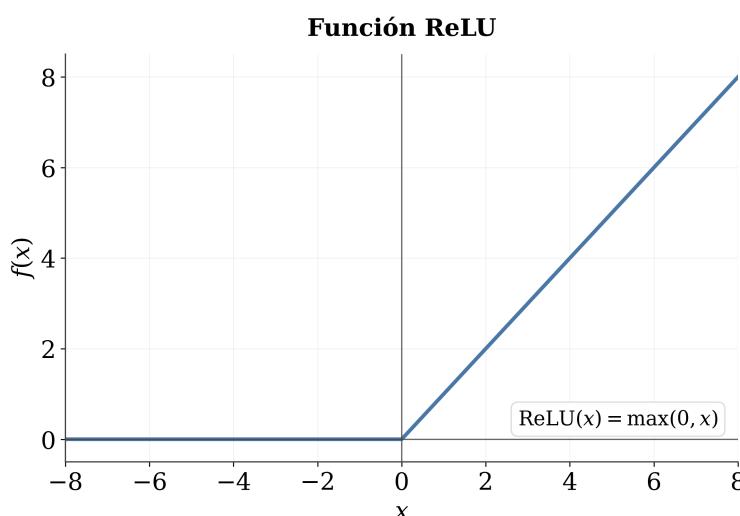


Figura 6: Función de activación ReLU. La activación es nula para entradas negativas y lineal para entradas no negativas.

Si bien el uso de activaciones no saturantes como ReLU mitiga el desvanecimiento del gradiente en redes profundas, introduce un problema distinto conocido como *neuronas muertas*, en el cual ciertas unidades permanecen inactivas y dejan de contribuir al aprendizaje. Ambos fenómenos están relacionados con la propagación de gradientes, pero difieren en su origen y en su impacto: el primero es un efecto global del entrenamiento, mientras que el segundo afecta localmente a neuronas individuales.

Existen variantes diseñadas para aliviar el fenómeno de las *neuronas muertas*, tales como la *Leaky ReLU* y la *Parametric ReLU*, que introducen una pendiente no nula en la región negativa. En *Leaky ReLU* dicha pendiente se fija de antemano, mientras que en *Parametric ReLU* se aprende durante el entrenamiento mediante un parámetro a , que controla explícitamente la inclinación para entradas negativas.

La Figura 7 muestra ambas variantes, destacando la diferencia entre una pendiente negativa fija (*Leaky ReLU*, subfigura 7a) y una pendiente aprendible (*PReLU*, subfigura 7b).

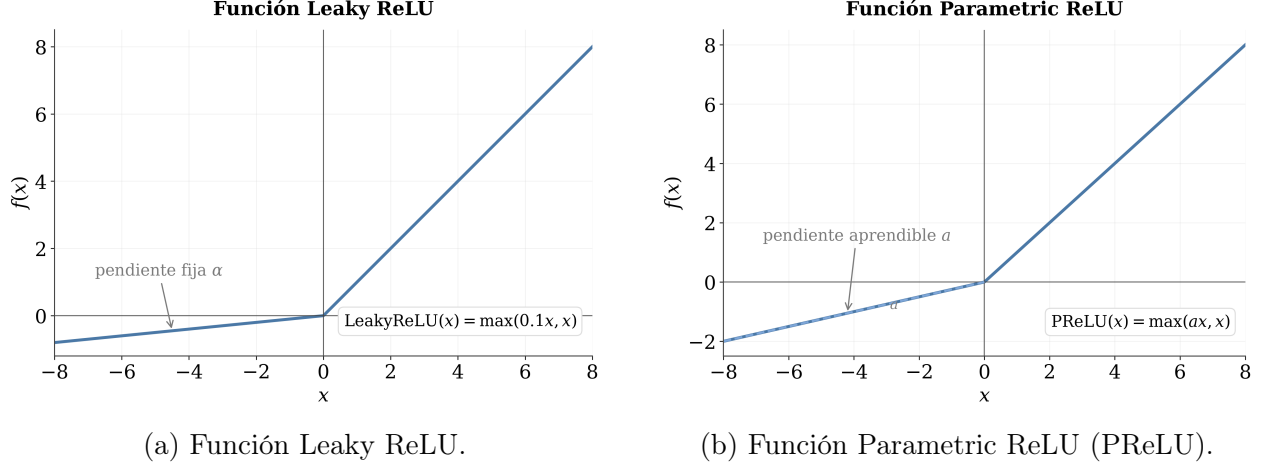


Figura 7: Comparación entre variantes rectificadas con pendiente negativa no nula: (a) Leaky ReLU y (b) Parametric ReLU. En PReLU, el parámetro aprendible a determina la pendiente en la región negativa.

2.3.3 Funciones modernas.

Más recientemente, se han propuesto activaciones suaves como *ELU* o *Swish*, que combinan no linealidad con derivabilidad en todo el dominio y pueden mejorar la estabilidad numérica en ciertas configuraciones [12, 13]. En general, la elección de la función de activación influye en la estabilidad del entrenamiento, la velocidad de convergencia y el desempeño final del modelo.

2.4 Salida de la red y tipo de predicción

Hasta aquí se describió cómo una red neuronal transforma una entrada en una sucesión de representaciones internas. Sin embargo, para formular con precisión el problema de aprendizaje, todavía falta explicitar cómo se interpreta la **salida final** de la red. Esta cuestión depende del tipo de tarea que se desea resolver: en algunos casos la red debe asignar una etiqueta discreta a cada entrada, mientras que en otros debe estimar una magnitud continua. Por ello, antes de introducir los detalles de entrenamiento de una red neuronal, conviene distinguir los dos esquemas de predicción más relevantes en esta tesis: clasificación y regresión.

2.4.1 Predicción en aprendizaje supervisado

Desde un punto de vista general, una red *feed-forward* implementa una función parametrizada

$$f(\mathbf{x}; \boldsymbol{\theta}),$$

que transforma la entrada $\mathbf{x}$ en una salida final. En aprendizaje supervisado, a cada entrada se le asocia además un **target** y , es decir, la salida correcta o deseada que el modelo debería reproducir. Según la tarea, ese target puede ser una etiqueta discreta, un valor real o, más en general, un objeto de salida perteneciente a cierto espacio predefinido.

La red produce entonces una salida propia, que puede interpretarse como una estimación del target. Esa estimación se denota habitualmente por $\hat{y}$: representa la **predicción del modelo**, en contraste con y , que representa el valor verdadero provisto por los datos. El entrenamiento consiste, en esencia, en ajustar los parámetros θ para que $\hat{y}$ se aproxime lo mejor posible a y en las muestras observadas.

En términos esquemáticos, el proceso puede resumirse como

$$\mathbf{x} \longrightarrow \text{capas ocultas} \longrightarrow \mathbf{z} \longrightarrow \hat{y},$$

donde $\mathbf{z}$ representa la salida cruda de la última capa de la red, antes de la interpretación final, y $\hat{y}$ representa la predicción resultante. En muchos casos, $\hat{y}$ se obtiene directamente a partir de $\mathbf{z}$; en otros, surge después de aplicar una transformación adicional. En clasificación multiclase, por ejemplo, $\mathbf{z}$ suele interpretarse como un vector de *logits*, es decir, puntajes numéricos sin normalizar que indican cuán favorecida está cada clase antes de aplicar una función como *softmax* para convertirlos en probabilidades. En regresión, en cambio, la salida final puede coincidir directamente con la cantidad predicha, de modo que no siempre es necesario distinguir de manera operativa entre $\mathbf{z}$ y $\hat{y}$.

La elección de la activación de salida y de la función de pérdida depende, por lo tanto, de la naturaleza del target y y de cómo se construye la predicción $\hat{y}$ a partir de la salida de la red. Esta correspondencia entre tipo de problema, interpretación de la salida y criterio de error es uno de los aspectos básicos del diseño de redes neuronales.

2.4.2 Clasificación

En una tarea de **clasificación**, el objetivo es asignar a cada entrada $\mathbf{x}$ una etiqueta discreta. El target (etiqueta) y representa entonces la clase verdadera asociada a la muestra. Si el problema es binario, puede ocurrir que $y \in \{0, 1\}$; si es multiclase, típicamente $y \in \{1, \dots, C\}$, donde C es el número de clases posibles.

En este contexto, la red no suele producir directamente una etiqueta, sino un conjunto de puntuaciones reales. En clasificación multiclase, la última capa genera un vector

$$\mathbf{z} = (z_1, \dots, z_C),$$

cuyos componentes indican cuán compatible es la entrada con cada clase. Estas cantidades son los ya mencionados *logits*. Para interpretarlas probabilísticamente, se aplica habitualmente la función *softmax*:

$$\hat{p}_c = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}, \quad c = 1, \dots, C,$$

de modo que $\hat{p}_c$ puede entenderse como la probabilidad predicha para la clase c .

La etiqueta predicha por la red se obtiene entonces como la clase de mayor probabilidad:

$$\hat{y} = \arg \max_c \hat{p}_c.$$

En clasificación binaria, en cambio, es frecuente que la red produzca un único logit z , que luego se transforma mediante una sigmoide:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}},$$

donde $\hat{p}$ representa la probabilidad predicha de una de las dos clases.

En ambos casos, la idea central es la misma: la red transforma la entrada en una predicción discreta o probabilística, que luego se compara con la etiqueta verdadera y . Esta comparación da lugar, naturalmente, a funciones de pérdida basadas en entropía cruzada.

2.4.3 Regresión

En una tarea de **regresión**, el objetivo no es asignar una clase, sino estimar una variable continua a partir de la entrada. En este caso, el target y es un valor real —o eventualmente un vector de valores reales— y la salida de la red se interpreta directamente como una aproximación numérica:

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta}).$$

A diferencia de lo que ocurre en clasificación, aquí no se busca modelar una distribución de probabilidad sobre clases excluyentes, sino aproximar una magnitud continua. Por ello, en la capa de salida suele emplearse una activación lineal, o incluso ninguna activación explícita, permitiendo que la red produzca valores en todo $\mathbb{R}^N$.

La calidad de la predicción se evalúa comparando el valor estimado $\hat{y}$ con el valor verdadero y . Una elección habitual es el error cuadrático medio, que penaliza desviaciones grandes de manera más intensa. De este modo, aunque la estructura interna de la red pueda ser similar a la utilizada en clasificación, la interpretación de la salida final y la función de pérdida asociada son diferentes.

En síntesis, clasificación y regresión comparten la misma idea general de aproximar una relación entre entradas y targets a través de una función parametrizada, pero difieren en la naturaleza de aquello que se predice. Esta distinción será importante en las secciones siguientes, ya que tanto la definición de la función de pérdida como el cálculo de gradientes por retropropagación dependen de cómo se interprete la salida del modelo.

2.4.4 Partición de datos: entrenamiento, validación y test

En aprendizaje supervisado, el conjunto de datos disponible no se utiliza de manera uniforme durante todo el proceso de desarrollo del modelo. Habitualmente se lo divide en tres subconjuntos con funciones diferentes: **conjunto de entrenamiento**, **conjunto de validación** y **conjunto de test**. Esta partición permite separar con claridad las etapas de ajuste de parámetros, selección de hiperparámetros y evaluación final del desempeño, evitando estimaciones sesgadas de la capacidad de generalización del modelo.

El **conjunto de entrenamiento** (*training set*) se utiliza para ajustar los parámetros de la red, es decir, los pesos y sesgos que se modifican mediante descenso por gradiente y retropropagación. En cada iteración, el modelo procesa ejemplos de este subconjunto, calcula la pérdida correspondiente y actualiza sus parámetros para reducirla. Por ello, el error medido sobre entrenamiento refleja principalmente qué tan bien la red logra adaptarse a los ejemplos que vio durante el proceso de aprendizaje.

El **conjunto de validación** (*validation set*) cumple una función diferente: no se utiliza para actualizar directamente los parámetros, sino para monitorear el comportamiento del modelo durante el desarrollo. Su papel es esencial en la selección de hiperparámetros —por ejemplo, tasa de aprendizaje, tamaño de la red, intensidad de regularización o probabilidad

de *dropout*— y también en decisiones como *early stopping*. En este sentido, la validación proporciona una estimación intermedia de la capacidad de generalización mientras el modelo todavía está siendo diseñado o ajustado.

Por último, el **conjunto de test** (*test set*) se reserva para la evaluación final del modelo una vez concluido todo el proceso de entrenamiento y selección. Este subconjunto no debe influir ni en el ajuste de parámetros ni en la elección de hiperparámetros, ya que su función es ofrecer una estimación lo más imparcial posible del desempeño sobre datos no vistos. En consecuencia, el resultado reportado sobre test se interpreta como la medida final de generalización del modelo.

Esta distinción de roles es central en aprendizaje automático. Mientras el conjunto de entrenamiento permite aprender, el conjunto de validación guía el diseño experimental y el conjunto de test estima el rendimiento final fuera de muestra. En las secciones siguientes, esta separación será importante para interpretar correctamente el entrenamiento, el sobreajuste y el efecto de las técnicas de regularización.

En particular, la comparación entre las métricas de entrenamiento y validación permite diagnosticar si la red está aprendiendo regularidades transferibles o si, por el contrario, comienza a ajustarse en exceso a peculiaridades del conjunto de entrenamiento. Por esta razón, la partición entre entrenamiento, validación y test no es solo una convención metodológica, sino una condición necesaria para discutir con precisión los conceptos de generalización y regularización que aparecerán más adelante.

2.5 Entrenamiento con retropropagación

Una vez fijada la interpretación de la salida de la red y definida una función de pérdida adecuada al problema, el paso siguiente consiste en ajustar los parámetros θ para que las predicciones del modelo se acerquen a los targets del conjunto de entrenamiento. Este ajuste se realiza mediante algoritmos de optimización basados en gradientes, y el procedimiento que permite calcular esos gradientes de manera eficiente en redes multicapa es la **retropropagación del error** (*backpropagation*).

Históricamente, los primeros perceptrones, como el propuesto por Rosenblatt [4], se entrenaban mediante la *regla del perceptrón*, que ajusta los pesos únicamente cuando la predicción es incorrecta. Aunque este procedimiento resulta adecuado en problemas linealmente separables, no se generaliza de manera directa a redes con capas ocultas y funciones de activación diferenciables.

El desarrollo que habilitó el entrenamiento sistemático de redes *feed-forward* fue el algoritmo de retropropagación del error, popularizado por Rumelhart et al. [6]. Si bien la utilización de gradientes en sistemas compuestos había sido explorada con anterioridad [14], la formulación de Rumelhart, Hinton y Williams consolidó la retropropagación como un método práctico y general para redes multicapa. Su idea central consiste en aplicar la **regla de la cadena** para calcular eficientemente cómo cambia la pérdida ante pequeñas variaciones de cada parámetro del modelo.

2.5.1 Gradientes por regla de la cadena.

Sea una red con parámetros θ , una entrada $\mathbf{x}$ y un target y —que puede representar una etiqueta discreta en clasificación o un valor continuo en regresión—, y sea $L(\theta; \mathbf{x}, y)$ una función de pérdida que cuantifica la discrepancia entre la predicción del modelo y ese valor verdadero. El objetivo del entrenamiento es ajustar θ para disminuir L , para lo cual se requieren las derivadas:

$$\frac{\partial L}{\partial \theta_i} \quad (2.5.1)$$

para cada parámetro $\theta_i \in \theta$. La retropropagación realiza este cómputo comenzando desde la capa de salida y propagando hacia atrás cantidades intermedias (errores locales) que, combinadas con las derivadas de las activaciones, permiten calcular gradientes en todas las capas con un costo computacional que escala linealmente con el número de parámetros.

2.5.2 Desvanecimiento y explosión del gradiente

En redes profundas, los gradientes pueden **desvanecerse** (tender a cero) o **explotar** (crecer sin control) al retropropagar a través de muchas capas. De forma esquemática, el gradiente con respecto a parámetros en capas tempranas involucra productos repetidos de términos del tipo “pesos $\times$ derivadas de activación”. Cuando dichas derivadas son típicamente menores que uno (como ocurre en regiones saturadas de σ o $\tanh$), el producto puede decrecer exponencialmente con la profundidad, haciendo que las capas iniciales aprendan muy lentamente. Por el contrario, si los factores efectivos superan la unidad, el producto puede crecer rápidamente, produciendo inestabilidad numérica.

Este fenómeno explica parte de la dificultad histórica para entrenar redes profundas con activaciones saturantes, y motiva decisiones de diseño ampliamente adoptadas:

- **Activaciones no saturantes** en capas ocultas, como ReLU, que evita saturación en el semieje positivo.
- **Inicializaciones adecuadas** que controlan la varianza de activaciones y gradientes (por ejemplo, inicialización de Glorot/Xavier y de He) [15, 16].
- **Normalización por lotes** (*batch normalization*) para estabilizar distribuciones internas [17].
- **Conexiones residuales** que facilitan el flujo de gradiente en arquitecturas muy profundas [18].

En los experimentos presentados en esta tesis se utilizará **ReLU** como función de activación en las capas ocultas, junto con inicialización adecuada de pesos, dado su buen comportamiento práctico frente al desvanecimiento del gradiente en redes *feed-forward* modernas.

2.6 Funciones de pérdida

En el aprendizaje supervisado, la **función de pérdida** (*loss function*) cuantifica la discrepancia entre la predicción de la red y el target verdadero asociado a cada muestra. Si

la tarea es de clasificación, ese target y representa una etiqueta o clase; si la tarea es de regresión, representa un valor continuo a estimar. Su papel es fundamental, ya que traduce el desempeño del modelo en una magnitud escalar o vectorial que puede optimizarse durante el entrenamiento mediante retropropagación.

La elección de la función de pérdida no es independiente del tipo de salida de la red, sino que debe ser coherente con la naturaleza del problema. En clasificación, la salida suele interpretarse en términos probabilísticos y se utilizan habitualmente variantes de entropía cruzada; en regresión, en cambio, la salida representa directamente una cantidad numérica y son más naturales pérdidas como el error cuadrático medio.

2.6.1 Entropía cruzada

En tareas de clasificación, la función de pérdida más utilizada es la **entropía cruzada** (*cross-entropy loss*), que mide la distancia entre la distribución de probabilidad verdadera y la distribución predicha por la red.

En este contexto, el target verdadero puede representarse como una etiqueta entera y_i o, de manera equivalente, mediante una codificación indicadora y_{ic} de tipo *one-hot*, según la formulación adoptada.

Sea un conjunto de entrenamiento compuesto por N pares $(\mathbf{x}_i, y_i)$, donde $y_i \in \{1, 2, \dots, C\}$ denota la clase correcta de la muestra i , y C es el número total de clases. El modelo produce un vector de puntuaciones (*logits*) $\mathbf{z}_i = (z_{i1}, z_{i2}, \dots, z_{iC})$, que se transforma en probabilidades mediante la función *softmax*:

$$\hat{p}_{ic} = \frac{e^{z_{ic}}}{\sum_{k=1}^C e^{z_{ik}}} \quad (2.6.1)$$

La función de pérdida de entropía cruzada se define como:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{p}_{ic}) \quad (2.6.2)$$

donde y_{ic} es un vector indicador que vale 1 si la muestra i pertenece a la clase c y 0 en caso contrario. En el caso de clasificación binaria se suele parametrizar la salida con un único logit z_i y definir la probabilidad positiva como $\hat{p}_i = \sigma(z_i)$, donde

$$\sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

Al sustituir esta relación en la entropía cruzada, se obtiene la expresión simplificada para entropía cruzada binaria:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (2.6.3)$$

Esta formulación penaliza predicciones alejadas de la clase correcta e incentiva distribuciones más seguras cuando el modelo tiene suficiente evidencia. Minimizar la entropía cruzada equivale a maximizar la verosimilitud del modelo, dado que ambas expresiones son equivalentes bajo el logaritmo de la probabilidad condicional de las etiquetas.

2.6.1.1 Gradientes

Una propiedad fundamental de la entropía cruzada combinada con la activación *softmax* es la simplicidad de su gradiente con respecto a los logits:

$$\frac{\partial L}{\partial z_{ic}} = \hat{p}_{ic} - y_{ic} \quad (2.6.4)$$

Esta forma evita calcular explícitamente la derivada del *softmax*, haciendo que el entrenamiento sea eficiente y numéricamente estable. Por ello, la mayoría de las librerías modernas implementan una versión combinada de ambas operaciones, como en *PyTorch*:

```
criterion = nn.CrossEntropyLoss()
```

Esta función recibe como argumento:

- Los logits producidos por la red, de tamaño (N, C) ,
- Las etiquetas verdaderas como enteros de tamaño N , donde cada valor indica la clase correcta.

El modelo no debe aplicar una capa *softmax* explícitamente, ya que la función `CrossEntropyLoss` la incorpora internamente.

2.6.1.2 Calibración posterior mediante *temperature scaling*

En clasificación, la *accuracy* no siempre refleja la calidad probabilística del modelo: dos redes pueden tener la misma precisión y, sin embargo, diferir de forma importante en *cross-entropy loss* por problemas de calibración.

Sea $\mathbf{z}(x)$ el vector de logits de un modelo ya entrenado. En *temperature scaling* no se cambian los pesos de la red: solo se modifica la *softmax* introduciendo un hiperparámetro escalar T que reescala los logits antes de normalizar. La distribución calibrada queda definida por

$$p_T(y = c \mid x) = \frac{\exp(z_c(x)/T)}{\sum_{k=1}^C \exp(z_k(x)/T)}, \quad T > 0$$

Para $T > 1$ las probabilidades se suavizan (menos confianza), mientras que para $0 < T < 1$ se vuelven más picudas (más confianza).

Para clasificación multiclase con decisión por $\arg\max$, este reescalamiento no cambia la clase predicha (porque dividir todos los logits por el mismo $T > 0$ preserva su orden relativo), por lo que la *accuracy*/precisión permanece invariante. Lo que sí cambia es la confianza probabilística de las salidas y, en consecuencia, la *cross-entropy loss*.

Procedimiento práctico.

1. Entrenar el modelo normalmente en el conjunto de entrenamiento.
2. Una vez finalizado el entrenamiento, congelar todos los pesos del modelo.
3. Ajustar únicamente el escalar de temperatura T usando el conjunto de validación.

4. Buscar el valor T^* que minimiza la entropía cruzada sobre validación:

$$T^* = \arg \min_{T>0} \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} -\log p_T(y_i | x_i)$$

5. Evaluar en test usando logits reescalados $\mathbf{z}/T^*$.

Este procedimiento funciona como prueba diagnóstica: si la diferencia de *loss* entre dos modelos se reduce significativamente tras calibrar, parte de la brecha se debía a descalibración probabilística; si la brecha persiste, la diferencia refleja mejor una disparidad real de modelo. El trabajo de Guo et al. [19] reporta que *temperature scaling* es el método *post-hoc* más efectivo y estable entre los evaluados para calibrar redes neuronales modernas. Además, Hinton et al. [20] señalan que una calibración deficiente puede dificultar la transferencia de conocimiento en distilación (*difficult distillation*), reforzando la importancia de controlar la temperatura de las salidas suaves.

2.6.2 Error cuadrático medio (Mean Squared Error, MSE)

Una de las funciones de pérdida más antiguas y clásicas en el aprendizaje automático es el **error cuadrático medio** (*Mean Squared Error*, MSE). Es ampliamente utilizada en tareas de **regresión**, aunque históricamente también se aplicó en redes neuronales tempranas de clasificación.

Dada una predicción $\hat{y}_i$ y una etiqueta verdadera y_i , el MSE se define como:

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (2.6.5)$$

Esta función penaliza los errores cuadráticamente, haciendo que las predicciones incorrectas con grandes desviaciones contribuyan más a la pérdida.

Aunque el MSE puede utilizarse en clasificación (tratando las salidas como probabilidades), su comportamiento suele ser inferior a la entropía cruzada, ya que los gradientes pueden volverse muy pequeños cuando la salida está saturada, dificultando el aprendizaje. Por ello, en la mayoría de las redes modernas de clasificación se prefiere la entropía cruzada, que produce gradientes más informativos y una interpretación probabilística coherente.

2.7 Algoritmos de optimización

El ajuste de parámetros en redes neuronales *feed-forward* constituye un problema de optimización no convexa de alta dimensión. Los algoritmos de optimización buscan minimizar una función de pérdida o *loss function* $L(\boldsymbol{\theta})$, donde $\boldsymbol{\theta}$ representa el conjunto de parámetros de la red. En general, este proceso puede formalizarse como:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) \quad (2.7.1)$$

donde $L : \mathbb{R}^n \rightarrow \mathbb{R}$ es típicamente diferenciable, permitiendo el uso de métodos basados en gradientes.

2.7.1 Descenso de gradiente clásico

El método más simple es el **descenso de gradiente clásico**, el cual actualiza los parámetros en la dirección opuesta al gradiente del error:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t) \quad (2.7.2)$$

donde $\eta > 0$ es la *tasa de aprendizaje* (*learning rate*), que controla la magnitud de cada paso de actualización.

Aunque este método es eficiente en problemas de pequeña escala, su uso directo en grandes volúmenes de datos es computacionalmente costoso, ya que requiere calcular el gradiente completo sobre todo el conjunto de entrenamiento en cada iteración:

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i) \quad (2.7.3)$$

donde N es el número total de muestras, $\ell(\cdot, \cdot)$ es la pérdida individual, y $f(\cdot; \boldsymbol{\theta})$ representa el modelo.

2.7.2 Descenso de gradiente estocástico (SGD)

Para mitigar este problema, se introduce el **descenso de gradiente estocástico** (*Stochastic Gradient Descent*, SGD) [21], que estima el gradiente utilizando un subconjunto aleatorio de muestras (minibatch) $\mathcal{B}_t \subset \{1, \dots, N\}$:

$$\nabla_{\boldsymbol{\theta}} L_{\mathcal{B}_t}(\boldsymbol{\theta}_t) = \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell(f(\mathbf{x}_i; \boldsymbol{\theta}_t), y_i) \quad (2.7.4)$$

Así, la actualización se convierte en:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L_{\mathcal{B}_t}(\boldsymbol{\theta}_t) \quad (2.7.5)$$

Este enfoque reduce el coste computacional por iteración y, además, el ruido introducido por la muestra aleatoria puede ayudar a escapar de mínimos locales o zonas planas (*plateaus*) en la superficie de error, contribuyendo a una mejor generalización.

2.7.3 Métodos adaptativos

A partir del SGD se desarrollaron algoritmos que ajustan dinámicamente la tasa de aprendizaje para cada parámetro, denominados *métodos adaptativos*. Estos métodos buscan adaptar η en función de la magnitud histórica de los gradientes.

2.7.3.1 AdaGrad

El algoritmo **AdaGrad** [22] acumula los cuadrados de los gradientes para cada dimensión:

$$G_t = G_{t-1} + \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t) \odot \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t), \quad (2.7.6)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t) \quad (2.7.7)$$

donde $\odot$ denota el producto elemento a elemento y ϵ es un valor pequeño para evitar divisiones por cero. Este mecanismo penaliza las actualizaciones de parámetros que han recibido gradientes grandes en el pasado, promoviendo una convergencia más estable. Sin embargo, su acumulación monótona puede hacer que la tasa de aprendizaje decrezca demasiado rápido.

2.7.3.2 RMSProp

El algoritmo **RMSProp** [23] corrige esta limitación mediante un promedio móvil exponencial:

$$v_t = \rho v_{t-1} + (1 - \rho) \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t)^2, \quad (2.7.8)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t) \quad (2.7.9)$$

donde $\rho \in [0, 1)$ controla el decaimiento exponencial. Este esquema suaviza las oscilaciones del gradiente y estabiliza el aprendizaje, especialmente útil en escenarios con gradientes altamente ruidosos.

2.7.3.3 Adam

El optimizador **Adam** (*Adaptive Moment Estimation*) [24] combina las ideas de momento e información adaptativa de segundo orden. Define dos estimadores de momentos:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L_{\mathcal{B}_t}(\theta_t), \quad (2.7.10)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L_{\mathcal{B}_t}(\theta_t))^2 \quad (2.7.11)$$

donde m_t y v_t son los momentos de primer y segundo orden, respectivamente. Para corregir el sesgo inicial, se aplican correcciones:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (2.7.12)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (2.7.13)$$

y la actualización final es:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t \quad (2.7.14)$$

Adam presenta una convergencia rápida y estable en las primeras fases del entrenamiento, siendo particularmente eficaz en problemas con gradientes dispersos o de magnitud variable.

2.7.4 Entrenamiento desde inicialización aleatoria y uso de *learning-rate schedules*

Cuando una red neuronal se entrena desde cero, sus parámetros se inicializan de manera aleatoria y todavía no codifican regularidades útiles del problema. En esta etapa inicial, el proceso de optimización suele ser especialmente sensible a decisiones como la inicialización y la elección de la tasa de aprendizaje η [25].

Por esta razón, en lugar de mantener η fija durante todo el entrenamiento, es habitual utilizar un *learning-rate schedule*, es decir, una regla que modifica su valor a lo largo de las iteraciones. Esta práctica permite adaptar el tamaño de las actualizaciones a distintas etapas del aprendizaje: al comienzo suele ser importante avanzar con suficiente amplitud para organizar la representación interna del modelo, mientras que en etapas posteriores conviene realizar ajustes más pequeños y controlados [21, 25, 26].

En problemas de entrenamiento a gran escala, este tipo de esquemas también se utiliza para estabilizar las primeras iteraciones. Por ejemplo, Goyal et al. [26] muestran que, en entrenamiento con lotes grandes, una estrategia de *warmup* ayuda a evitar dificultades de optimización al inicio y facilita una evolución más estable de la red.

Además, en varios métodos de compresión y poda, el entrenamiento no se plantea como un proceso uniforme, sino en etapas. Una idea recurrente es permitir primero que el modelo aprenda una estructura útil y aplicar después mecanismos que favorezcan la eliminación de parámetros o canales menos relevantes. En esta línea, Han et al. [27] proponen un esquema de entrenar, podar y reentrenar; Zhu et al. [28] estudian poda gradual durante el entrenamiento; y Liu et al. [29] incorporan regularización de esparsidad para identificar canales prescindibles y podarlos posteriormente.

En términos prácticos, esto sugiere que no siempre resulta conveniente imponer desde el comienzo una regularización estructural fuerte. Durante las primeras etapas, muchas unidades todavía están ajustándose y su contribución no es fácil de evaluar. Por eso, una estrategia por fases puede resultar más estable: primero se deja que la red forme una organización inicial y, una vez alcanzado ese régimen, se introducen criterios de poda o atenuación más selectivos. Este enfoque suele ofrecer un mejor equilibrio entre compactación del modelo y conservación del desempeño [27-29].

2.8 Regularización en aprendizaje profundo

2.8.1 El papel de la regularización en redes neuronales profundas

Las redes neuronales *feed-forward* poseen una gran capacidad expresiva, lo que les permite aproximar relaciones altamente complejas a partir de los datos. Sin embargo, esa misma flexibilidad incrementa el riesgo de que el modelo ajuste no solo regularidades genuinas de la distribución subyacente, sino también fluctuaciones particulares del conjunto de entrenamiento. En este contexto, la regularización ocupa un lugar central: reúne un conjunto de estrategias destinadas a favorecer soluciones con mejor capacidad de generalización.

De manera general, las técnicas de regularización pueden actuar sobre distintos aspectos del proceso de aprendizaje. Algunas introducen restricciones explícitas sobre los parámetros

del modelo; otras modifican la dinámica del entrenamiento; y otras incorporan perturbaciones estocásticas que vuelven más robustas las representaciones internas aprendidas por la red. Antes de analizar técnicas específicas, conviene entonces precisar cuál es el problema que la regularización busca mitigar y por qué resulta especialmente relevante en modelos de alta capacidad como las redes profundas.

2.8.2 Sobreajuste (overfitting), generalización y compromiso sesgo-varianza

Para entender con precisión qué significa **regularizar**, primero es necesario definir el problema que se busca evitar: el **sobreajuste** (*overfitting*). Como se discutió previamente al introducir la partición entre conjuntos de entrenamiento, validación y test, el aprendizaje supervisado no se evalúa únicamente por su capacidad de reducir el error sobre los datos usados para ajustar parámetros, sino por su desempeño sobre muestras no vistas de la misma distribución.

En aprendizaje supervisado, los datos se asumen como muestras de una distribución subyacente desconocida. En la práctica, la separación entre entrenamiento, validación y test permite distinguir tres funciones complementarias: aprender los parámetros del modelo, monitorear y seleccionar configuraciones durante el desarrollo, y estimar finalmente el rendimiento fuera de muestra. Esta distinción metodológica es la que da contenido empírico a conceptos como *generalización* y *overfitting*.

Desde esta perspectiva, la definición más rigurosa de sobreajuste está asociada al desempeño fuera de muestra, especialmente en el conjunto de **test**: un modelo sobreajusta cuando, pese a obtener error muy bajo en entrenamiento, presenta error significativamente mayor en datos no vistos. Dicho de otro modo, aprende patrones idiosincráticos del entrenamiento (incluyendo ruido) que no se transfieren bien a nuevos ejemplos.

Conviene distinguir entonces roles: el *error y la pérdida de test* se utilizan para estimar desempeño final del modelo ya entrenado, mientras que el *error y la pérdida de validación* se usan durante el desarrollo como señal diagnóstica para seleccionar hiperparámetros, aplicar *early stopping* y detectar aparición de sobreajuste. Por eso, en la práctica cotidiana se identifica el inicio del sobreajuste cuando la métrica de validación empeora mientras la de entrenamiento sigue mejorando.

Dicho de otro modo, la existencia misma de un conjunto de validación separado permite detectar cuándo una mejora aparente del ajuste deja de traducirse en una mejora real de generalización. Esta observación será especialmente importante en esta tesis, ya que varios de los métodos estudiados —como *dropout*, *DropConnect* o la variante propuesta *BoostDropout*— se evalúan justamente por su capacidad de modificar esa relación entre desempeño en entrenamiento y desempeño fuera de muestra.

Un ejemplo intuitivo es la clasificación de imágenes de gatos y perros. Una red con capacidad muy alta (esto es, con suficiente flexibilidad o número de parámetros como para representar funciones muy complejas) puede memorizar el conjunto de entrenamiento sin construir una representación robusta de los rasgos que distinguen ambas clases. En consecuencia, al recibir imágenes nuevas puede fallar más de lo esperado. Una red con capacidad más acotada, o bien entrenada con regularización adecuada, se ve forzada a aprender patrones más estables y, por tanto, generaliza mejor.

Una forma complementaria de interpretar este fenómeno es el compromiso **sesgo-varianza**. Modelos de baja capacidad tienden a alto sesgo y subajuste (*underfitting*); modelos muy flexibles reducen sesgo pero pueden aumentar varianza, volviéndose sensibles a fluctuaciones de muestreo. El sobreajuste aparece justamente en ese régimen de alta varianza [9].

Para diagnosticarlo durante el entrenamiento, suele preferirse la curva de **pérdida de validación** (*validation loss*) antes que la de **error de validación**. La pérdida es más informativa porque es sensible al grado de confianza de las predicciones: puede empeorar aunque el **error de validación** permanezca casi constante. Por ello, un patrón típico de sobreajuste es que la **pérdida de entrenamiento** continúe descendiendo mientras la **pérdida de validación** deja de bajar y comienza a subir.

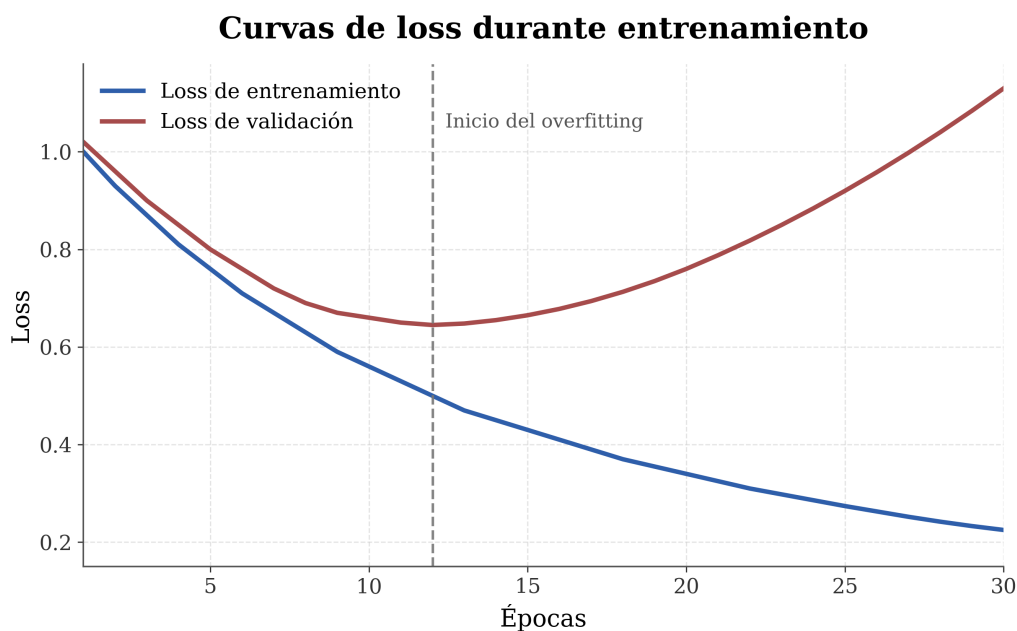


Figura 8: Esquema típico de sobreajuste: la pérdida de entrenamiento sigue disminuyendo mientras la pérdida de validación aumenta a partir de cierta época.

En este contexto, la regularización cumple un papel central. Siguiendo la definición de Goodfellow et al. [25], puede entenderse como cualquier modificación introducida en un algoritmo de aprendizaje con el objetivo de reducir el error de generalización, aun cuando ello no implique disminuir el error de entrenamiento. Desde esta perspectiva, la regularización no constituye una técnica única, sino una familia amplia de estrategias orientadas a controlar la complejidad efectiva del modelo y a promover soluciones más robustas frente a datos no vistos.

Entre estas estrategias se encuentran, por ejemplo, las penalizaciones sobre los parámetros, la detención temprana del entrenamiento (*early stopping*), la inyección de ruido y distintos mecanismos que alteran de manera controlada la propagación de la información dentro de la red. Esta diversidad de enfoques hace conveniente introducir, antes de estudiar métodos particulares, un criterio más preciso sobre cuáles de estas técnicas resultan más relevantes para el problema abordado en esta tesis.

2.8.3 Sparsity en representaciones y en parámetros

En este trabajo conviene distinguir dos nociones diferentes de *sparsity*, que traduciremos como **esparsidad**. La primera es la **esparsidad representacional** (*representational sparsity*), que describe cuántas neuronas permanecen activas ante un dato dado: una representación es más esparsa representacionalmente cuando, para cada entrada, solo un subconjunto pequeño de unidades presenta activaciones no nulas o significativamente altas. La segunda es la **esparsidad en la parametrización** (*parametrization sparsity*), que se refiere a la estructura de los parámetros del modelo: muchos pesos son exactamente cero o se fuerzan a ser despreciables, reduciendo la conectividad efectiva y la complejidad del modelo.

Ambas formas de esparsidad pueden contribuir a mejorar el desempeño fuera de muestra, pero actúan en niveles distintos. La esparsidad representacional organiza cómo fluye la información dentro de la red durante el *forward*; la esparsidad paramétrica controla la capacidad efectiva de la arquitectura mediante restricciones directas sobre los pesos. En términos prácticos, técnicas como *dropout* suelen impactar primero en la dinámica de activaciones, mientras que enfoques como poda de pesos o penalizaciones estructuradas apuntan explícitamente a la esparsidad de parámetros.

2.8.4 Criterio de selección de los métodos de regularización considerados

En el aprendizaje de redes neuronales *feed-forward* existen múltiples estrategias de regularización, y no todas actúan sobre el modelo del mismo modo. Algunas operan mediante restricciones directas sobre los parámetros, como las penalizaciones de norma; otras intervienen sobre el procedimiento de entrenamiento, como *early stopping*; y otras introducen perturbaciones que favorecen representaciones internas más estables y menos dependientes de configuraciones particulares de activación.

En esta tesis, el interés estará puesto especialmente en esta última familia de métodos: aquellos que, durante el entrenamiento, modifican de manera estocástica la actividad efectiva de la red. Se trata de enfoques particularmente relevantes para el problema estudiado aquí, ya que permiten analizar cómo la supresión parcial de unidades o conexiones afecta la propagación de la información, la robustez de las representaciones y la capacidad de generalización del modelo.

Desde esta perspectiva, *Dropout* y *DropConnect* constituyen dos casos paradigmáticos de regularización basada en enmascaramiento multiplicativo aleatorio. Aunque comparten una filosofía común, actúan sobre niveles distintos de la arquitectura: *Dropout* opera sobre las activaciones, mientras que *DropConnect* lo hace sobre las conexiones. Ambos métodos resultan especialmente relevantes para este trabajo no solo por su importancia histórica y práctica, sino también porque ofrecen un marco conceptual adecuado para estudiar variantes más generales o diseñadas *ad hoc*.

En esta tesis, dichos métodos clásicos serán tomados como punto de partida para analizar mecanismos de regularización estocástica más flexibles, en los que la perturbación introducida durante el entrenamiento no se limite necesariamente a la anulación de activaciones o conexiones. En particular, se explorará una variante propia que conserva la lógica de enmascaramiento multiplicativo, pero reemplaza la supresión binaria por una modulación controlada

de la amplitud de las activaciones. De este modo, el recorrido teórico desde *Dropout* y *Drop-Connect* hacia *BoostDropout* no debe entenderse como una ruptura, sino como una extensión natural dentro de la misma familia de métodos de regularización estocástica.

2.8.5 Dropout

Entre los métodos de regularización que introducen perturbaciones estocásticas sobre la actividad interna de la red, *Dropout* ocupa un lugar especialmente destacado por su simplicidad conceptual, su efectividad práctica y su amplia adopción en aprendizaje profundo. Introducido por Srivastava et al. [1], este método busca reducir el sobreajuste evitando que las neuronas desarrollen dependencias excesivamente específicas de otras unidades durante el entrenamiento.

La idea central de *Dropout* consiste en desactivar aleatoriamente una fracción de las activaciones en cada pasada *forward*. De este modo, la red no puede apoyarse sistemáticamente en un mismo subconjunto de neuronas y se ve forzada a distribuir la información de manera más robusta. Como resultado, favorece el aprendizaje de representaciones más reutilizables y con mejor capacidad de generalización frente a datos no vistos.

La motivación original del método se apoya en una analogía con la biología evolutiva. En los sistemas biológicos, la pérdida o inactivación de ciertos genes no suele provocar una disfunción total, ya que la existencia de rutas redundantes permite preservar la funcionalidad del organismo. Trasladada al contexto de las redes neuronales, esta idea sugiere que un modelo resulta más robusto cuando su desempeño no depende de manera crítica de unidades individuales ni de configuraciones muy específicas de interacción entre ellas.

En términos prácticos, *Dropout* opera poniendo a cero cada activación con una probabilidad p durante el entrenamiento. Según Srivastava et al. [1], este procedimiento dificulta la aparición de coadaptaciones complejas que pueden funcionar muy bien sobre el conjunto de entrenamiento, pero fallar al enfrentar ejemplos nuevos. Al introducir esta incertidumbre en cada iteración, el método induce a las neuronas a contribuir de manera útil en múltiples contextos, promoviendo detectores más simples y representaciones internas más estables.

Un aspecto interesante de Dropout es cómo se interpreta su efecto global. Srivastava et al. [1] argumentan que entrenar con dropout se asemeja a entrenar un **conjunto exponencialmente grande de modelos subredes** (cada una formada por un subconjunto distinto de neuronas activas). Durante la fase de entrenamiento, cada mini-batch actualiza parámetros que son compartidos entre muchas de estas subredes. En la fase de inferencia, cuando todas las neuronas están activas, se puede entender que la red resultante está aproximando una forma de **promedio de modelos** (model averaging). Este fenómeno es crucial, porque entrenar y promediar explícitamente tantos modelos sería inviable, pero Dropout lo logra de manera aproximada y eficiente.

No obstante, es importante ser críticos con esta interpretación. Aunque Dropout ciertamente introduce un efecto similar a un ensamble, no se trata de un promedio estricto sobre modelos independientes: las subredes no son entrenadas por separado, sino que comparten pesos, lo cual introduce correlaciones no presentes en un ensamble clásico. Además, en la práctica, la mejora de desempeño atribuida a Dropout no se debe únicamente a esta visión de “ensamble implícito”, sino también a la fuerte regularización que produce al romper coadaptaciones y obligar a que cada neurona aporte información útil incluso en ausencia de

otras. En ese sentido, se puede ver a Dropout tanto como una forma de ensamble eficiente como una penalización estructural que favorece la robustez de las representaciones internas.

Para ilustrar este efecto de manera directa, Srivastava et al. [1] comparan las características de primer nivel aprendidas por dos autoencoders idénticos sobre MNIST (256 unidades ocultas ReLU), uno sin dropout y otro con dropout en la capa oculta ($p = 0,5$). Ambos modelos logran errores de reconstrucción de test similares, pero las representaciones difieren de forma marcada: sin dropout predominan filtros ruidosos y fuertemente coadaptados, mientras que con dropout emergen detectores más interpretables (bordes, trazos y manchas), coherente con una menor coadaptación y mejor generalización. Las imágenes de la Figura 9 fueron extraídas de Srivastava et al. [1].

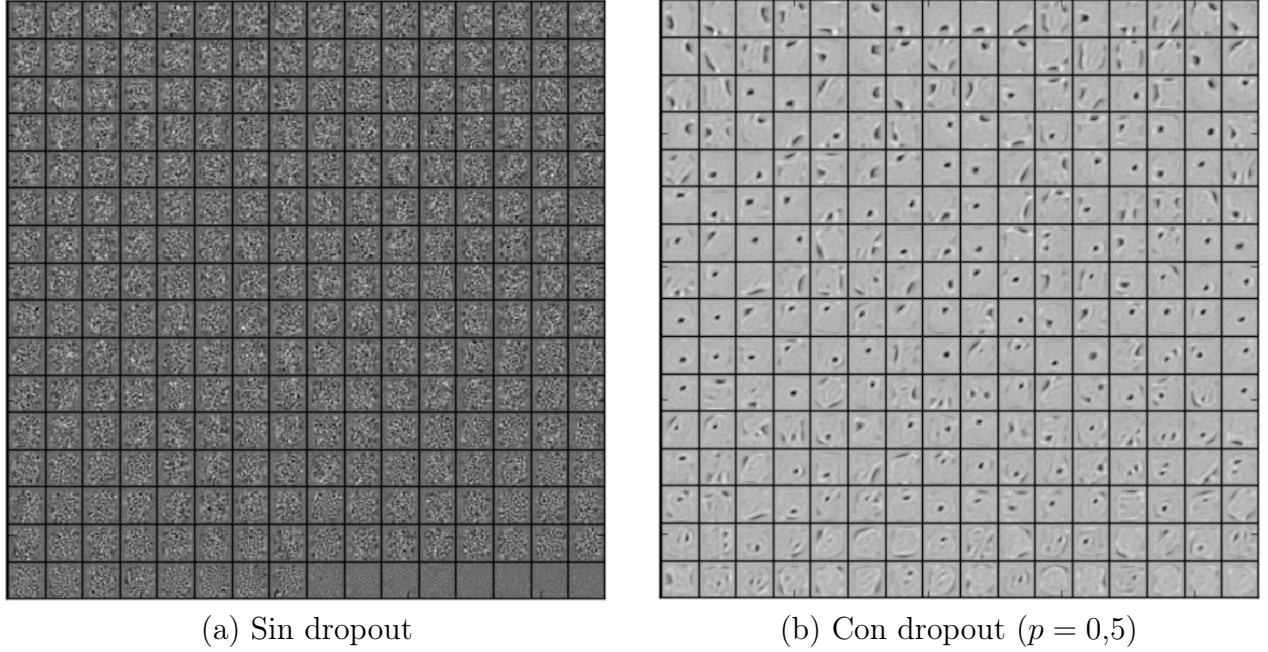


Figura 9: Características aprendidas en MNIST por autoencoders con una capa oculta de 256 unidades ReLU: comparación entre entrenamiento sin dropout y con dropout. Imágenes extraídas y adaptadas de Srivastava et al. [1].

Como efecto colateral relevante, Srivastava et al. [1] reportan que *dropout* induce **espar-sidad representacional** aun sin agregar regularizadores explícitos de escasez. Al evaluar autoencoders equivalentes con y sin dropout sobre mini-batches de test, observan que con dropout disminuye la activación media de las unidades ocultas y también la fracción de unidades con activaciones altas. En otras palabras, para cada ejemplo quedan menos neuronas fuertemente activas y la actividad promedio por unidad se mantiene más baja, coherente con representaciones internas más esparsas y menos coadaptadas.

2.8.5.1 Detalles de implementación: Dropout

Sea $h \in \mathbb{R}^d$ el vector de activaciones de una capa oculta y $z = Wh + b$ la entrada pre-activación de la capa siguiente. En entrenamiento, *dropout* introduce una máscara aleatoria

$$m \sim \text{Bernoulli}(q)^d$$

es decir, para cada componente m_i de la máscara,

$$\mathbb{P}(m_i = 1) = q, \quad \mathbb{P}(m_i = 0) = 1 - q = p, \quad m_i \in \{0, 1\}.$$

con probabilidad de retención $q = 1 - p$, y reemplaza h por

$$\tilde{h} = m \odot h$$

donde $\odot$ denota producto elemento a elemento.

2.8.5.2 Escalado en entrenamiento e inferencia.

Existen dos formas equivalentes de implementar dropout:

- **Dropout clásico:** en entrenamiento se usa $\tilde{h} = m \odot h$. Como $m_i \sim \text{Bernoulli}(q)$, se tiene

$$\mathbb{E}[\tilde{h}_i] = \mathbb{E}[m_i h_i] = q h_i$$

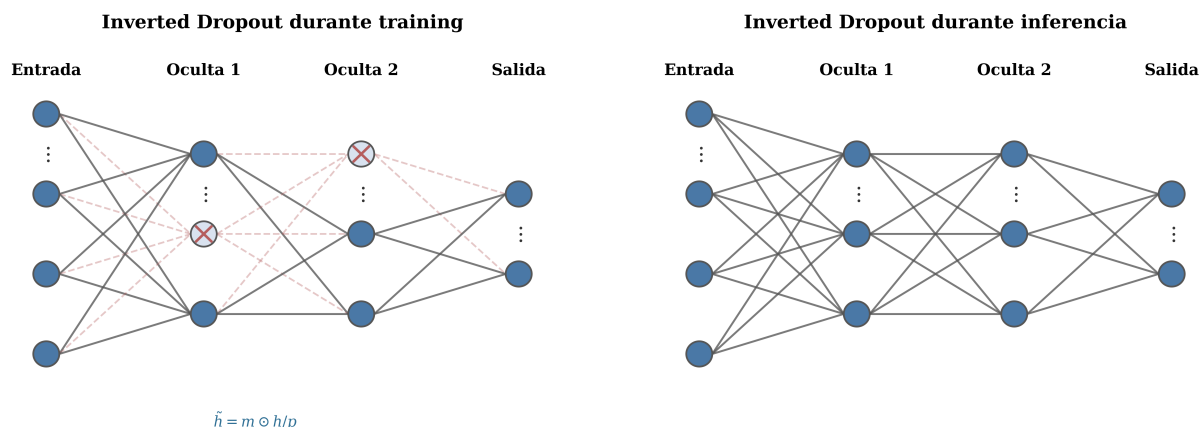
Es decir, durante entrenamiento la activación media que “ve” la capa siguiente queda reducida por un factor q . Para mantener en test la misma escala promedio de señales (y aproximar el promedio de subredes), en inferencia se usa qh en lugar de h (equivalentemente, se escalan por q los pesos salientes) [1].

- **Inverted dropout (práctica actual):** en entrenamiento se usa

$$\tilde{h} = \frac{m}{q} \odot h, \quad \Rightarrow \quad \mathbb{E}[\tilde{h}] = h$$

de modo que la capa siguiente recibe, en promedio, la misma magnitud de activaciones que recibiría sin dropout. Por eso, en inferencia no se aplica ni máscara ni escalado adicional.

Esta distinción entre fase de entrenamiento e inferencia se ilustra en la Figura 10, con detalle en las subfiguras 10a y 10b. En la formulación *inverted*, el factor de normalización $1/q$ (equivalentemente $1/p$ si se define p como probabilidad de retención) se aplica durante entrenamiento para conservar la escala esperada de activaciones, mientras que en inferencia se utiliza la red completa sin enmascaramiento estocástico.



(a) *Inverted dropout* durante entrenamiento: se aplica una máscara aleatoria sobre activaciones y se normaliza por el factor correspondiente para preservar la escala esperada.

(b) *Inverted dropout* durante inferencia: desaparece el enmascaramiento aleatorio y la propagación es completamente determinista.

Figura 10: Esquema conceptual de una red profunda con dos capas ocultas bajo *inverted dropout*: comparación entre entrenamiento e inferencia.

2.8.5.3 Interpretación y efecto regularizador.

Cada realización de m define una subred distinta, de modo que el entrenamiento comparte parámetros entre un conjunto exponencial de submodelos. Esta estrategia reduce coadaptaciones frágiles: una unidad no puede depender sistemáticamente de otras unidades específicas para corregir errores, porque su presencia es aleatoria en cada paso [1]. Además, el ruido multiplicativo inducido por m actúa como regularización efectiva sobre las activaciones intermedias.

2.8.5.4 Observaciones.

- **Implementación en PyTorch:** La capa `nn.Dropout` de PyTorch implementa **inverted dropout**.
- **Mini-batches:** En entrenamiento por mini-batches, la máscara de dropout se muestrea **independientemente para cada ejemplo** del batch (y para cada pasada forward). No se usa, en general, una única máscara compartida por todo el mini-batch. Esta independencia incrementa la diversidad del ruido regularizador y evita correlaciones artificiales entre ejemplos.

2.8.6 DropConnect

Una extensión natural de la lógica introducida por *Dropout* consiste en preguntarse qué ocurre si la perturbación aleatoria no se aplica sobre las activaciones de las neuronas, sino directamente sobre las conexiones entre ellas. Esa es precisamente la idea detrás de *DropConnect*, propuesto por Wan et al. [2]. En lugar de anular unidades completas durante el

entrenamiento, este método enmascara de manera estocástica pesos individuales, dando lugar a una forma más fina y localizada de regularización.

2.8.6.1 Formulación matemática

Considérese una capa completamente conectada con entrada $h \in \mathbb{R}^d$, matriz de pesos $W \in \mathbb{R}^{m \times d}$ y sesgo $b \in \mathbb{R}^m$. Sin regularización, la preactivación de la capa viene dada por

$$z = Wh + b,$$

En *DropConnect*, se introduce una máscara binaria $M \in \{0, 1\}^{m \times d}$, con entradas independientes

$$M_{ij} \sim \text{Bernoulli}(q),$$

donde $q = 1 - p$ es la probabilidad de retención. La matriz efectiva de pesos durante esa pasada *forward* es entonces

$$\tilde{W} = M \odot W,$$

y la salida de la capa pasa a ser

$$z = \tilde{W}h + b = (M \odot W)h + b$$

De este modo, cada peso W_{ij} puede estar activo o anulado en una iteración dada. A diferencia de *dropout*, donde una sola variable aleatoria decide si una activación completa sobrevive o no, aquí la aleatoriedad opera a nivel de cada conexión individual.

2.8.6.2 Intuición: diferencia con Dropout

Esta distinción no es menor. Si se aplica *dropout* a una activación h_j , entonces todas las contribuciones de esa neurona hacia la siguiente capa desaparecen simultáneamente. En cambio, con *DropConnect*, la activación h_j sigue existiendo, pero algunas de sus conexiones pueden estar presentes y otras no.

Por eso puede decirse que *DropConnect* introduce una forma de ruido estructural más localizada: no elimina unidades completas, sino que modifica la topología efectiva de la red en cada iteración. Como consecuencia, el conjunto de subredes implícitas que se generan durante el entrenamiento es aún más amplio que en *dropout*, ya que no solo varía qué neuronas participan, sino qué enlaces específicos quedan disponibles entre capas.

2.8.6.3 Interpretación como ensemble implícito

Al igual que *dropout*, *DropConnect* puede interpretarse como una forma aproximada de entrenar un conjunto muy grande de modelos. Cada máscara M define una realización distinta de la red, y el entrenamiento recorre implícitamente muchas de estas configuraciones compartiendo parámetros entre ellas.

Sin embargo, conviene remarcar que esta interpretación debe entenderse como una intuición útil y no como un ensemble clásico en sentido estricto. Las distintas subredes no son

independientes: comparten pesos y son entrenadas de manera acoplada. Por ello, el beneficio práctico de *DropConnect* no proviene solo de un supuesto promedio de modelos, sino también del efecto regularizador que produce al perturbar sistemáticamente la conectividad durante el aprendizaje.

2.8.6.4 Inferencia

Durante la inferencia no resulta conveniente seguir muestreando máscaras aleatorias, ya que eso volvería estocástica la predicción. En la práctica se reemplaza la matriz aleatoria $M \odot W$ por su valor esperado

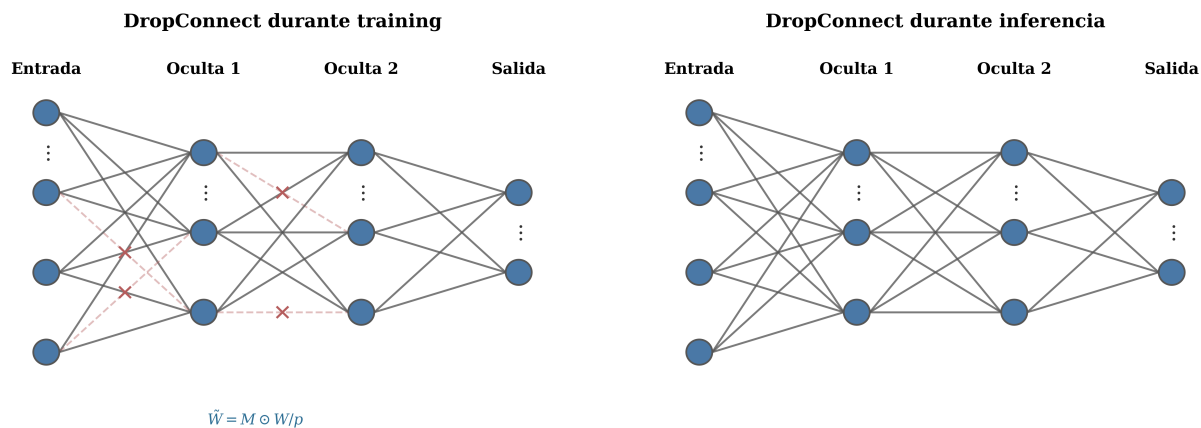
$$\mathbb{E}[M \odot W] = qW,$$

de modo que se utiliza la aproximación determinista

$$z \approx qWh + b.$$

Es importante notar que esta igualdad corresponde al valor esperado de los pesos enmascarados, y por lo tanto constituye una aproximación eficiente al comportamiento promedio del modelo estocástico. En redes con activaciones no lineales, este reemplazo no reproduce exactamente la esperanza de la salida final, pero suele funcionar bien en la práctica y reduce significativamente el costo computacional en test.

La Figura 11 resume esta lógica para *DropConnect*: durante entrenamiento (subfigura 11a) se muestrea una máscara aleatoria a nivel de conexiones, mientras que en inferencia (subfigura 11b) se utiliza una versión determinista basada en el valor esperado de los pesos efectivos.



(a) *DropConnect* durante entrenamiento: la máscara estocástica se aplica sobre conexiones individuales de la red.

(b) *DropConnect* durante inferencia: no se muestrea máscara y se emplea una aproximación determinista del modelo promedio.

Figura 11: Esquema conceptual de una red profunda con dos capas ocultas bajo *DropConnect*: comparación entre entrenamiento e inferencia.

2.8.6.5 Efecto regularizador

Desde el punto de vista conceptual, *DropConnect* puede entenderse como una regularización que actúa directamente sobre la parametrización efectiva del modelo. Mientras que *dropout* perturba la representación intermedia al anular activaciones, *DropConnect* perturba la conectividad misma de la red. Esto tiende a desalentar soluciones que dependan demasiado de pesos individuales o de caminos de propagación demasiado específicos.

En arquitecturas densas, este mecanismo puede resultar particularmente atractivo, ya que obliga a repartir la contribución predictiva entre muchas conexiones posibles. No obstante, esa misma granularidad vuelve a *DropConnect* potencialmente más costoso que *dropout*, dado que la máscara debe generarse a nivel de cada peso y no solo a nivel de cada unidad.

2.8.6.6 Comparación conceptual con Dropout

En síntesis, ambos métodos comparten la misma filosofía general de inyectar ruido multiplicativo durante el entrenamiento para mejorar la generalización, pero lo hacen en niveles distintos:

- **Dropout** actúa sobre activaciones: decide qué neuronas participan en cada pasada
- **DropConnect** actúa sobre pesos: decide qué conexiones específicas permanecen activas

Por ello, puede decirse que *DropConnect* es una generalización natural de *dropout*: en lugar de suprimir nodos completos, permite una dilución más localizada de la conectividad.

2.8.6.7 Evidencia empírica y comparación con Dropout

En el trabajo original, Wan et al. [2] no presentan a *DropConnect* como un reemplazo universal de *dropout*, sino como una alternativa que en varios escenarios logra una regularización competitiva o superior en capas completamente conectadas. Sus experimentos comparan redes sin este tipo de regularización (*No-Drop*), redes con *dropout* y redes con *DropConnect*, en tareas de clasificación de imágenes y bajo distintos esquemas de arquitectura y aumento de datos.

En MNIST, usando una red con dos capas densas de 800 neuronas, los resultados dependen de la función de activación. Con ReLU, *DropConnect* obtiene menor error que *dropout* (1.20 % frente a 1.28 % en modelo individual y 1.12 % frente a 1.20 % con *voting*). Con activaciones *tanh* también mejora (1.36 % frente a 1.58 %). Con sigmoides ocurre lo contrario y *dropout* resulta mejor (1.38 % frente a 1.55 %). En ese mismo conjunto, los autores muestran además que, al aumentar el tamaño del modelo, *No-Drop* tiende a sobreajustar mientras que *dropout* y *DropConnect* atenúan ese efecto, y señalan que *DropConnect* converge más lentamente aunque termina con menor error final en ese experimento.

Cuando en MNIST se utiliza un extractor convolucional más potente y se combinan múltiples redes, ambos métodos vuelven a quedar cercanos, pero *DropConnect* conserva ventajas en parte de los protocolos. Sin aumento de datos, *dropout* logra menor error en modelo individual (0.59 % frente a 0.63 %), mientras que con *voting* *DropConnect* obtiene menor error final (0.52 % frente a 0.57 %). Con recortes aleatorios, ambos empatan en error individual

(0.39 %), pero *DropConnect* mejora con *voting* (0.32 % frente a 0.35 %). Con recortes y escalado/rotación, ambos alcanzan 0.28 % por modelo, pero *DropConnect* llega a 0.21 % con *voting*, por debajo de *dropout* (0.27 %).

En CIFAR-10 (un benchmark de clasificación de imágenes en color de objetos, con 10 clases), los experimentos reportados también favorecen a *DropConnect*. Con un extractor convolucional simple y sin aumento de datos, *DropConnect* obtiene 18.7 % de error frente a 19.7 % de *dropout*. Con un extractor más grande, el error por modelo individual es 11.10 % para *DropConnect* y 11.52 % para *dropout*. Además, al combinar varias redes, *DropConnect* alcanza 9.41 % con 5 modelos y 9.32 % con 12 modelos.

En SVHN (*Street View House Numbers*, un conjunto de imágenes de dígitos extraídos de números de casas), las diferencias son pequeñas. Los autores remarcan que, por el gran tamaño del conjunto de entrenamiento, tanto *dropout* como *DropConnect* quedan muy cerca de *No-Drop*. Aun así, *DropConnect* muestra una ventaja marginal en modelo individual (2.23 % frente a 2.25 %) y empata prácticamente en *voting* (1.94 % frente a 1.96 %).

En NORB (un conjunto de imágenes de objetos tridimensionales tomados desde distintos ángulos e iluminaciones) aparece el caso inverso: *dropout* supera levemente a *DropConnect* (3.96 % frente a 4.14 % por modelo individual, y 3.03 % frente a 3.23 % con *voting*). Los autores señalan explícitamente que, en ese benchmark, ambos métodos mejoran sobre *No-Drop*, pero *dropout* resulta algo mejor.

En conjunto, la evidencia experimental del artículo sugiere una lectura prudente: *DropConnect* supera a *dropout* en varios experimentos y empata en otros, pero no de forma uniforme. Por ello, una formulación fiel al paper es afirmar que *DropConnect a menudo* mejora a *dropout*, y que el resultado depende de la arquitectura, la activación, el conjunto de datos y el protocolo experimental empleado.

Los dos métodos discutidos hasta aquí comparten un rasgo central: durante el entrenamiento introducen una perturbación multiplicativa aleatoria que altera la señal propagada por la red y, con ello, su dinámica de aprendizaje. Esta observación sugiere una generalización natural: en lugar de restringir la perturbación a una decisión binaria de retener o anular unidades/conexiones, puede considerarse una modulación estocástica de la magnitud de las activaciones. Bajo esta idea se introduce, en la sección siguiente, el mecanismo propuesto en este trabajo.

2.9 Modelo propuesto: *BoostDropout*

El método *BoostDropout* se introduce en este trabajo como una variante de regularización estocástica inspirada en la familia de métodos discutida en las secciones anteriores. Mientras que en *Dropout* la perturbación consiste en anular activaciones y en *DropConnect* en anular conexiones individuales, en *BoostDropout* la idea es diferente: la señal no se suprime necesariamente, sino que puede ser modificada multiplicativamente mediante un factor aleatorio de amplificación o atenuación.

Desde un punto de vista conceptual, este mecanismo puede interpretarse como una extensión del enmascaramiento multiplicativo clásico. La diferencia esencial es que la máscara ya no actúa de manera binaria sobre la información propagada, sino que introduce una modulación controlada de su amplitud. Esto permite estudiar una familia más amplia de perturbaciones

estocásticas sobre las activaciones internas de la red, manteniendo el espíritu regularizador de *Dropout*, pero habilitando grados intermedios entre “conservar” y “anular” la señal.

En el contexto de esta tesis, *BoostDropout* será considerado como un ejemplo de método de regularización *custom*, diseñado para explorar de qué manera una perturbación multiplicativa no binaria puede afectar la dinámica de entrenamiento, la estabilidad de las representaciones internas y la capacidad de generalización del modelo.

2.9.1 Definición formal de la capa *BoostDropout*

Sea $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ el vector de activaciones de la capa l . Para cada componente j , se define una variable aleatoria multiplicativa

$$m_j^{(l)} = \begin{cases} 1 + \lambda, & \text{con probabilidad } p, \\ 1, & \text{con probabilidad } 1 - p, \end{cases} \quad (2.9.1)$$

donde $p \in [0, 1]$ controla la frecuencia de modulación y $\lambda \in \mathbb{R}$ su intensidad. La salida de la capa regularizada queda dada por

$$\tilde{h}_j^{(l)} = m_j^{(l)} h_j^{(l)}, \quad j = 1, \dots, d_l. \quad (2.9.2)$$

En forma vectorial, esto equivale a

$$\tilde{\mathbf{h}}^{(l)} = \mathbf{m}^{(l)} \odot \mathbf{h}^{(l)}. \quad (2.9.3)$$

El valor esperado de la máscara es

$$\mathbb{E}[m_j^{(l)}] = p(1 + \lambda) + (1 - p) = 1 + p\lambda. \quad (2.9.4)$$

Por lo tanto, cuando $\lambda \neq 0$, la perturbación modifica en promedio la escala de activación. Si se desea preservar la media durante entrenamiento (análogo al principio de *inverted dropout*), puede usarse una máscara normalizada

$$\hat{m}_j^{(l)} = \frac{m_j^{(l)}}{1 + p\lambda}, \quad \mathbb{E}[\hat{m}_j^{(l)}] = 1. \quad (2.9.5)$$

En ese caso, la activación regularizada pasa a ser

$$\tilde{h}_j^{(l)} = \hat{m}_j^{(l)} h_j^{(l)}. \quad (2.9.6)$$

Esta normalización es opcional y su conveniencia depende del objetivo experimental: mantener la escala esperada estable o permitir un desplazamiento medio controlado por $p\lambda$.

Nótese además que, si se toma $\lambda = -1$, la variable $m_j^{(l)}$ pasa a ser Bernoulli sobre $\{0, 1\}$ con probabilidad de retención $1 - p$; al aplicar la normalización de la ecuación (2.9.5), se recupera exactamente la forma de *inverted dropout*.

La Figura 12 ilustra este mecanismo en una red neuronal con dos capas ocultas, destacando que en *BoostDropout* las activaciones no se anulan necesariamente, sino que se modulan multiplicativamente durante entrenamiento. En esta formulación, la máscara puede denotarse

como m o como $\hat{m}$ si se emplea la versión normalizada; dicha normalización es opcional, tal como se discutió antes, y su conveniencia depende del objetivo experimental.

BoostDropout durante training

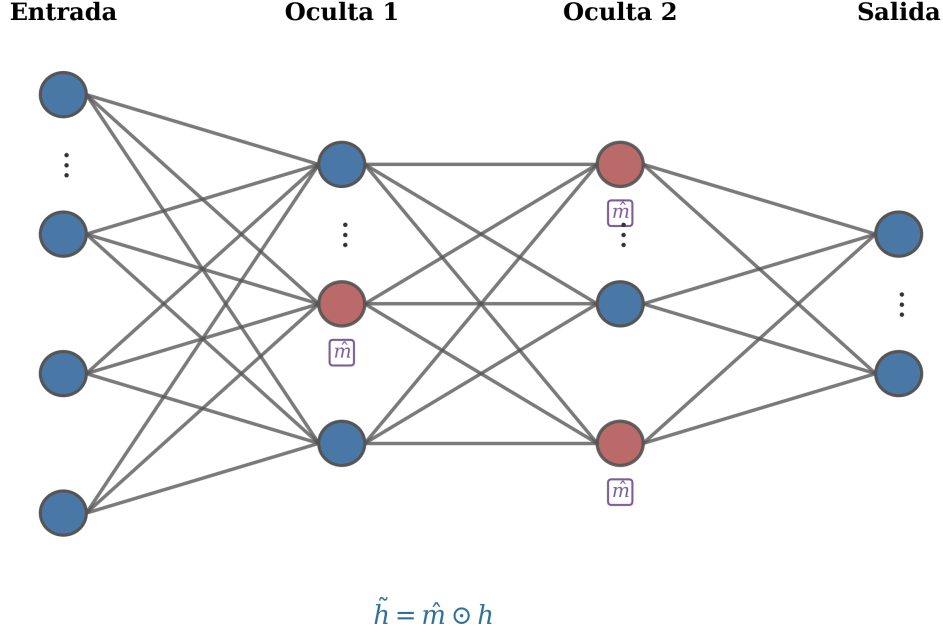


Figura 12: Esquema ilustrativo de una red neuronal con dos capas ocultas regularizada con *BoostDropout* durante entrenamiento. A diferencia de *Dropout*, las activaciones destacadas no se eliminan necesariamente, sino que se modulan multiplicativamente mediante una máscara aleatoria. Si se utiliza la versión normalizada, la máscara puede expresarse como $\hat{m} = m/(1 + p\lambda)$ para preservar la escala esperada; esta normalización es opcional y depende del objetivo experimental.

2.9.1.1 Interpretación conceptual

A diferencia de *Dropout*, donde una fracción de activaciones se anula, *BoostDropout* introduce una perturbación multiplicativa no binaria que conserva la señal base y la modula estocásticamente. En consecuencia, el mecanismo explora una región intermedia entre “dejar pasar” y “apagar”, y permite estudiar cómo variaciones de amplitud (en lugar de supresiones estrictas) impactan en la robustez de las representaciones internas.

Desde una perspectiva de regularización, este esquema desalienta dependencias excesivamente rígidas de magnitud en activaciones particulares y promueve soluciones más estables frente a fluctuaciones multiplicativas durante entrenamiento.

2.10 Autoencoders

Aunque el eje de esta tesis está puesto en arquitecturas *feed-forward* supervisadas y en mecanismos de regularización estocástica, resulta conveniente introducir brevemente la noción de *autoencoder*, ya que este tipo de modelo aparece con frecuencia en la literatura como herramienta para estudiar representaciones internas, reconstrucción de datos y efectos de regularización [25, 30].

De manera general, un autoencoder es una red neuronal entrenada para reproducir su propia entrada en la salida. A diferencia de un clasificador, cuyo objetivo es predecir una etiqueta, el autoencoder busca aprender una representación interna de los datos que conserve la información esencial necesaria para reconstruir la muestra original. Esta idea lo convierte en una arquitectura especialmente útil para estudiar codificaciones latentes, compresión y aprendizaje de características.

La estructura general de un autoencoder puede visualizarse en la Figura 13. En ella se distinguen el codificador, el espacio latente y el decodificador, organizados en una arquitectura simétrica de tipo *feed-forward*.

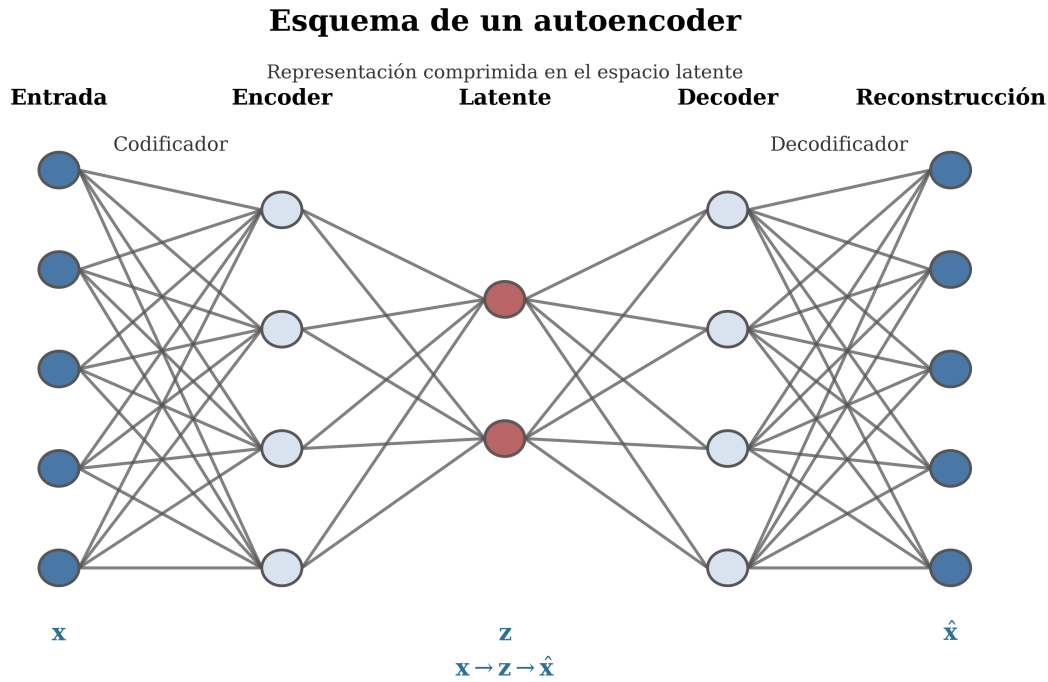


Figura 13: Esquema ilustrativo de un autoencoder. El codificador transforma la entrada $\mathbf{x}$ en una representación latente $\mathbf{z}$ de menor dimensión o más estructurada, mientras que el decodificador utiliza dicha representación para producir una reconstrucción $\hat{\mathbf{x}}$ de la muestra original.

2.10.1 Definición general

Sea $\mathbf{x} \in \mathbb{R}^n$ una muestra de entrada. El codificador transforma dicha entrada en una representación latente $\mathbf{z}$ mediante una aplicación del tipo

$$\mathbf{z} = \vec{f}_{\text{enc}}(W_{\text{enc}}^{\top} \mathbf{x} + \mathbf{b}_{\text{enc}}) \quad (2.10.1)$$

donde $\mathbf{z} \in \mathbb{R}^k$ representa el código latente, con frecuencia de dimensión menor que la entrada original.

A partir de esa representación, el decodificador genera una reconstrucción de la muestra inicial según

$$\hat{\mathbf{x}} = \vec{g}_{\text{dec}}(W_{\text{dec}}^{\top} \mathbf{z} + \mathbf{b}_{\text{dec}}) \quad (2.10.2)$$

donde $\hat{\mathbf{x}}$ denota la reconstrucción producida por la red.

El entrenamiento del autoencoder consiste en ajustar sus parámetros de manera que la reconstrucción sea lo más parecida posible a la entrada original. Para ello se utiliza una función de pérdida de reconstrucción, que en el caso continuo puede escribirse como

$$\mathcal{L}_{\text{rec}}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{n} \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 \quad (2.10.3)$$

aunque en otros contextos también pueden emplearse funciones de entropía cruzada u otras medidas adecuadas al tipo de dato considerado [25].

2.10.2 Componentes fundamentales

Desde un punto de vista estructural, un autoencoder puede descomponerse en los siguientes componentes:

- **Entrada $\mathbf{x}$:** corresponde a la muestra original que se desea codificar y reconstruir.
- **Codificador:** es la parte de la red que transforma la entrada en una representación interna más compacta o más abstracta. Su función principal es extraer regularidades relevantes de los datos y descartar, en la medida de lo posible, redundancias o variaciones irrelevantes.
- **Espacio latente $\mathbf{z}$:** constituye el cuello de botella de la arquitectura. En esta representación intermedia se condensa la información necesaria para reconstruir la entrada. Cuando la dimensión de $\mathbf{z}$ es menor que la de $\mathbf{x}$, el modelo se ve forzado a aprender una compresión efectiva de los datos.
- **Decodificador:** recibe el código latente y produce una aproximación de la muestra original. Su tarea no es clasificar ni decidir, sino reconstruir a partir de la información contenida en el espacio latente.
- **Pérdida de reconstrucción:** cuantifica la discrepancia entre la entrada original y su reconstrucción. Esta función guía el proceso de aprendizaje y determina qué tipo de similitud entre $\mathbf{x}$ y $\hat{\mathbf{x}}$ se considera relevante.

2.10.3 Usos principales

Los autoencoders poseen diversos usos dentro del aprendizaje automático profundo. Uno de los más conocidos es la **reducción de dimensionalidad**, ya que la representación latente puede capturar estructuras relevantes de los datos en un espacio de menor dimensión, de manera análoga pero no necesariamente lineal respecto de técnicas clásicas como PCA [30].

Otro uso importante es el **aprendizaje de representaciones**. Al entrenarse para reconstruir la entrada, el modelo desarrolla variables internas que pueden interpretarse como características útiles para tareas posteriores, ya sea supervisadas o no supervisadas. En este sentido, los autoencoders han sido empleados como mecanismos de preentrenamiento, extracción de características y análisis de activaciones internas [25].

También pueden utilizarse como herramientas de **compresión** y de **detección de anomalías**. En el primer caso, la idea es representar cada muestra a través de su código latente. En el segundo, una muestra anómala suele reconstruirse peor que una muestra similar a aquellas observadas durante entrenamiento, lo que permite usar el error de reconstrucción como señal de rareza o desviación respecto del patrón esperado.

En el contexto de esta tesis, los autoencoders resultan particularmente relevantes porque ofrecen un marco natural para estudiar cómo distintos mecanismos de regularización modifican la calidad de las representaciones internas. Por esa razón, constituyen un ejemplo útil cuando se analizan efectos como la esparsidad de activaciones, la robustez de las características aprendidas o la aparición de coadaptaciones entre neuronas. Además, como se verá en el Capítulo 3, se los incorpora no como tarea principal de comparación cuantitativa, sino como un protocolo complementario y controlado para inspeccionar cualitativamente las representaciones que producen *Dropout*, *DropConnect* y *BoostDropout*.

Capítulo 3. Detalles de implementación y diseño experimental

3.1 Diseño experimental y criterios de implementación

En esta parte se describe el protocolo experimental utilizado para comparar las arquitecturas consideradas en este trabajo. El objetivo de esta etapa no fue construir un benchmark exhaustivo ni agotar el espacio posible de configuraciones, sino diseñar un esquema de experimentación controlado, reproducible y metodológicamente consistente que permitiera estudiar, en condiciones comparables, el efecto de distintos mecanismos de regularización sobre redes neuronales *feed-forward*.

Desde el punto de vista técnico, una decisión central del trabajo fue priorizar la trazabilidad completa de cada corrida, la comparabilidad justa entre arquitecturas y la separación estricta entre las etapas de ajuste, selección y evaluación. Además, el pipeline experimental se estructuró en dos fases bien diferenciadas: una etapa inicial de exploración de hiperparámetros y una etapa posterior de evaluación final.

Conviene además distinguir entre el diseño experimental propiamente dicho y la herramienta computacional utilizada para implementarlo. La notebook de Jupyter y el repositorio asociado no constituyen en sí mismos el experimento, sino el soporte técnico que permitió ejecutar de manera sistemática las corridas, conservar artefactos intermedios, registrar metadatos y reconstruir posteriormente cada resultado. Con este objetivo, se desarrolló un repositorio específico para la organización del flujo experimental, la persistencia de archivos y la generación de tablas y figuras: <https://github.com/joacogamalerio/tesis-project-repository>

3.1.1 Conjunto de datos, subset de entrenamiento y partición

Los experimentos se realizaron sobre MNIST, un conjunto de datos clásico de clasificación de imágenes compuesto por dígitos manuscritos en escala de grises pertenecientes a diez clases (0–9). Cada imagen tiene resolución de 28×28 píxeles, esta configuración puede interpretarse como una matriz bidimensional, pero también como un vector de 784 intensidades, el cual puede ser usado como *input* en una red neuronal. El dataset se encuentra particionado de manera estándar en 60 000 imágenes de entrenamiento y 10 000 imágenes de test, y constituye un benchmark ampliamente utilizado para analizar el desempeño de arquitecturas de redes neuronales y mecanismos de regularización. La Figura 14 muestra ejemplos representativos del tipo de imágenes contenidas en este conjunto.

Utilizando esta base, en este proyecto se trabajó con un subconjunto controlado del conjunto de entrenamiento en lugar de entrenar sobre la totalidad de los datos disponibles. Esta elección fue deliberada, dado que el trabajo implicó explorar múltiples arquitecturas, múltiples configuraciones de hiperparámetros y múltiples semillas por configuración, entrenar sistemáticamente sobre el dataset completo hubiera incrementado de manera considerable el costo computacional total, dificultando tanto la iteración experimental como la repetición de corridas bajo condiciones controladas.

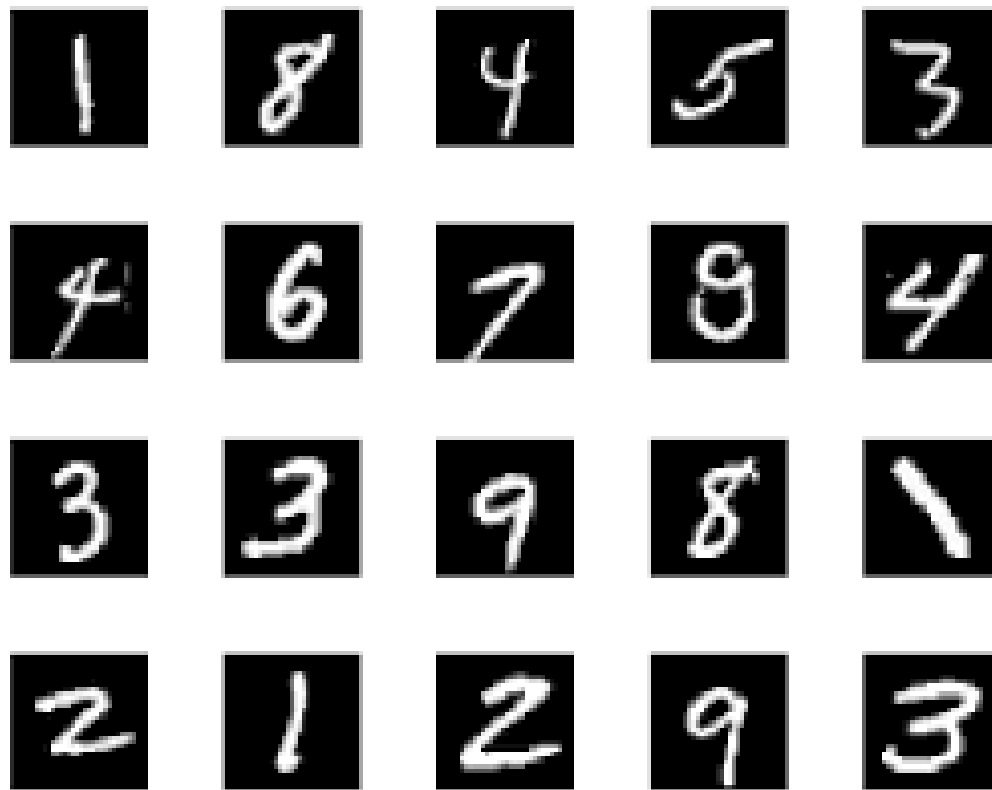


Figura 14: Ejemplos representativos del dataset MNIST. Se muestran imágenes manuscritas en escala de grises de 28×28 píxeles correspondientes a distintas clases del conjunto.

Además del costo computacional, el uso de un subset más reducido respondió a una segunda motivación metodológica. Trabajar con un conjunto de entrenamiento más pequeño favorece la aparición de sobreajuste en la arquitectura sin regularización, lo cual vuelve más visible el problema que justamente se desea estudiar. En ese contexto, los mecanismos de regularización comparados pueden analizarse en un escenario donde su necesidad resulta más clara y donde las diferencias entre estrategias tienden a manifestarse de manera más marcada. Por esa razón, el subset no solo reduce costos, sino que también funciona como una decisión experimental orientada a facilitar el estudio del fenómeno de interés.

En consecuencia, se adoptó un esquema de trabajo sobre un subset estratificado por clase, construido de manera reproducible a partir de una semilla específica de partición. Este subset se divide a su vez en entrenamiento y validación.

Esta decisión introduce, naturalmente, un alcance más acotado para los resultados. En esta tesis no se pretende establecer conclusiones definitivas sobre el mejor desempeño posible de cada arquitectura sobre MNIST, sino obtener una aproximación experimental razonable al problema, suficientemente robusta para identificar tendencias consistentes y para discutir con fundamento el papel de los mecanismos de regularización estudiados. En otras palabras, se privilegió un protocolo experimental replicable y controlado por sobre una búsqueda exhaustiva y computacionalmente costosa.

3.2 Protocolo experimental principal: redes *feed-forward* para clasificación

El núcleo del estudio experimental desarrollado en esta tesis está dado por la comparación entre arquitecturas *feed-forward* aplicadas a una tarea de clasificación sobre MNIST. En esta sección se describen las decisiones de diseño, entrenamiento y evaluación que estructuran ese protocolo principal, manteniendo constantes los elementos necesarios para que las diferencias observadas puedan atribuirse, en la medida de lo posible, al mecanismo de regularización empleado.

3.2.1 Arquitecturas comparadas y criterios de comparación

El protocolo principal compara arquitecturas densas de tipo *feed-forward* para clasificación multiclase que comparten una misma topología base y difieren únicamente en el mecanismo de regularización utilizado. En particular, se consideran una red base sin regularización estocástica, denominada *OverFitNet*, una variante con *Dropout*, una variante con *DropConnect* y una variante con el mecanismo propuesto *BoostDropout*.

La arquitectura común a las cuatro redes puede resumirse como

$$784 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 10,$$

donde la entrada corresponde a la imagen de MNIST vectorizada, las tres capas ocultas son capas lineales densas de 256 unidades con activación ReLU, y la capa de salida produce 10 *logits* para clasificación multiclase. Esta especificación se representa esquemáticamente en la Figura 15.

En términos de implementación, tanto *BoostDropout* como *DropConnect* fueron construidos mediante capas propias. En el caso de *BoostDropout*, esto responde a que se trata de una arquitectura propuesta en este trabajo, por lo que su mecanismo de regularización no forma parte de las capas estándar de PyTorch. En el caso de *DropConnect*, la necesidad de una implementación propia surge porque PyTorch no ofrece una capa nativa con esta funcionalidad. En ambos casos, las capas se desarrollaron utilizando las herramientas que brinda PyTorch para definir módulos personalizados, manteniendo compatibilidad con el resto del pipeline de entrenamiento y evaluación.

Asimismo, se mantiene una semántica consistente para el hiperparámetro p dentro de cada familia de modelos: en *Dropout* representa la probabilidad de apagar activaciones, en *DropConnect* la probabilidad de enmascarar conexiones, y en *BoostDropout* la probabilidad de aplicar la máscara multiplicativa correspondiente. Esta consistencia facilita tanto la exploración de hiperparámetros como la interpretación comparativa de los resultados.

Arquitectura base

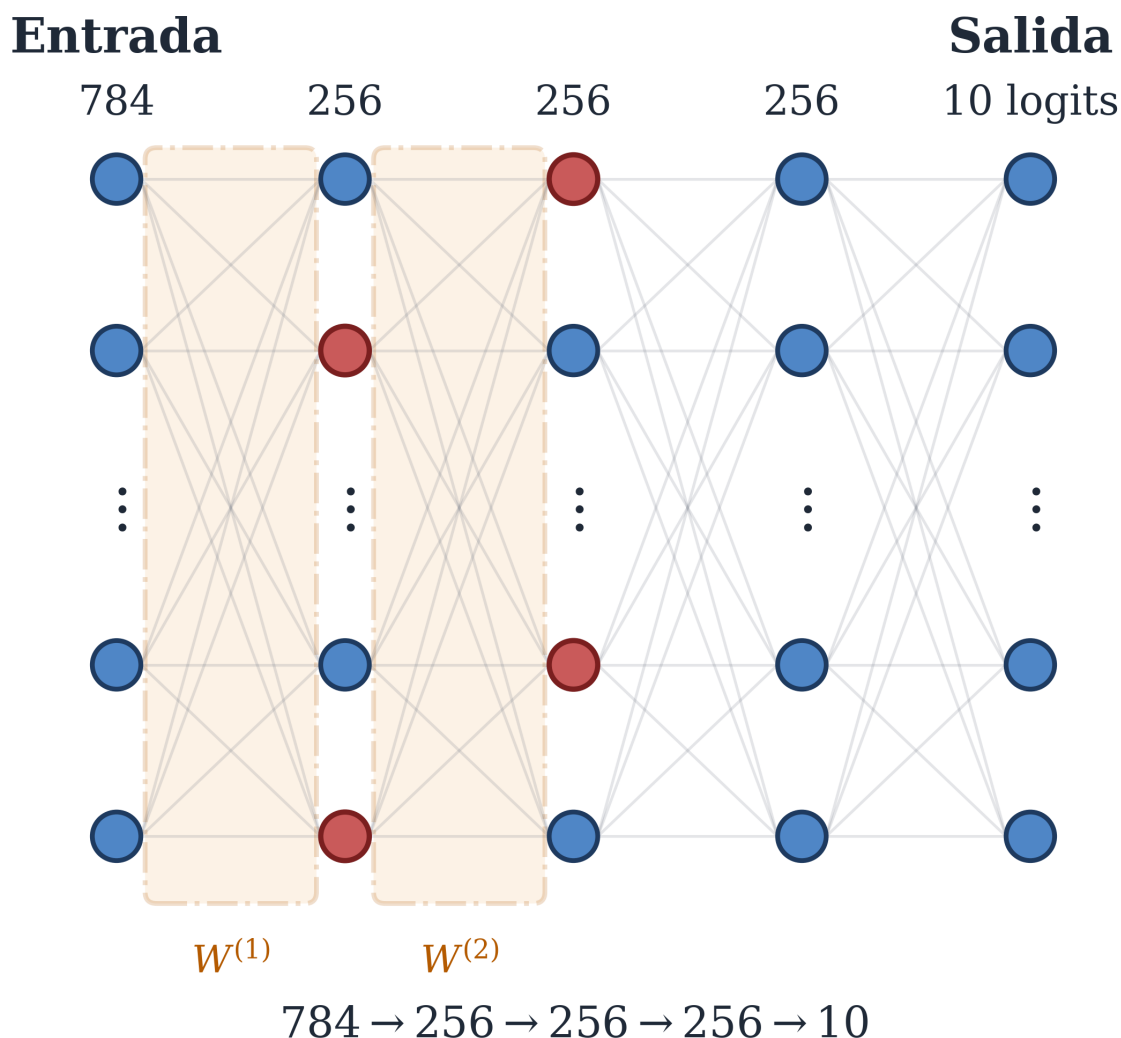


Figura 15: Esquema de la arquitectura base utilizada en el protocolo principal. Las regiones sombreadas sobre $W^{(1)}$ y $W^{(2)}$ indican las matrices de pesos sobre las que actúa *DropConnect* en la implementación considerada. Las neuronas resaltadas en rojo representan, de manera ilustrativa, activaciones intermedias sobre las que actúan los mecanismos de *Dropout* y *Boost-Dropout* luego de la primera y de la segunda capa oculta.

3.2.2 Hiperparámetros fijos

Para sostener una comparación justa entre arquitecturas, una parte de las decisiones de entrenamiento se mantuvo fija en todas las corridas del protocolo principal, con leves variaciones entre la fase de búsqueda de hiperparámetros y los entrenamientos propios de la fase final de evaluación con múltiples semillas. Esta elección permitió concentrar la exploración únicamente en los aspectos más relevantes para el problema estudiado, evitando expandir

innecesariamente el espacio experimental.

La elección de estos hiperparámetros base no fue arbitraria. Se buscó deliberadamente un régimen en el que la red sin regularización exhibiera el patrón típico de sobreajuste discutido en la Sección 2.8.2 y esquematizado en la Figura 8: la pérdida de entrenamiento continúa descendiendo mientras la pérdida de validación deja de mejorar y luego comienza a aumentar. Como se verá con mayor claridad en las secciones de resultados, esta configuración efectivamente induce un contexto de *overfitting*, lo cual resulta metodológicamente conveniente para evaluar el efecto de las capas de regularización. En otras palabras, el objetivo no fue evitar de antemano el sobreajuste en la arquitectura base, sino utilizarlo como escenario controlado para medir hasta qué punto *Dropout*, *DropConnect* y *BoostDropout* logran mitigarlo y mejorar la generalización.

La Figura 16 resume los hiperparámetros y decisiones de entrenamiento que permanecieron constantes en todas las corridas del protocolo principal.

Hiperparámetros fijos del entrenamiento

- Optimizador utilizado: `[torch.optim.Adam]`
- Parámetros del optimizador: `[Default]`.
- Tasa de aprendizaje base: `[1e-3]`.
- Tamaño de batch: `[256]`.
- Número máximo de épocas: `[30]` en Grid Search; `[60]` en entrenamiento principal.
- Paciencia para *early stopping*: `[6]` en Grid Search; `[10]` en entrenamiento principal.
- Criterio monitoreado para selección de modelos: **pérdida en validación**.
- Normalización del dataset: `[True]`.
- Tamaño del subset de entrenamiento: 10 % del conjunto de entrenamiento de MNIST.
- Tamaño del subset de validación: 20 % del subset de entrenamiento (es decir 2 % del conjunto de entrenamiento de MNIST).
- Semilla de partición para generar subset de entrenamiento y validación.

Figura 16: Resumen de los hiperparámetros fijos y de las decisiones de entrenamiento mantenidas constantes en todas las corridas del protocolo principal.

3.2.3 Entrenamiento individual, selección de modelos y trazabilidad

Cada corrida individual del pipeline fue diseñada para permitir varios niveles de análisis sobre un mismo proceso de entrenamiento. En primer lugar, se identifica el **modelo de mejor validación**, es decir, el estado de parámetros asociado al menor valor de pérdida de validación observado durante la corrida, para una cantidad total de épocas definida. Este

modelo es el análogo conceptual de lo que, a nivel de implementación, suele guardarse como el mejor checkpoint del entrenamiento. A menudo nos referiremos a él como **MMV**.

En segundo lugar, se considera el **modelo seleccionado por *early stopping***, entendido como el modelo que sería elegido bajo la política de detención temprana definida para el experimento. Esta distinción entre criterios de selección es importante porque ambas alternativas responden a preguntas metodológicas diferentes y serán utilizadas de manera separada tanto en la búsqueda de hiperparámetros como en la evaluación final multisemilla.

Además de los pesos asociados a estas distintas formas de selección, cada entrenamiento persiste historiales por época, figuras de evolución de métricas y archivos de metadata. Esta persistencia exhaustiva es una parte estructural del diseño. El archivo de historial permite reconstruir curvas de entrenamiento y validación, mientras que la metadata deja registro de semillas, hiperparámetros, estadísticas del dataset, resultados finales y archivos generados. Gracias a ello, cada resultado reportado en la tesis puede rastrearse hasta la corrida concreta que lo produjo.

3.2.4 Métricas de evaluación

En el protocolo principal de clasificación, el desempeño de cada modelo se evaluó mediante la pérdida de entropía cruzada y el error de clasificación. Para una muestra i , la red produce un vector de logits $\mathbf{z}_i$, a partir del cual se obtiene la clase predicha como

$$\hat{y}_i = \arg \max_c z_{ic}.$$

Dado un conjunto de N muestras, la exactitud se define como la proporción de muestras correctamente clasificadas:

$$\text{Acc} = \frac{\text{número de predicciones correctas}}{N}.$$

A partir de esta cantidad, el error de clasificación se define como

$$\text{Error} = 1 - \text{Acc}.$$

Cuando se reporta en porcentaje, se utiliza equivalentemente

$$\text{Error}(\%) = 100(1 - \text{Acc}).$$

De este modo, una exactitud de 0,98 corresponde a un error de clasificación del 2 %. El error expresa directamente la fracción de ejemplos mal clasificados. Reportar esta métrica resulta particularmente útil cuando las exactitudes son altas: por ejemplo, pasar de 98,5 % a 99,0 % de exactitud puede parecer una mejora pequeña, pero equivale a reducir el error de 1,5 % a 1,0 %, es decir, aproximadamente un tercio menos de clasificaciones incorrectas.

3.2.5 Búsqueda de hiperparámetros

La primera etapa del protocolo experimental consiste en una búsqueda de hiperparámetros con múltiples semillas de entrenamiento por combinación. En lugar de elegir configuraciones a partir de una sola corrida, cada combinación de hiperparámetros se repite varias veces con

diferentes semillas experimentales y luego se agregan los resultados correspondientes. Esta decisión responde a una motivación clara: reducir la dependencia de corridas particularmente favorables o desfavorables y obtener una estimación más estable del comportamiento de cada configuración, atenuando así el impacto de fluctuaciones libradas al azar.

La Tabla 1 resume el espacio de búsqueda considerado para las arquitecturas regularizadas. Como ya se explicó, los hiperparámetros mantenidos fijos fueron elegidos de modo tal que la arquitectura base exhibiera sobreajuste; por ello, la exploración se concentró en los parámetros propios de los mecanismos de regularización.

Cuadro 1: Espacio de hiperparámetros explorado durante la búsqueda en grilla.

Arquitectura	Hiperparámetro	Valores explorados
DropoutNet	p	[0.1, 0.3, 0.5, 0.7, 0.9]
DropConnectNet	p	[0.1, 0.3, 0.5, 0.7, 0.9]
BoostDropoutNet	p	[0.1, 0.3, 0.5, 0.7, 0.9]
BoostDropoutNet	λ	[-0.6, -0.4, 0.0, 0.3, 0.6, 0.9]
BoostDropoutNet	normalización de máscara	[True, False]

En el diseño elegido, la etapa de exploración se realiza con tres semillas experimentales por combinación de hiperparámetros. Para cada arquitectura y para cada criterio de selección de modelo definido en la sección anterior, se computan métricas agregadas entre semillas, incluyendo medias y desviaciones estándar de pérdida y exactitud en validación. La regla formal de selección de hiperparámetros utiliza exclusivamente la pérdida media de validación. En otras palabras, la mejor configuración se define como aquella que minimiza la pérdida media de validación dentro de cada arquitectura y dentro de cada criterio de selección considerado.

Esta forma de selección tiene varias ventajas. En primer lugar, mantiene coherencia con la separación metodológica entre validación y test. En segundo lugar, incorpora de manera explícita una medida de robustez inter-seed ya desde la etapa de ajuste de hiperparámetros. Finalmente, permite trasladar a la fase final configuraciones que ya fueron filtradas por un criterio menos sensible al azar de una corrida individual. El objetivo no fue realizar una optimización exhaustiva del espacio de hiperparámetros, sino identificar configuraciones razonables y prometedoras bajo un presupuesto de cómputo acotado.

En términos de costo experimental total, esta etapa de exploración comprendió 210 entrenamientos independientes, cada uno de 30 épocas. Esta cantidad surge de combinar todas las configuraciones exploradas para *Dropout*, *DropConnect* y *BoostDropout*, junto con las tres semillas experimentales utilizadas por combinación.

3.2.6 Evaluación final multiseed

Una vez seleccionados los hiperparámetros a partir de validación, se ejecuta una segunda etapa de evaluación final. En esta fase se utilizan nuevas semillas, diferentes de las empleadas durante la exploración, para obtener una comparación más robusta entre arquitecturas bajo configuraciones ya fijadas. En la implementación utilizada en este trabajo, la evaluación final se realiza con diez semillas compartidas entre arquitecturas. El uso de una lista común de semillas finales para todas las arquitecturas dentro de una misma evaluación mejora la

comparabilidad de los resultados, ya que expone a los modelos a condiciones experimentales análogas en términos de inicialización y dinámica estocástica. A su vez, los resultados finales se mantienen separados según el criterio de selección de modelo analizado. Esta separación es importante porque el **modelo de mejor validación** y el modelo seleccionado por *early stopping* responden a preguntas distintas sobre el entrenamiento, y mezclarlos en un único análisis ocultaría diferencias relevantes del comportamiento de las arquitecturas.

La fase final produce tanto tablas detalladas por corrida como resúmenes agregados por arquitectura y por criterio de selección. También genera gráficos comparativos, como boxplots de métricas finales y curvas de validación representativas.

En total, la etapa de evaluación final comprendió 40 entrenamientos independientes, cada uno de 60 épocas, correspondientes a 10 corridas por cada una de las cuatro arquitecturas comparadas.

Como complemento posterior a la búsqueda inicial de hiperparámetros y a la evaluación final multisevilla, se realizaron además entrenamientos adicionales de análisis de sensibilidad, orientados a estudiar por separado la dependencia de las métricas respecto de p y de λ . Estos barridos no formaron parte del *grid search* utilizado para seleccionar configuraciones, sino que fueron incorporados en una etapa posterior con fines descriptivos e interpretativos. En particular, para cada valor explorado se entrenaron tres semillas durante 40 épocas y se promediaron las métricas correspondientes.

3.2.7 Reproducibilidad y separación entre fuentes de aleatoriedad

Aunque las secciones anteriores ya introdujeron el uso de semillas específicas en las distintas etapas del protocolo, conviene explicitar por separado el criterio general de reproducibilidad adoptado en este trabajo. En lugar de utilizar una única semilla global para todo el proceso, se distinguieron explícitamente dos fuentes de aleatoriedad con roles diferentes: la semilla de partición y la semilla experimental.

La semilla de partición controla la construcción del conjunto de datos utilizado en los experimentos. En primer lugar, determina qué 10 % del conjunto original de entrenamiento de MNIST se selecciona como subconjunto de trabajo. Luego, dentro de ese subconjunto fijo, determina la partición estratificada entre entrenamiento y validación, con proporciones 80 % y 20 %, respectivamente. Así, esta semilla fija el escenario de datos sobre el cual se entrenan y comparan todas las arquitecturas.

La semilla experimental, en cambio, controla la aleatoriedad propia de cada corrida de entrenamiento: inicialización de pesos, orden de los mini-batches, generadores aleatorios de `torch`, `numpy` y `random`, y máscaras estocásticas utilizadas por *Dropout*, *DropConnect* y *BoostDropout*. Esta separación permite distinguir la variabilidad debida a la elección del subconjunto de datos de la variabilidad inducida por el entrenamiento.

Bajo este esquema, dos corridas realizadas con el mismo código, el mismo entorno, la misma semilla de partición y la misma semilla experimental deben reproducir la misma ejecución, incluyendo inicialización, orden de lotes, secuencia de máscaras aleatorias y resultados finales. Esta propiedad no es trivial, porque en experimentos con regularización estocástica pequeñas modificaciones en el consumo del generador aleatorio pueden alterar la corrida completa. Por ello, se evitó resembrar generadores dentro del *forward* o introducir reinicializaciones artificiales por batch. La aleatoriedad de *Dropout*, *DropConnect* y *BoostDropout* se

conserva así como una secuencia natural de máscaras distintas a lo largo del entrenamiento, pero reproducible cuando la corrida se repite bajo las mismas condiciones iniciales.

3.3 Protocolo complementario: autoencoders para análisis de representaciones

Además del protocolo principal de clasificación, en esta tesis se incorpora un esquema complementario basado en autoencoders. Su propósito no es extender la comparación cuantitativa principal, sino disponer de un marco controlado para analizar cualitativamente las representaciones internas aprendidas bajo distintos mecanismos de regularización.

3.3.1 Motivación y objetivo del análisis cualitativo

La motivación de esta elección se apoya directamente en los experimentos presentados por Srivastava et al. [1], en particular en las Figuras 7.1 y 7.2 de dicho trabajo. En ese estudio, los autores entrenan autoencoders sobre MNIST y muestran que, aun cuando el error de reconstrucción es similar, las características aprendidas difieren de manera significativa entre modelos con y sin *dropout*. En particular, reportan que el uso de *dropout* reduce la coadaptación entre neuronas y favorece la aparición de representaciones más interpretables y estructuradas.

Siguiendo esa línea, en esta tesis se implementa un autoencoder de referencia con una arquitectura simple y controlada, con el objetivo de reproducir de manera aproximada ese tipo de análisis. El interés principal no radica en optimizar el desempeño de reconstrucción, sino en utilizar el modelo como una herramienta para inspeccionar las activaciones internas, la esparsidad de las representaciones (*sparsity*) y los posibles patrones de coadaptación entre neuronas.

3.3.2 Arquitectura del autoencoder de referencia

El modelo utilizado en este análisis complementario consiste en un autoencoder denso de una sola capa oculta, diseñado para mantener una arquitectura suficientemente simple como para facilitar la inspección de las representaciones aprendidas. La red recibe como entrada la imagen de MNIST vectorizada en un arreglo de 784 componentes, la transforma mediante un codificador en una representación latente de 256 dimensiones y, a partir de ella, reconstruye la muestra original por medio de un decodificador. Esta arquitectura es la que se usó en Srivastava et al. [1] para generar las Figuras 7.1 y 7.2 del *paper*.

En términos de arquitectura, el modelo puede resumirse por la cantidad de neuronas o unidades en cada capa como

$$784 \rightarrow 256 \rightarrow 784.$$

Aquí, las 784 unidades de entrada corresponden a los píxeles de la imagen vectorizada, las 256 unidades intermedias conforman la representación latente, y las 784 unidades de salida corresponden a la reconstrucción de la imagen original. En la implementación utilizada, la representación latente emplea activación ReLU y la reconstrucción final se obtiene aplicando

una sigmoide a la salida del decodificador. El entrenamiento se realizó utilizando una función de pérdida de entropía cruzada binaria (*Binary Cross-Entropy*, BCE).

La elección de BCE se debe a que las imágenes de MNIST fueron convertidas a tensores con intensidades escaladas al intervalo $[0, 1]$, y la salida del decodificador se obtuvo mediante una función sigmoide, por lo que cada componente reconstruida también pertenece a ese intervalo. Bajo esta formulación, la reconstrucción puede evaluarse píxel a píxel mediante una pérdida de entropía cruzada binaria. Esta elección es habitual cuando las entradas se interpretan como intensidades normalizadas o como valores compatibles con probabilidades de activación. No obstante, no es la única opción posible: también podría utilizarse error cuadrático medio si se interpreta la reconstrucción como una regresión directa sobre intensidades continuas.

La intención de este esquema no es competir con arquitecturas especializadas de reconstrucción ni maximizar el desempeño absoluto sobre el problema, sino ofrecer un entorno controlado para comparar cómo los distintos mecanismos de regularización modifican la estructura de las características internas aprendidas por la red. El Cuadro 2 organiza los parámetros principales del autoencoder.

Cuadro 2: Resumen de la arquitectura y de los componentes principales del autoencoder de referencia utilizado para el análisis de representaciones.

Componente	Especificación
Tipo de entrada	Imágenes de MNIST vectorizadas de dimensión 784
Arquitectura del codificador	Linear (784, 256)
Dimensión del espacio latente	256
Activación en el espacio latente	ReLU
Arquitectura del decodificador	Linear (256, 784)
Activación de salida	Sigmoide
Función de pérdida de entrenamiento	BCE
Métricas reportadas	BCE por ejemplo y MSE por ejemplo

3.3.3 Variantes regularizadas analizadas

A partir de este esquema base, el análisis se extiende a las distintas variantes de regularización consideradas en el trabajo. En particular, se entrenan versiones del autoencoder utilizando:

- una configuración sin regularización estocástica,
- *Dropout*,
- *DropConnect*,
- *BoostDropout*.

En todas las variantes se mantuvo fija la misma arquitectura base del autoencoder, de tipo denso y con una sola capa oculta. En las versiones con regularización sobre activaciones, *Dropout* y *BoostDropout* se aplicaron sobre la representación latente luego de la activación

ReLU del codificador y antes del decodificador. En la variante con *DropConnect*, en cambio, la regularización se aplicó sobre los pesos de la capa codificadora, mientras que el decodificador se mantuvo como una capa lineal densa sin regularización explícita.

De este modo, se busca comparar cómo cada uno de estos mecanismos afecta no solo el error de reconstrucción, sino también la estructura cualitativa de las representaciones internas aprendidas por la red.

3.3.4 Entrenamiento y evaluación del autoencoder

A diferencia del protocolo principal de clasificación, en este análisis complementario no se realizó una búsqueda específica de hiperparámetros para el autoencoder. En su lugar, se entrenó una única vez cada arquitectura considerada, con el objetivo de disponer de un conjunto de modelos comparables para el análisis cualitativo de representaciones.

Cada variante se entrenó durante 60 épocas sobre el conjunto de entrenamiento de MNIST, sin partición adicional de validación, y posteriormente se evaluó sobre el conjunto de test. Para mantener una configuración de trabajo consistente entre arquitecturas y evitar un costo computacional excesivo, se reutilizaron los hiperparámetros seleccionados en el protocolo principal de clasificación para las versiones correspondientes de *Dropout*, *DropConnect* y *BoostDropout*.

Esta decisión implica que las configuraciones utilizadas en el autoencoder no deben interpretarse como necesariamente óptimas para reconstrucción, sino como un compromiso metodológico orientado a facilitar una comparación cualitativa controlada entre representaciones internas.

También en este protocolo complementario se controló explícitamente la aleatoriedad experimental. Para cada comparación entre variantes se utilizaron semillas reproducibles, de modo que los modelos partieran de la misma inicialización de pesos y fueran expuestos al mismo orden de mini-batches. En consecuencia, las diferencias observadas entre *OverFitNet*, *Dropout*, *DropConnect* y *BoostDropout* pueden atribuirse con mayor claridad al mecanismo de regularización aplicado, y no a variaciones accidentales en la inicialización o en la secuencia de entrenamiento.

3.3.5 Criterios de análisis de representaciones

El análisis representacional se centrará principalmente en tres aspectos. En primer lugar, se observarán los patrones de activación de las unidades ocultas y las características aprendidas por la red, con el objetivo de evaluar el grado de interpretabilidad visual de los filtros o detectores emergentes. En segundo lugar, se analizará el grado de esparsidad de las representaciones, atendiendo a cuántas neuronas permanecen activas ante una entrada dada y a cómo se distribuye su magnitud. En tercer lugar, se inspeccionará la posible presencia de coadaptaciones entre neuronas, es decir, dependencias excesivamente rígidas entre unidades que puedan indicar redundancia o escasa robustez de la representación.

Los detalles específicos de implementación del autoencoder se presentaron en la sección anterior, mientras que los aspectos operativos más puntuales de algunas visualizaciones — en particular aquellas cuya interpretación depende fuertemente del modo de construcción, como los gráficos de *sparsity* representacional— se introducen directamente en la parte de

resultados experimentales. Allí se discutirá con mayor profundidad en qué medida los efectos observados reproducen —o se apartan de— los reportados en Srivastava et al. [1], y cómo se comportan las variantes *DropConnect* y *BoostDropout* dentro de este mismo marco de análisis.

3.3.6 Construcción de las visualizaciones representacionales

El análisis cualitativo se apoyó en cuatro tipos de visualizaciones complementarias. En primer lugar, se inspeccionaron las *features* asociadas a la primera capa del codificador, reorganizando los pesos de cada neurona latente sobre la grilla espacial original de MNIST. Esta representación se utilizó como aproximación visual al tipo de patrón al que responde cada unidad.

En segundo lugar, se construyeron decodificaciones a partir de neuronas latentes individuales. Para ello, se activó una única componente del espacio latente y se propagó esa activación a través del decodificador, con el fin de visualizar qué tipo de estructura reconstruye cada unidad por separado.

En tercer lugar, se analizaron visualizaciones asociadas a la esparsidad de activaciones en el espacio latente, con el objetivo de comparar cualitativamente la *representational sparsity* inducida por cada mecanismo de regularización.

Por último, se inspeccionó la distribución de pesos de la primera capa mediante histogramas, con el fin de explorar de manera cualitativa una posible tendencia hacia configuraciones con mayor concentración de parámetros cerca de cero.

Los detalles operativos específicos de cada figura se introducen directamente en la sección de resultados correspondiente, cuando contribuyen de manera inmediata a la interpretación de la visualización presentada.

3.4 Alcances, limitaciones y sentido de los resultados

Ya introducidos los protocolos, es importante explicitar el alcance de las conclusiones extraídas a partir de este diseño experimental. El protocolo experimental principal adoptado busca un equilibrio entre rigor metodológico, costo computacional y posibilidad real de iteración. Por esa razón, se trabajó sobre un subset controlado de MNIST, con una etapa de exploración multiseed acotada y una evaluación final más robusta pero todavía computacionalmente manejable. Este compromiso fue una decisión consciente del trabajo.

En consecuencia, los resultados deben interpretarse como evidencia experimental controlada sobre el problema estudiado, y no como una afirmación definitiva acerca de la superioridad universal de una arquitectura o de un mecanismo de regularización en todo contexto posible. El valor principal del diseño no reside en haber agotado todas las variantes experimentales imaginables, sino en haber construido un procedimiento suficientemente consistente como para comparar modelos de manera justa, reproducible y trazable.

Esta cautela es particularmente importante en el protocolo complementario, en el cual se realiza un análisis con autoencoders. Allí el objetivo no es establecer una comparación exhaustiva de desempeño en reconstrucción, sino disponer de un instrumento cualitativo para

estudiar representaciones internas y explorar si ciertos efectos asociados a *dropout* también aparecen, con matices propios, en *DropConnect* y *BoostDropout*.

Esta postura es especialmente importante en esta tesis. Más que perseguir un rendimiento absoluto máximo a cualquier costo, el interés estuvo en desarrollar una aproximación experimental sólida al problema, capaz de sostener conclusiones razonables sobre tendencias observadas, sensibilidad a la aleatoriedad del entrenamiento y diferencias entre criterios de selección de modelos. En ese marco, la robustez inter-seed, la separación estricta entre validación y test, y la persistencia completa de artefactos y metadata resultan tan importantes como las métricas finales de desempeño.

Capítulo 4. Resultados experimentales

4.1 Resultados experimentales en clasificación con redes neuronales *feed-forward*

En esta sección se presentan los resultados obtenidos para el protocolo principal de clasificación introducido en el Capítulo 3, en particular en las Secciones 3.2, 3.2.5 y 3.2.6. El objetivo es comparar, bajo un esquema metodológicamente controlado, el comportamiento de la arquitectura base sin regularización estocástica (*OverFitNet*) frente a las variantes regularizadas con *Dropout* (*DropoutNet*), *DropConnect* (*DropConnectNet*) y *BoostDropout* (*BoostDropoutNet*).

Siguiendo el criterio adoptado en el diseño experimental, la discusión se organiza en dos niveles. En primer lugar, se resumen los resultados de la etapa de exploración de hiperparámetros, destacando las configuraciones seleccionadas para cada arquitectura y para cada criterio de selección de modelo. En segundo lugar, se presentan los resultados del entrenamiento principal multisentencia, incluyendo distribuciones de métricas en validación y test, análisis de la mejor época y curvas representativas de entrenamiento.

4.1.1 Resultados del *grid search*

Tal como se explicó en la Sección 3.2.5, la búsqueda de hiperparámetros se realizó separadamente para cada arquitectura, agregando resultados sobre múltiples semillas experimentales y seleccionando configuraciones a partir de la pérdida media de validación. En esta parte no se repiten los hiperparámetros fijos del protocolo, ya resumidos en la Sección 3.2.2; aquí se reportan únicamente los hiperparámetros efectivamente explorados y seleccionados para cada método de regularización.

4.1.1.1 Configuraciones seleccionadas para *Dropout*

El Cuadro 3 resume la configuración elegida para la arquitectura *DropoutNet* según los dos criterios de selección considerados en esta tesis.

Cuadro 3: Hiperparámetros seleccionados para la arquitectura con *Dropout*.

Hiperparámetro	MMV	Early stopping
p	0.5	0.5

4.1.1.2 Configuraciones seleccionadas para *DropConnect*

El Cuadro 4 sintetiza los valores finalmente retenidos para *DropConnectNet*.

Cuadro 4: Hiperparámetros seleccionados para la arquitectura con *DropConnect*.

Hiperparámetro	MMV	Early stopping
p	0.5	0.5

4.1.1.3 Configuraciones seleccionadas para *BoostDropoutNet*

Cuadro 5: Hiperparámetros seleccionados para la arquitectura con *BoostDropout*.

Hiperparámetro	Mejor modelo de validación	Early stopping
p	0.7	0.7
λ	0.9	0.9
Normalización de máscara	False	False

A diferencia de *DropoutNet* y *DropConnectNet*, en *BoostDropout* la exploración involucró más de un hiperparámetro libre, por lo que el Cuadro 5 resume no solo la probabilidad p , sino también el valor de λ y la decisión respecto de la normalización de la máscara.

4.1.2 Resultados del entrenamiento principal

Una vez fijadas las configuraciones seleccionadas en la etapa de exploración, se ejecutó la evaluación final multisentencia descrita en la Sección 3.2.6. En esta etapa se entrenaron nuevamente las cuatro arquitecturas consideradas: *OverFitNet*, *DropoutNet*, *DropConnectNet* y *BoostDropoutNet*. Para *OverFitNet* se utilizaron los hiperparámetros base definidos en el Capítulo 3, mientras que para las arquitecturas regularizadas se emplearon las configuraciones seleccionadas en la subsección anterior.

Con el fin de mantener la separación conceptual entre modelos elegidos por entrenamiento, los resultados se presentan distinguiendo explícitamente entre el criterio de **MMV** (mejor modelo de validación) y el criterio de *early stopping*.

4.1.2.1 Criterio: MMV

La Figura 17 resume, para el criterio de mejor modelo de validación, la distribución de error y pérdida tanto en validación como en test. En todos los casos, cada boxplot resume la variabilidad obtenida sobre el conjunto de semillas finales para cada una de las cuatro arquitecturas comparadas.

Cuadro 6: Resumen estadístico de las métricas de **pérdida en test** para el criterio de **MMV**.

Modelo	Media	Mediana	Desv. est.	Mín.	Máx.
OverFitNet	0.2311	0.231	0.0063	0.2187	0.2429
DropoutNet	0.2129	0.2132	0.0074	0.2005	0.225
DropConnectNet	0.2133	0.2142	0.0084	0.1991	0.2246
BoostDropoutNet	0.1598	0.1557	0.012	0.1483	0.1909

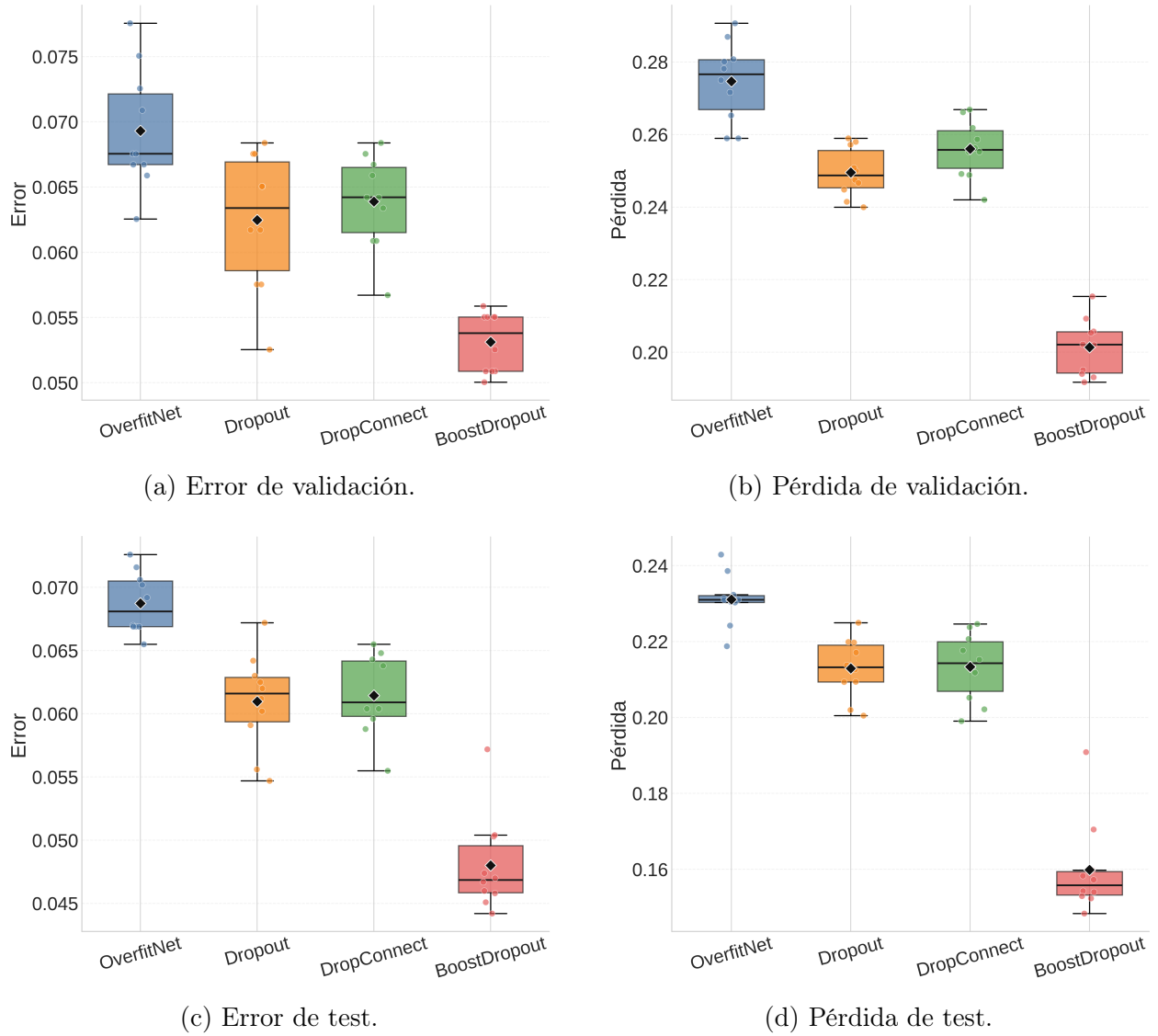


Figura 17: Distribución de métricas bajo el criterio de mejor modelo de validación. En cada panel se compara *OverFitNet* con las tres variantes regularizadas. La disposición conserva la convención adoptada en esta tesis: métricas de error a la izquierda y métricas de pérdida a la derecha.

Cuadro 7: Resumen estadístico de la métrica de **error en test** para el criterio de **MMV**.

Modelo	Media	Mediana	Desv. est.	Mín.	Máy.
OverFitNet	0.0687	0.0681	0.0023	0.0655	0.0726
DropoutNet	0.0609	0.0616	0.0036	0.0547	0.0672
DropConnectNet	0.0614	0.0609	0.0029	0.0555	0.0655
BoostDropoutNet	0.0480	0.0468	0.0036	0.0442	0.0572

Los resultados del criterio de **MMV** muestran un patrón consistente tanto en la Figura 17 como en los Cuadros 7 y 6. En primer lugar, *BoostDropoutNet* presenta el mejor desempeño global dentro de las arquitecturas comparadas: en test alcanza la menor pérdida media y también el menor error medio, quedando por debajo de *OverFitNet*, *DropoutNet* y *DropConnectNet*. Esta mejora no aparece solo en el valor medio, sino también en la mediana, lo que indica que el desplazamiento hacia valores menores no depende de una corrida aislada particularmente favorable, sino que se sostiene a lo largo del conjunto de semillas.

En segundo lugar, *DropoutNet* y *DropConnectNet* mejoran con claridad respecto de *OverFitNet*, pero permanecen muy próximos entre sí. Esta cercanía se observa tanto en las tablas como en los boxplots: sus medianas y promedios son similares, y sus distribuciones aparecen parcialmente superpuestas, especialmente en las métricas de error. En consecuencia, dentro de este criterio de selección, ambos métodos clásicos de regularización muestran un comportamiento competitivo y comparable, aunque sin alcanzar el nivel de desempeño de *BoostDropoutNet*.

Un punto importante es la interpretación de la dispersión. Si bien la desviación estándar de *BoostDropoutNet* en la pérdida de test es mayor que la de las otras arquitecturas, la inspección visual de los boxplots indica que su distribución completa se encuentra desplazada hacia valores inferiores. En particular, la caja central —determinada por los cuartiles— y los bigotes permanecen en una región claramente más baja, con escasa o nula superposición respecto de *OverFitNet* y con una separación también visible frente a *DropoutNet* y *DropConnectNet*. Por ello, la desviación estándar no debe leerse de manera aislada: aunque exista variabilidad entre semillas, dicha variabilidad ocurre alrededor de un régimen de pérdida sistemáticamente más favorable.

Este mismo patrón aparece al mirar la validación. En las subfiguras correspondientes de la Figura 17, *BoostDropoutNet* no solo conserva menores valores centrales, sino que además muestra una distribución más concentrada en la zona de mejor desempeño. En otras palabras, la arquitectura propuesta no parece beneficiarse de unas pocas corridas extremas, sino que tiende a ubicarse de forma estable en un rango de validación y test más bajo que el resto.

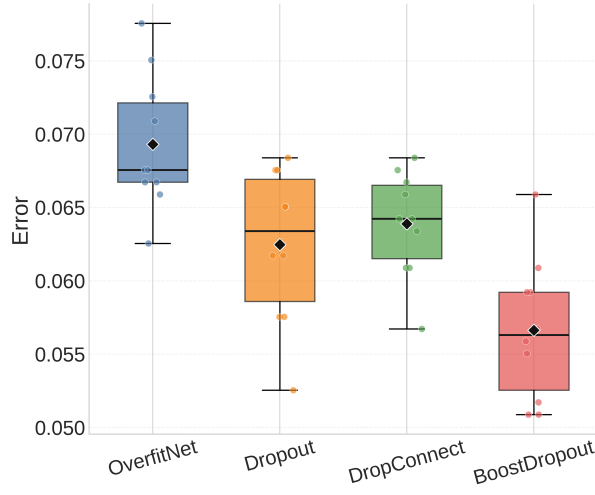
En conjunto, bajo el criterio de mejor modelo de validación, los resultados sugieren que *BoostDropoutNet* logra la mejor combinación entre calidad predictiva y consistencia global del desempeño. *DropoutNet* y *DropConnectNet* también regularizan eficazmente la arquitectura base y reducen el deterioro observado en *OverFitNet*, pero la separación visual y numérica favorece de forma clara a la variante propuesta.

4.1.2.2 Criterio: *early stopping*

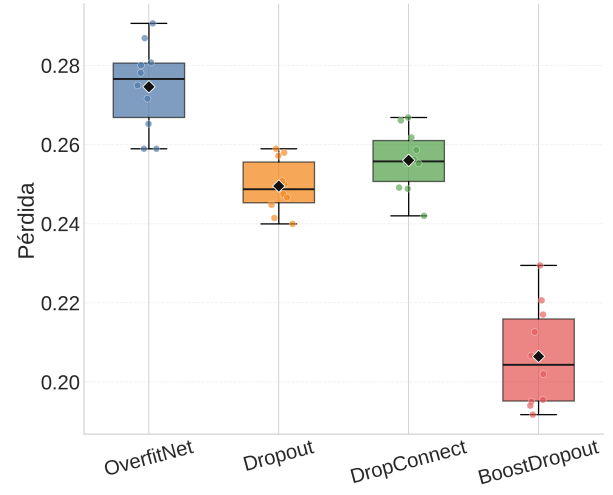
La Figura 18 presenta el mismo análisis anterior, pero ahora utilizando como artefacto seleccionado el modelo determinado por la política de *early stopping*.

La Figura 18 y los Cuadros 9 y 8 muestran que el patrón general observado para el **mejor modelo de validación** se conserva también bajo el criterio de *early stopping*. Nuevamente, *BoostDropoutNet* obtiene los menores valores de pérdida y error en test, mientras que *DropoutNet* y *DropConnectNet* se ubican en una posición intermedia y *OverFitNet* permanece como la arquitectura de peor desempeño global.

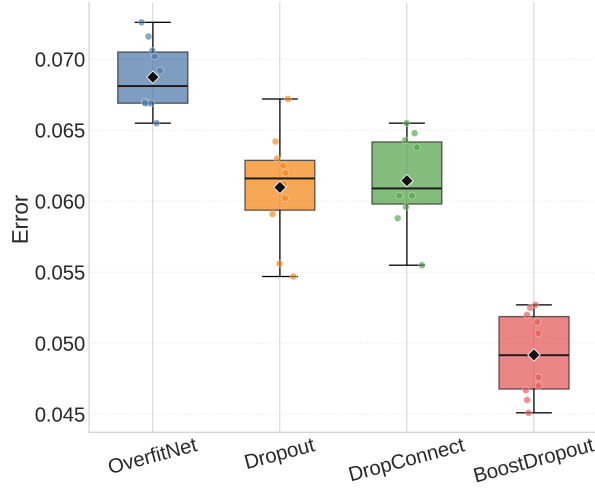
Sin embargo, al comparar esta subsección con la anterior, aparece una diferencia conceptual importante. En *BoostDropout*, el criterio de *early stopping* produce resultados levemente



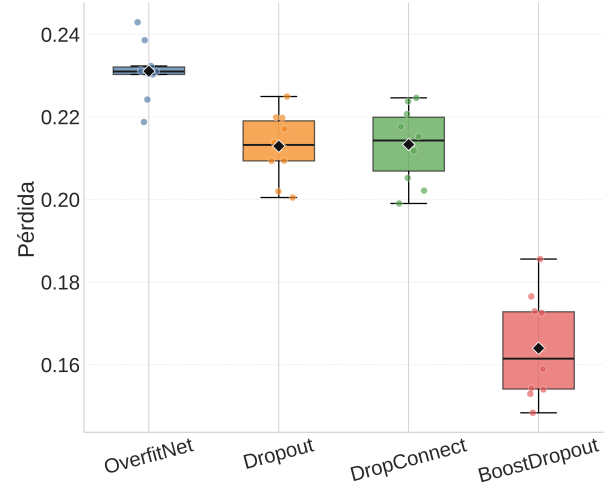
(a) Error de validación.



(b) Pérdida de validación.



(c) Error de test.



(d) Pérdida de test.

Figura 18: Distribución de métricas bajo el criterio de *early stopping*. Nuevamente, el error se muestra a la izquierda y la pérdida a la derecha, tanto para validación como para test.

Cuadro 8: Resumen estadístico de las métricas de **pérdida en test** para el criterio de *early stopping*.

Modelo	Media	Mediana	Desv. est.	Mín.	Máx.
OverFitNet	0.2310	0.231	0.0018	0.2187	0.2429
Dropout	0.2129	0.2132	0.0097	0.2004	0.2249
DropConnect	0.2133	0.2142	0.0131	0.1991	0.22465
BoostDropout	0.1640	0.1614	0.0187	0.1483	0.1856

Cuadro 9: Resumen estadístico de la métrica de error test para el criterio de *early stopping*.

Modelo	Media	Mediana	Desv. est.	Mín.	Máx.
OverFitNet	0.0687	0.0681	0.0023	0.0655	0.0726
Dropout	0.0609	0.0616	0.0036	0.0547	0.0672
DropConnect	0.0614	0.0609	0.0029	0.0555	0.0655
BoostDropout	0.0491	0.0491	0.0028	0.0451	0.0527

peores que el criterio de mejor modelo de validación, tanto en pérdida como en error. Esto sugiere que, para esta arquitectura, las mejoras más significativas tienden a consolidarse en un régimen algo más tardío del entrenamiento, de modo que la detención temprana puede cortar parcialmente una fase en la que la red todavía sigue refinando su desempeño. En cambio, para *OverFitNet*, *DropoutNet* y *DropConnectNet*, la diferencia entre ambos criterios resulta bastante menor, lo que sugiere una dinámica de convergencia más temprana o más alineada con la lógica de *early stopping*.

También aquí conviene interpretar con cuidado la desviación estándar. En la pérdida de test, *BoostDropoutNet* vuelve a exhibir una desviación estándar mayor que la de las demás arquitecturas. No obstante, los boxplots muestran que la caja intercuartílica y los bigotes se mantienen concentrados en una zona visiblemente más baja, sin superposiciones importantes con *OverFitNet* y con una separación todavía clara respecto de *Dropout* y *DropConnect*. Por ello, la mayor dispersión no invalida la conclusión principal: aun admitiendo una variabilidad inter-seed algo más marcada, la distribución de *BoostDropout* sigue estando desplazada hacia un régimen de mejor generalización.

En el caso de *DropoutNet* y *DropConnectNet*, la lectura vuelve a ser similar a la de la subsección anterior. Ambos mejoran de manera estable a la red sin regularización, pero sus distribuciones se mantienen cercanas entre sí, con diferencias pequeñas tanto en las tablas como en la inspección visual de las cajas. Así, el criterio de *early stopping* no altera la conclusión cualitativa central: las dos variantes clásicas resultan efectivas para reducir el sobreajuste de la arquitectura base, aunque no alcanzan el nivel de desempeño de *BoostDropoutNet*.

En síntesis, el análisis bajo *early stopping* refuerza dos ideas. Primero, la regularización mejora de manera sistemática a la red base. Segundo, la arquitectura *BoostDropoutNet* sigue siendo la más favorable, aunque en este caso el propio criterio de selección parece capturar de forma menos completa su potencial que el criterio de **MMV**.

4.1.2.3 Distribución de la mejor época

Además de las métricas finales de desempeño, resulta útil analizar en qué época aparece el artefacto seleccionado en cada corrida. Esto permite comparar no solo la calidad final de los modelos, sino también el régimen temporal en el que cada arquitectura tiende a estabilizar sus mejores resultados.

La Figura 19 y los Cuadros 10 y 11 permiten comparar no solo qué arquitectura obtiene mejores métricas finales, sino también en qué escala temporal aparece el artefacto seleccionado. El resultado más llamativo es el comportamiento de *BoostDropoutNet*, cuya mejor época se ubica claramente más tarde que la del resto de los modelos. Bajo el criterio de mejor modelo de validación, su media se encuentra muy por encima de *OverFitNet*, *DropoutNet*

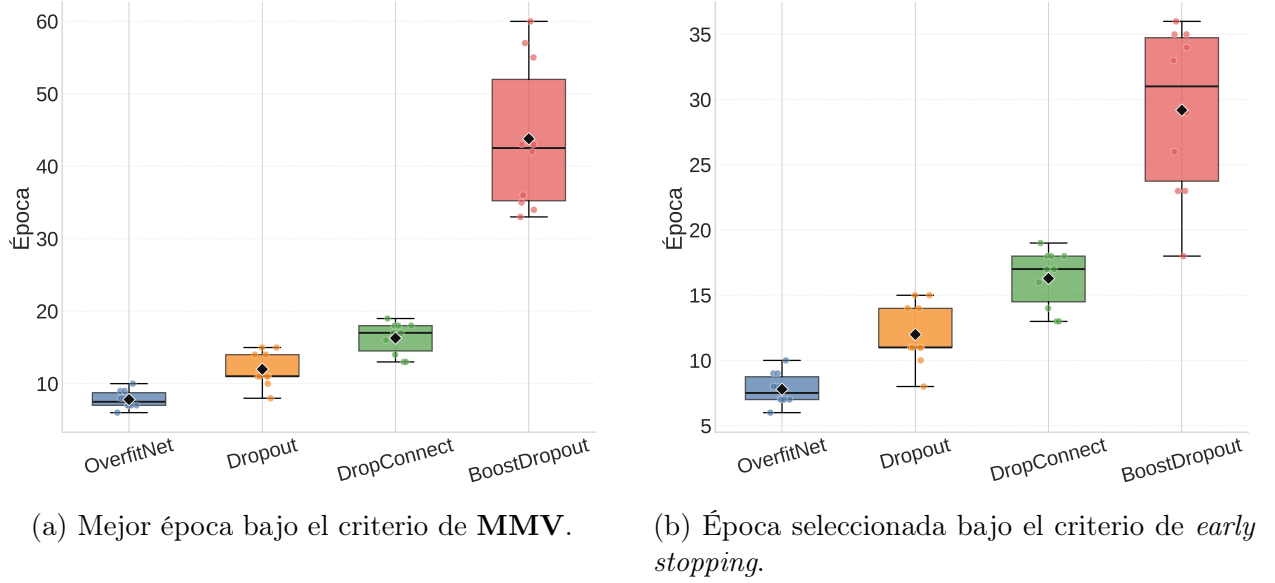


Figura 19: Distribución de la época seleccionada por cada criterio de artefacto.

y *DropConnectNet*; y aun con *early stopping*, sigue siendo la arquitectura que alcanza sus mejores configuraciones más tardíamente.

Este resultado es coherente con la lectura realizada en las subsecciones anteriores. Si *BoostDropoutNet* obtiene las menores pérdidas y errores finales, pero además lo hace en épocas considerablemente más altas, entonces su dinámica de entrenamiento parece más lenta o, al menos, más gradual. En otras palabras, la arquitectura propuesta no solo converge hacia una región de mejor desempeño, sino que tiende a hacerlo más adelante en el entrenamiento. Esto ayuda a entender por qué el criterio de mejor modelo de validación la favorece más que el criterio de *early stopping*: una parte de su ventaja comparativa parece consolidarse precisamente en esa fase tardía.

Cuadro 10: Resumen estadístico de la mejor época bajo el criterio de **MMV**.

Modelo	Media	Mediana	Desv. est.	Mín.	Máx.
OverFitNet	7.8	7.5	1.1	6	10
Dropout	12	11	2.2	8	15
DropConnect	16.3	17	2.1	13	19
BoostDropout	43.8	42.5	9.6	33	60

Cuadro 11: Resumen estadístico de la época seleccionada bajo el criterio de *early stopping*.

Modelo	Media	Mediana	Desv. est.	Mín.	Máx.
OverFitNet	7.8	7.5	1.2	6	10
Dropout	12	11	2.2	8	15
DropConnect	16.3	17	2.1	13	19
BoostDropout	29.2	31	6	18	36

En contraste, *OverFitNet* alcanza sus mejores épocas de manera muy temprana, con valores concentrados en torno a las primeras iteraciones del entrenamiento. *DropoutNet* aparece en una posición intermedia, mientras que *DropConnectNet* requiere en promedio más épocas que *DropoutNet*, pero todavía muy por debajo de *BoostDropoutNet*. Esta jerarquía temporal sugiere que los distintos mecanismos de regularización no solo modifican el nivel final de desempeño, sino también la escala de tiempo en la que cada arquitectura estabiliza su mejor comportamiento.

La dispersión de la mejor época también resulta informativa. En *BoostDropoutNet* la desviación estándar es mayor, especialmente bajo el criterio de **MMV**, lo que indica una mayor sensibilidad inter-seed respecto del momento exacto en que se alcanza el mejor punto del entrenamiento. Sin embargo, el boxplot muestra que esta variabilidad ocurre dentro de un régimen temporal claramente separado del resto de las arquitecturas. Es decir, aunque la época óptima varíe entre corridas, sigue apareciendo sistemáticamente mucho más tarde que en *OverFitNet*, *DropoutNet* y *DropConnectNet*.

En conjunto, la distribución de la mejor época refuerza la interpretación general del entrenamiento principal: *BoostDropoutNet* no solo conduce a mejores métricas finales, sino que lo hace mediante una dinámica distinta, caracterizada por una mejora sostenida durante más épocas. Esta observación será importante al interpretar las curvas representativas de validación de la subsección siguiente.

4.1.2.4 Curvas representativas de validación

Para complementar la información agregada de los boxplots, resulta útil observar curvas de validación representativas. En las figuras siguientes se muestran, para cada criterio de selección, las curvas correspondientes a una corrida representativa por arquitectura, elegida a partir del menor valor de pérdida de validación dentro del conjunto de corridas consideradas.

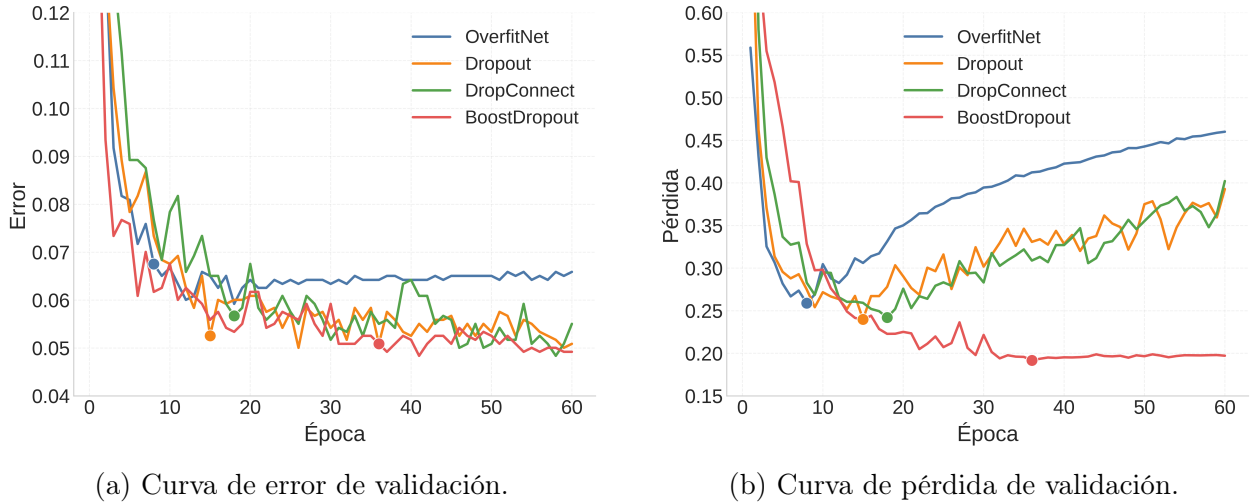


Figura 20: Curvas representativas de validación para el criterio de **MMV**.

Las Figuras 20 y 21 complementan la información agregada de los boxplots al mostrar la dinámica temporal de una corrida representativa por arquitectura, seleccionada en ambos casos a partir de la menor pérdida de validación entre las semillas consideradas. En primer

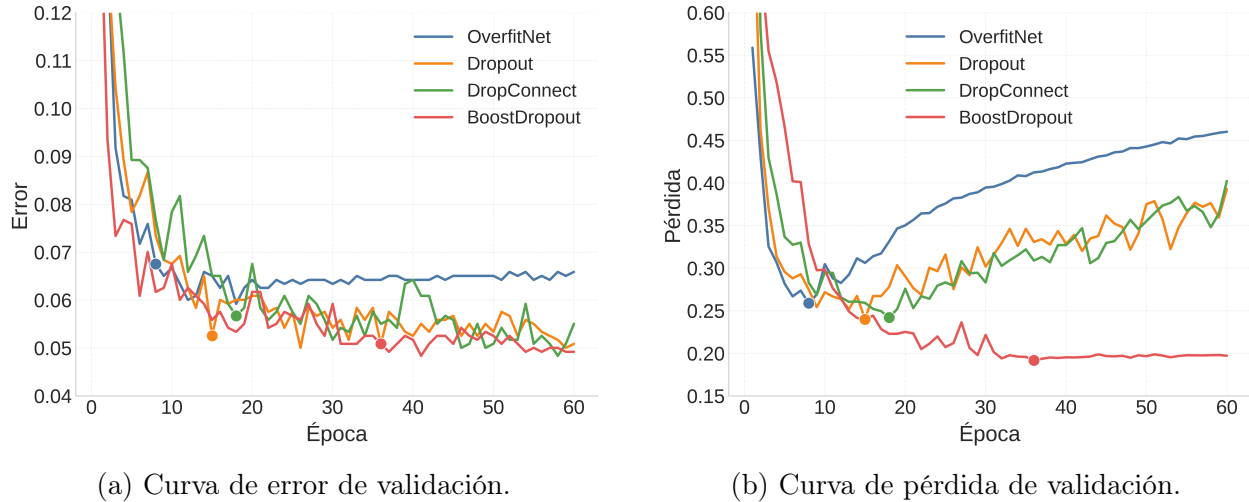


Figura 21: Curvas representativas de validación para el criterio de *early stopping*.

lugar, estas curvas refuerzan con claridad la conclusión obtenida en las subsecciones anteriores: *BoostDropoutNet* alcanza los mejores valores finales de error y, sobre todo, de pérdida en validación. Sin embargo, esa mejora no aparece de manera temprana, sino que se consolida en un régimen más tardío del entrenamiento. En otras palabras, la arquitectura propuesta parece requerir más épocas para desplegar por completo su ventaja comparativa, lo cual resulta coherente con la distribución de mejor época discutida en la Sección 4.1.2.3.

Un segundo aspecto llamativo es que las curvas representativas asociadas al criterio de **MMV** y al criterio de *early stopping* resultan prácticamente indistinguibles. Esto sugiere que, al menos para las corridas representativas seleccionadas en esta visualización, el mismo entrenamiento terminó dando lugar al artefacto elegido bajo ambos criterios. Conviene notar, de todos modos, que esta coincidencia debe interpretarse como una propiedad de las corridas seleccionadas para la figura, y no necesariamente como una identidad exacta entre todos los entrenamientos considerados en el análisis multisesilla.

La diferencia más marcada entre arquitecturas aparece al observar la pérdida de validación en épocas tardías. En *OverFitNet*, la curva desciende con rapidez al comienzo, pero luego comienza a elevarse de manera visible, reproduciendo el patrón típico de sobreajuste discutido en el Capítulo 2: una mejora inicial seguida por un deterioro progresivo de la validación a medida que el entrenamiento continúa. Este comportamiento se atenúa en *DropoutNet* y *DropConnectNet*, donde la subida tardía de la pérdida sigue presente pero resulta menos pronunciada. En ese sentido, ambas regularizaciones clásicas logran amortiguar el sobreajuste de la arquitectura base, aun cuando no lo eliminan por completo.

El caso de *BoostDropoutNet* es cualitativamente distinto. En su curva de pérdida de validación no se observa el ascenso tardío que sí aparece en las demás arquitecturas; por el contrario, la evolución permanece mucho más estable y prácticamente no muestra señales visibles de deterioro en las épocas finales. Esta estabilidad constituye uno de los resultados más interesantes del protocolo principal, ya que sugiere que la modulación multiplicativa introducida por *BoostDropoutNet* no solo mejora el nivel final de desempeño, sino que también altera favorablemente la dinámica del entrenamiento, haciendo menos evidente el proceso de

sobreajuste.

Algo similar puede apreciarse en las curvas de error de validación. Aunque las diferencias entre arquitecturas son más sutiles que en la pérdida, *BoostDropoutNet* vuelve a mantenerse en el rango más bajo hacia el final del entrenamiento, y lo hace sin exhibir fluctuaciones tardías importantes. Esto refuerza la idea de que la principal ventaja del método no reside únicamente en obtener un mínimo más bajo, sino también en sostener un régimen de validación más estable a lo largo de las épocas.

En conjunto, estas curvas aportan una lectura dinámica que complementa la información resumida en tablas y boxplots. *OverFitNet* muestra el patrón más claro de sobreajuste; *DropoutNet* y *DropConnect* reducen ese efecto, pero todavía conservan una tendencia ascendente en la pérdida de validación hacia el final; y *BoostDropoutNet* se destaca no solo por alcanzar mejores valores finales, sino también por hacerlo mediante una trayectoria de validación más estable y menos compatible con un sobreajuste visible.

4.1.3 Dependencia de las métricas respecto del parámetro p

Una vez comparadas las arquitecturas bajo sus configuraciones seleccionadas, resulta natural analizar con mayor detalle cómo varía el desempeño al modificar la probabilidad p , que controla la intensidad del enmascaramiento o de la modulación estocástica según el método considerado. Las figuras siguientes muestran la evolución de error y pérdida de test al variar dicho parámetro.

4.1.3.1 Dependencia respecto de p para el criterio MMV

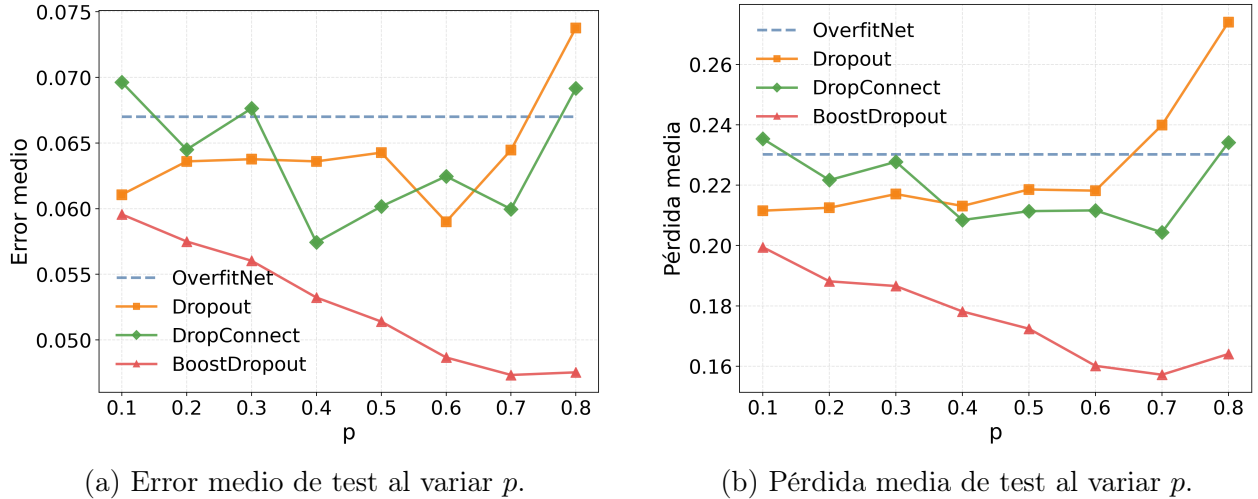


Figura 22: Dependencia de las métricas de test respecto de p bajo el criterio de MMV.

Las curvas de la Figura 22 fueron obtenidas a partir de entrenamientos adicionales realizados fuera de la etapa de *grid search*, con el objetivo de ampliar el rango de valores explorados para p manteniendo fijos los mejores hiperparámetros seleccionados previamente para cada arquitectura. En este caso, para cada valor de p se entrenaron tres semillas experimenta-

les durante 40 épocas, y las métricas reportadas corresponden al promedio sobre esas tres corridas.

El resultado más claro de la figura es que la curva de *BoostDropoutNet* se mantiene sistemáticamente por debajo de las curvas de referencia en una amplia región del eje p , tanto en error como en pérdida de test. Esto indica que, una vez fijada una configuración adecuada de λ , la variante propuesta no solo mejora en un punto aislado del espacio de hiperparámetros, sino que conserva una ventaja estable frente a *OverFitNet*, *Dropout* y *DropConnect* al variar la frecuencia de modulación.

Dentro de ese recorrido, el mejor comportamiento de *BoostDropoutNet* se observa alrededor de $p = 0,7$, en concordancia con la configuración seleccionada en la etapa de exploración inicial. Este punto es importante porque refuerza la consistencia entre ambas etapas experimentales: el valor de p elegido por el *grid search* no aparece como una selección arbitraria, sino como el entorno donde la arquitectura propuesta vuelve a alcanzar su mejor desempeño al ampliar el espectro de exploración.

Por contraste, las arquitecturas *DropoutNet* y *DropConnectNet* muestran una sensibilidad más moderada a p , con curvas relativamente más cercanas entre sí y sin una separación comparable a la observada para *BoostDropoutNet*. *OverFitNet*, al no depender estructuralmente de este hiperparámetro, funciona en esta figura como una referencia horizontal útil para contextualizar la magnitud de las mejoras observadas.

En conjunto, la Figura 22 sugiere que la dependencia respecto de p no solo confirma la superioridad relativa de *BoostDropoutNet*, sino que también muestra que dicha ventaja es robusta en torno al valor óptimo encontrado. En otras palabras, la arquitectura propuesta no parece beneficiarse de un ajuste extremadamente fino de p , sino que mantiene un régimen favorable en una zona relativamente amplia del espacio explorado.

4.1.3.2 Dependencia respecto de p para el criterio de *early stopping*

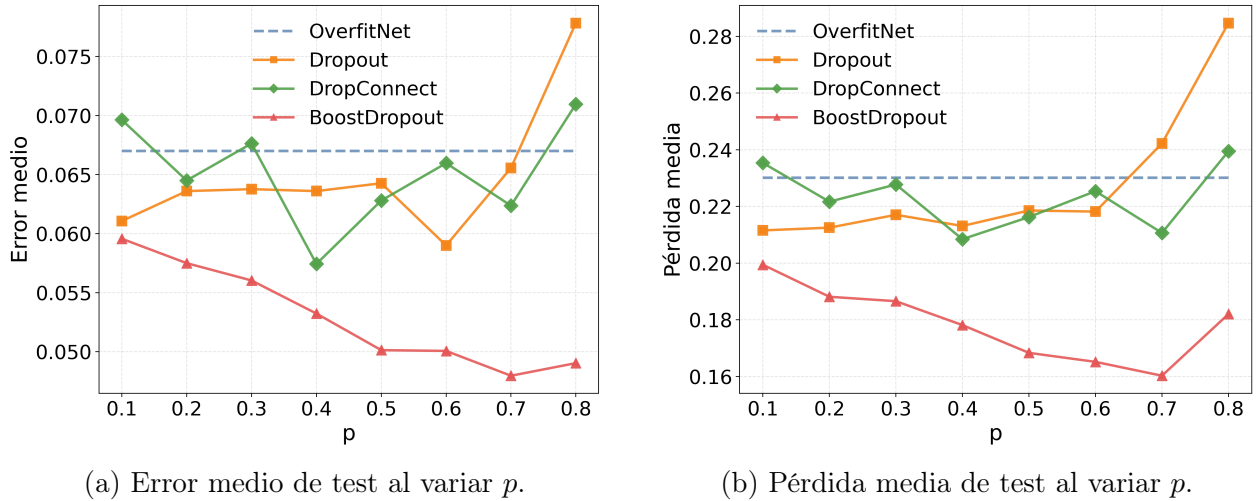


Figura 23: Dependencia de las métricas de test respecto de p bajo el criterio de *early stopping*.

La Figura 23 reproduce el mismo análisis anterior, pero utilizando como criterio de selección el artefacto determinado por *early stopping*. También aquí los resultados provienen de

entrenamientos adicionales de 40 épocas con tres semillas por valor de p , promediadas para construir cada punto de las curvas.

El patrón general se conserva con notable claridad: *BoostDropoutNet* vuelve a ocupar la región inferior de la figura en la mayor parte del rango explorado, tanto para el error como para la pérdida de test. Esta estabilidad refuerza la idea de que la mejora asociada al método no depende exclusivamente del criterio de mejor modelo de validación, sino que sigue siendo visible incluso cuando la selección del modelo se realiza mediante detención temprana.

Al igual que en la subsección anterior, el valor $p = 0,7$ vuelve a destacarse como la zona de mejor desempeño para *BoostDropoutNet*. La repetición de este resultado bajo ambos criterios de selección constituye una señal favorable de consistencia experimental: el hiperparámetro óptimo no parece cambiar de forma relevante al pasar de un criterio al otro, lo que simplifica la interpretación del comportamiento del método.

Las curvas de *Dropout* y *DropConnectNet* se mantienen nuevamente cercanas entre sí y por encima de *BoostDropoutNet*, mientras que *OverFitNet* actúa como referencia base. De este modo, la lectura cualitativa no se modifica: la principal separación sigue estando dada por la arquitectura propuesta frente al resto de los modelos, más que por diferencias marcadas entre los dos métodos clásicos de regularización.

En síntesis, el análisis de sensibilidad respecto de p bajo *early stopping* refuerza las conclusiones obtenidas para el mejor modelo de validación. La ventaja de *BoostDropoutNet* persiste en una región amplia del parámetro, y el valor $p = 0,7$ vuelve a aparecer como el punto de mejor compromiso dentro del rango estudiado.

4.1.4 Dependencia de las métricas respecto de λ

Además de las dependencias ya estudiadas respecto de p , en el caso de *BoostDropout* también resulta natural analizar la dependencia respecto de λ , parámetro introducido específicamente por este método y encargado de controlar la intensidad de la modulación multiplicativa. Los demás modelos considerados no dependen de λ ; por eso, en los gráficos correspondientes aparecen como líneas horizontales. Esas líneas representan el mejor desempeño global alcanzado por cada modelo y se utilizan como referencia para comparar cómo varía *BoostDropout* al modificar λ .

4.1.4.1 Criterio: mejor modelo de validación

La Figura 24 muestra la dependencia de las métricas de test respecto del parámetro λ , manteniendo fijos los mejores hiperparámetros encontrados previamente y promediando, para cada valor de λ , tres corridas de 40 épocas. A diferencia del caso de p , aquí la variación efectiva afecta únicamente a *BoostDropoutNet*. Por esa razón, las curvas correspondientes a *OverFitNet*, *DropoutNet* y *DropConnectNet* aparecen como líneas horizontales de referencia.

El resultado más importante es que *BoostDropoutNet* mejora la pérdida de test respecto de las arquitecturas de referencia aproximadamente en el intervalo $0,3 \lesssim \lambda \lesssim 1,5$. En esa región, la curva roja se ubica por debajo de las otras tres, indicando que la modulación multiplicativa controlada por λ permite acceder a un régimen de generalización más favorable que el alcanzado por *DropoutNet*, *DropConnectNet* y la red base sin regularización estocástica.

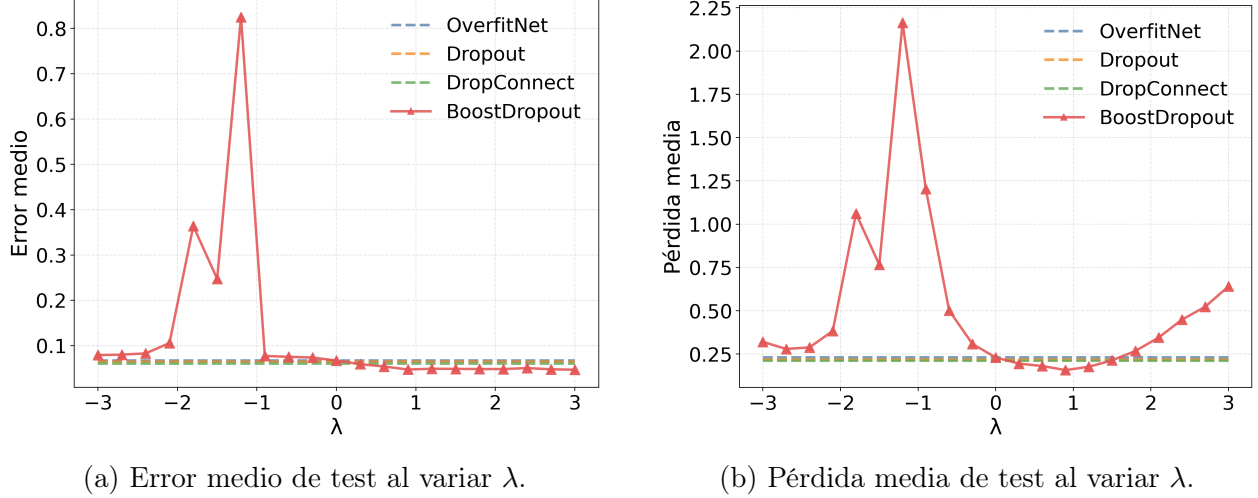


Figura 24: Dependencia de las métricas de *BoostDropout* respecto de λ bajo el criterio **MMV**. Las curvas de referencia se encuentran muy juntas entre si, debido a las escalas de los ejes verticales de cada imagen producto de los valores elevados que toman las métricas de *BoostDropout* para ciertos valores de λ .

Fuera de ese intervalo, el comportamiento cambia. Para valores grandes y positivos de λ , el error puede seguir siendo relativamente bajo, pero la pérdida de test comienza a aumentar. Esto sugiere una situación interesante: la arquitectura conserva una buena capacidad de clasificación en términos de aciertos, pero el modelo se vuelve probabilísticamente más costoso según la entropía cruzada. En otras palabras, aun cuando la decisión final siga siendo correcta con frecuencia, la confianza o calibración implícita de las salidas deja de ser tan favorable. Por esa razón, no resulta conveniente interpretar una mejora en error como evidencia suficiente de mejor desempeño cuando la pérdida ya muestra un deterioro claro.

En la región negativa del eje aparece otro fenómeno destacable. Entre $\lambda \approx -1$ y $\lambda \approx -2$, tanto el error como la pérdida empeoran de manera muy marcada, hasta el punto de expandir fuertemente la escala vertical de la figura y comprimir visualmente las curvas de referencia. Este deterioro es metodológicamente interesante porque en $\lambda = -1$ la máscara *BoostDropout* recupera formalmente una variable de tipo Bernoulli sobre $\{0, 1\}$, pero en estos experimentos se mantuvo desactivada la normalización de máscara. Por lo tanto, aunque la estructura binaria recuerde a *dropout*, la implementación no preserva la escala esperada de las activaciones, y el desempeño resultante se degrada de manera considerable. Este comportamiento ilustra de forma empírica que la normalización es indispensable para reproducir adecuadamente el régimen de *dropout* clásico.

Más aún, para $\lambda < -1$ el factor multiplicativo $1 + \lambda$ se vuelve negativo, lo que implica una inversión del signo de las activaciones moduladas. En ese régimen, la capa deja de funcionar como un simple mecanismo de atenuación o supresión y pasa a introducir una inversión explícita de señal, lo cual altera de manera cualitativa la dinámica de propagación. La fuerte degradación observada en esa zona es compatible con este cambio de régimen. Aun así, la curva sugiere que cerca de $\lambda \approx -2,5$ puede aparecer un mínimo local de la pérdida, antes de que el desempeño vuelva a deteriorarse para valores todavía menores. Este detalle sugiere que la dependencia respecto de λ no es monótona y que el parámetro controla transiciones

entre regímenes de comportamiento cualitativamente distintos.

En conjunto, la Figura 24 muestra que el parámetro λ cumple un papel central en *BoostDropout*: existe una región positiva amplia donde el método supera con claridad a las arquitecturas de referencia, mientras que regiones negativas particulares introducen fenómenos mucho menos favorables, especialmente cuando la máscara no se normaliza.

4.1.4.2 Criterio: *early stopping*

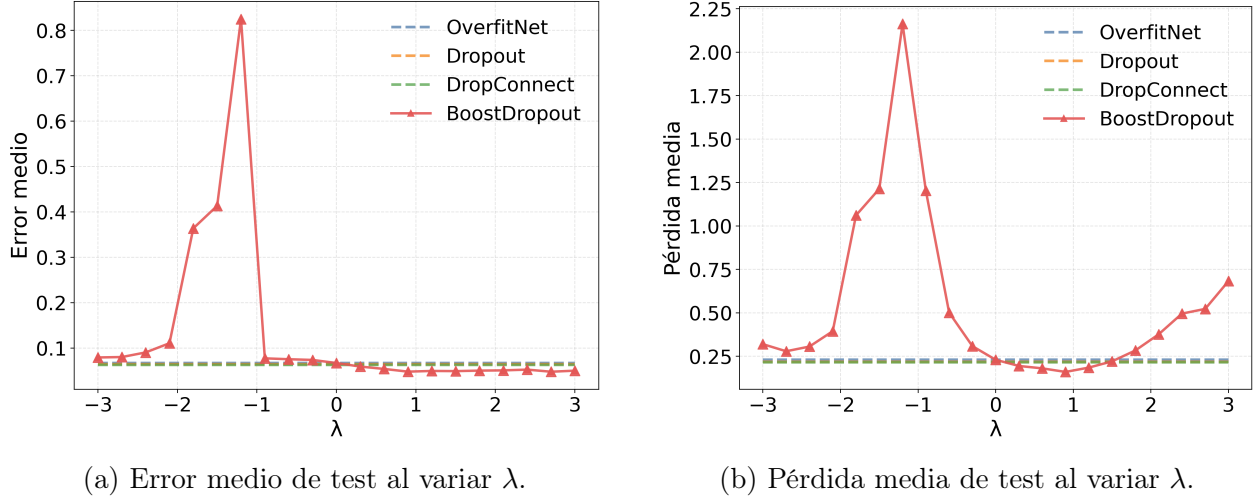


Figura 25: Dependencia respecto de λ bajo el criterio de *early stopping*. Las curvas de referencia se encuentran muy juntas entre si, debido a las escalas de los ejes verticales de cada imagen producto de los valores elevados que toman las métricas de *BoostDropout* para ciertos valores de λ .

La Figura 25 confirma, bajo el criterio de *early stopping*, el mismo panorama cualitativo observado para el mejor modelo de validación. Nuevamente, las curvas de *OverFitNet*, *DropoutNet* y *DropConnectNet* permanecen constantes con λ , mientras que la dependencia efectiva se concentra en *BoostDropoutNet*.

También aquí la región positiva de λ contiene los mejores resultados de la arquitectura propuesta. En particular, el intervalo aproximado $0,3 \lesssim \lambda \lesssim 1,5$ vuelve a destacarse como la zona en la que la pérdida de test de *BoostDropoutNet* permanece por debajo de las tres curvas de referencia. Esta coincidencia con la subsección anterior refuerza la idea de que no se trata de un efecto accidental ligado a un criterio de selección específico, sino de una propiedad robusta del método dentro del protocolo explorado.

Del mismo modo, para valores muy positivos de λ se mantiene la diferencia entre error y pérdida: el error puede continuar siendo competitivo, pero la pérdida aumenta. Este desacople entre ambas métricas vuelve a sugerir que, más allá de cierto punto, el modelo conserva capacidad de clasificación correcta pero pierde calidad probabilística en la salida. Por ello, la región de mejor comportamiento práctico no debe definirse únicamente por el mínimo error, sino por un compromiso entre ambas métricas.

En la zona negativa reaparece con la misma fuerza el deterioro abrupto alrededor de $\lambda = -1$ y $\lambda \approx -2$. Este resultado es especialmente elocuente porque conecta directamente

con la formulación presentada en la Sección 2.9: cuando $\lambda = -1$, la máscara adquiere una forma Bernoulli, pero sin la normalización dada en la ecuación (2.9.5) no se recupera el comportamiento efectivo de *inverted dropout*. En consecuencia, la degradación observada no es accidental, sino coherente con la ausencia de reescala en una zona donde dicha corrección resulta esencial.

Para $\lambda < -1$, la inversión del signo de activaciones vuelve a introducir un régimen cualitativamente distinto, acompañado de un fuerte deterioro del desempeño. La posible aparición de un mínimo local cerca de $\lambda \approx -2,5$, seguida de un nuevo ascenso de la pérdida para valores más negativos, refuerza la idea de que el parámetro λ no induce una dependencia simple ni lineal, sino una sucesión de regímenes con comportamiento diferente.

En síntesis, la Figura 25 reproduce el mensaje central de la subsección anterior: *BoostDropout* supera a las arquitecturas de referencia en una región positiva bien definida de λ , mientras que los valores negativos —en especial cerca de $\lambda = -1$ sin normalización— ponen en evidencia la sensibilidad del método a la escala efectiva de las activaciones y la relevancia estructural de la normalización en el caso límite vinculado a *dropout*.

Comentario metodológico sobre calibración posterior.

En la Sección 2.6.1.2 se introdujo la posibilidad de comparar modelos mediante calibración posterior por *temperature scaling*. Sin embargo, esta técnica no se aplicó en este trabajo. La razón principal es que su efecto se limita a reescalar los logits para modificar la distribución de probabilidades predichas, sin alterar la clase asignada por la regla de decisión $\arg \max$. En consecuencia, *temperature scaling* puede cambiar la *cross-entropy loss*, pero no modifica el error de clasificación.

Desde el punto de vista de esta tesis, este punto es importante porque *BoostDropoutNet* no solo obtuvo menor pérdida que las arquitecturas de referencia, sino también menor error. Por lo tanto, aun en el caso de que una calibración posterior redujera parcialmente la pérdida de *OverFitNet*, *DropoutNet* o *DropConnectNet*, esas arquitecturas seguirían sin poder competir con *BoostDropoutNet* en términos de error. En ese sentido, la calibración habría afectado únicamente una parte de la comparación, pero no el resultado cualitativo principal del capítulo.

Por esta razón, *temperature scaling* se mantiene aquí como una observación metodológica útil y como una posible herramienta diagnóstica para trabajos posteriores, pero no como un paso necesario dentro del protocolo experimental adoptado.

4.1.5 Conclusiones del protocolo principal

A partir de los resultados presentados en esta sección, puede extraerse una imagen global coherente del comportamiento de las arquitecturas comparadas bajo el protocolo principal de clasificación. En primer lugar, la regularización estocástica mejora de manera sistemática el desempeño de la red base sin regularización. Tanto *DropoutNet* como *DropConnectNet* reducen el error y la pérdida respecto de *OverFitNet*, y además atenúan el patrón de sobreajuste visible en la evolución de la validación. En ese sentido, los resultados experimentales confirman que la arquitectura base efectivamente se beneficia de mecanismos de perturbación multiplicativa durante el entrenamiento.

Sin embargo, la contribución más importante del capítulo es que la variante propuesta *BoostDropoutNet* se destaca de manera consistente por encima de las otras arquitecturas. Esta superioridad aparece en distintos niveles de análisis: en las tablas resumen del entrenamiento principal, en los boxplots de validación y test, en la distribución de la mejor época, en las curvas representativas de validación y también en los estudios adicionales de sensibilidad respecto de p y de λ . En otras palabras, la ventaja de *BoostDropoutNet* no depende de un único criterio o de una sola visualización, sino que se repite a lo largo de toda la sección.

Desde el punto de vista de las métricas finales, *BoostDropoutNet* obtiene en general la menor *cross-entropy loss* y también el menor error, aunque la diferencia resulta especialmente marcada en la pérdida. Esto es relevante porque la pérdida, al ser sensible al grado de confianza de las predicciones, ofrece una medida más fina del comportamiento probabilístico del modelo que el error de clasificación por sí solo. Bajo esta óptica, la arquitectura propuesta no solo clasifica mejor, sino que además lo hace con un perfil probabilístico más favorable dentro del régimen donde sus hiperparámetros están bien ajustados.

Desde el punto de vista dinámico, *BoostDropoutNet* también exhibe un comportamiento distintivo. Las curvas representativas muestran que su mejor desempeño se alcanza más tarde que en las demás arquitecturas, lo cual coincide con la distribución de la mejor época observada en la Subsección 4.1.2.3. Esto sugiere que la mejora inducida por el método no es meramente un desplazamiento instantáneo de las curvas, sino el resultado de una dinámica de entrenamiento más gradual. Al mismo tiempo, dicha dinámica se mantiene notablemente estable: mientras *OverFitNet* muestra un sobreajuste tardío muy claro y *DropoutNet* y *DropConnectNet* lo amortiguan solo parcialmente, en *BoostDropoutNet* ese deterioro de la pérdida de validación se vuelve mucho menos visible.

Los estudios de sensibilidad agregan información complementaria importante. Por un lado, la exploración ampliada respecto de p muestra que *BoostDropoutNet* se mantiene por debajo de las arquitecturas de referencia en una región amplia y alcanza su mejor comportamiento alrededor de $p = 0,7$, en acuerdo con el valor elegido en la etapa de *grid search*. Por otro lado, la exploración en λ muestra que existe un intervalo positivo aproximadamente entre 0,3 y 1,5 donde la pérdida de test mejora con claridad respecto de *OverFitNet*, *DropoutNet* y *DropConnectNet*. Esto refuerza la idea de que la modulación multiplicativa de amplitud constituye una extensión útil del esquema clásico de enmascaramiento binario.

A su vez, la exploración sobre λ pone de manifiesto un aspecto conceptual muy interesante del método. La degradación observada alrededor de $\lambda = -1$ cuando la máscara no se normaliza muestra de manera concreta que la corrección de escala resulta estructuralmente indispensable en el caso límite asociado a *dropout*. Más aún, para $\lambda < -1$ aparece un cambio cualitativo de régimen vinculado al cambio de signo de las activaciones moduladas, acompañado de un deterioro pronunciado del desempeño. Estos resultados no solo ayudan a interpretar el comportamiento de *BoostDropoutNet*, sino que también iluminan retrospectivamente el papel que cumple la normalización en métodos clásicos como *dropout*.

En conjunto, el capítulo muestra que *BoostDropoutNet* puede entenderse como una extensión efectiva de la familia de regularizaciones multiplicativas estudiadas en esta tesis. Frente a *DropoutNet* y *DropConnectNet*, no solo ofrece un mejor desempeño final en el protocolo analizado, sino también una dinámica de validación más estable y una estructura de sensibilidad a hiperparámetros que resulta informativa desde el punto de vista conceptual. Dentro de los alcances y limitaciones del diseño experimental adoptado, estos resultados constituyen una

evidencia favorable para la hipótesis central del trabajo: reemplazar la supresión binaria por una modulación multiplicativa controlada puede conducir a una regularización más robusta y a una mejor generalización en redes neuronales *feed-forward*.

4.2 Visualización cualitativa con autoencoder

Además del protocolo principal de clasificación, se presenta aquí el análisis cualitativo basado en autoencoders introducido en la Sección 3.3. El propósito de esta parte no es comparar exhaustivamente métricas de reconstrucción, sino inspeccionar cómo distintos mecanismos de regularización modifican la estructura de las representaciones internas aprendidas por el modelo.

Con el fin de facilitar la comparación visual, en todas las figuras de esta sección se mantiene el mismo orden de paneles: arriba a la izquierda se ubica el modelo base sin regularización, arriba a la derecha *Dropout*, abajo a la izquierda *DropConnect* y abajo a la derecha *BoostDropout*.

Desempeño de reconstrucción y ejemplos reconstruidos

Antes de pasar a la inspección cualitativa de las representaciones internas, conviene resumir brevemente el desempeño de reconstrucción de los autoencoders analizados.

La Tabla 12 resume las métricas de reconstrucción obtenidas en entrenamiento y en test, medidas mediante entropía cruzada binaria por ejemplo (*BCE per example*) y error cuadrático medio por ejemplo (*MSE per example*).

Cuadro 12: Resumen de métricas de reconstrucción del autoencoder para las cuatro arquitecturas consideradas.

Modelo	Train BCE	Train MSE	Test BCE	Test MSE
sin regularización	50.15	0.801	50.10	0.854
con <i>Dropout</i>	62.34	4.377	62.13	4.375
con <i>DropConnect</i>	59.36	3.574	58.96	3.527
con <i>BoostDropout</i>	61.58	2.178	61.51	2.244

Desde un punto de vista puramente numérico, la arquitectura sin regularización obtiene las mejores métricas de reconstrucción. Entre los modelos regularizados, *DropConnect* alcanza la menor *BCE*, mientras que *BoostDropout* obtiene un *MSE* sensiblemente menor que *Dropout* y *DropConnect*. En consecuencia, ninguna de las variantes regularizadas supera a la variante sin regularización en reconstrucción bruta. Esto no contradice el objetivo de esta parte de la tesis: aquí el interés principal no es optimizar el desempeño reconstructivo, sino utilizar el autoencoder como herramienta para estudiar la organización interna de las representaciones aprendidas.

La Figura 26 muestra ejemplos concretos de reconstrucción. En la primera fila se presentan imágenes de entrada tomadas del conjunto de test, mientras que en las filas inferiores se muestran las reconstrucciones producidas por cada una de las cuatro arquitecturas.

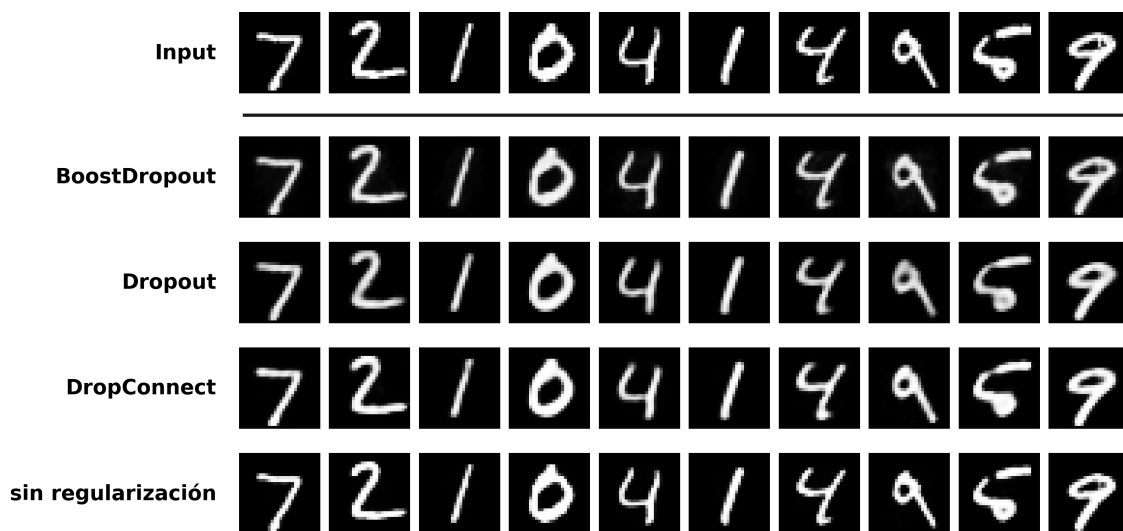


Figura 26: Comparación cualitativa de reconstrucciones del autoencoder. La primera fila contiene ejemplos de entrada y las filas siguientes muestran las reconstrucciones obtenidas por Dropout, DropConnect, BoostDropout y sin regularización.

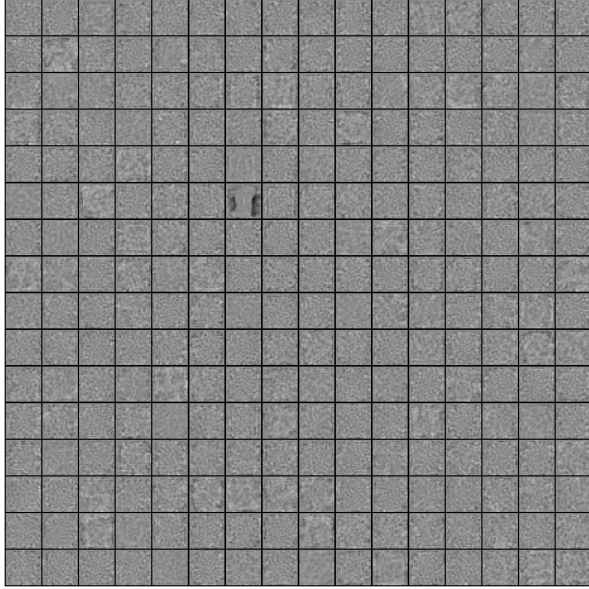
A nivel visual, todas las arquitecturas reconstruyen correctamente la estructura global de los dígitos y preservan de manera razonable los rasgos más relevantes de cada ejemplo. Por ello, esta figura cumple principalmente una función contextual: muestra que los modelos aprendieron una representación útil del conjunto y que las diferencias que se discutirán en las subsecciones siguientes no deben interpretarse como fallas gruesas de reconstrucción, sino como diferencias en la organización interna de las *features* y de las activaciones latentes.

4.2.1 Visualización de coadaptaciones

La Figura 27 reúne las visualizaciones de coadaptación obtenidas para las cuatro variantes analizadas. Este tipo de figura permite inspeccionar, de manera cualitativa, hasta qué punto las unidades internas parecen depender de patrones compartidos o exhiben estructuras más independientes y reutilizables.

Cada panel de esta figura contiene una grilla de 16×16 imágenes, una por cada neurona del espacio latente. Dado que se trata de la primera capa del codificador, estas visualizaciones pueden interpretarse directamente a partir de los pesos que conectan la entrada con cada unidad oculta: al reorganizar esos pesos sobre la grilla espacial original de MNIST, se obtiene una representación aproximada de la característica o *feature* aprendida por cada neurona.

Desde este punto de vista, los resultados obtenidos para la arquitectura sin regularización y para aquella que incorpora *Dropout* son cualitativamente consistentes con lo reportado en el trabajo clásico de Srivastava et al. [1]. En la arquitectura sin regularización predominan patrones ruidosos y poco localizados, mientras que en *Dropout* emergen estructuras más nítidas y reconocibles. En particular, aparecen pequeños *spots* oscuros o claros, semejantes a manchas localizadas —casi como pequeñas “manchas solares” (*sunspots*) sobre un fondo tenue—, lo cual refuerza la impresión de que las unidades aprenden detectores más específicos



(a) sin regularización.

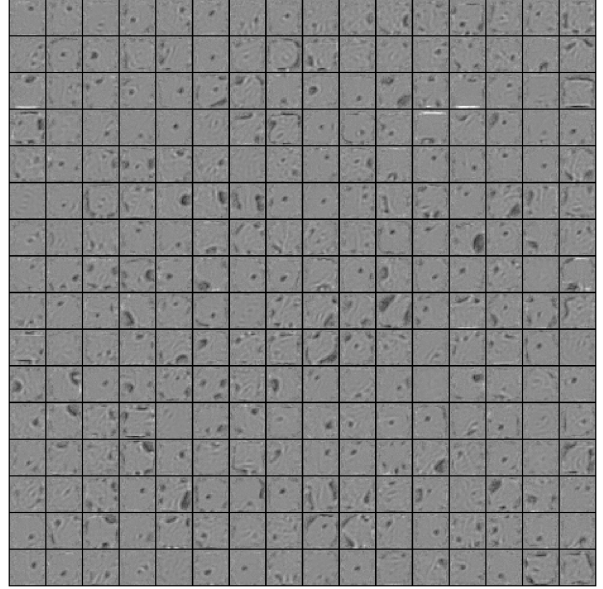
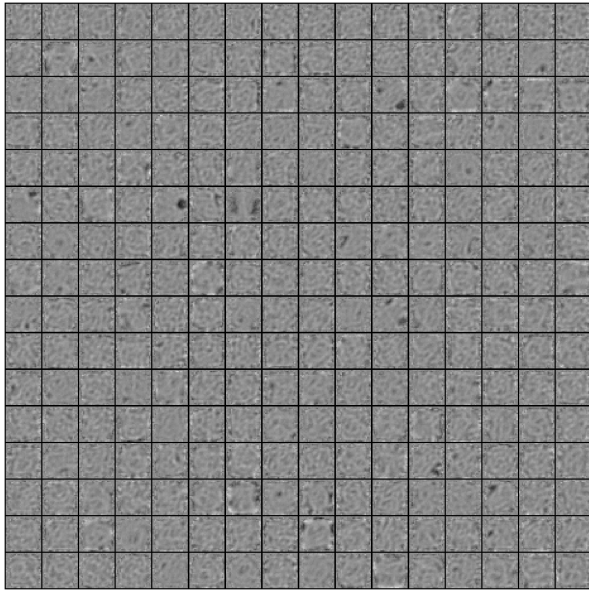
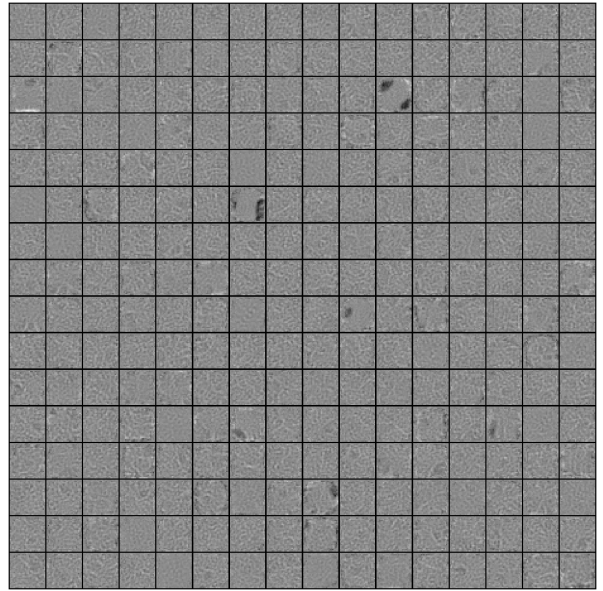
(b) *Dropout*.(c) *DropConnect*.(d) *BoostDropout*.

Figura 27: Comparación cualitativa de coadaptaciones en el autoencoder para las cuatro arquitecturas consideradas.

y menos coadaptados.

En el caso de *DropConnect* el panorama resulta intermedio. Sigue habiendo una componente de ruido apreciable, pero comienzan a emerger estructuras que no son idénticas a las de *Dropout*. Se observan también *spots* localizados, aunque acompañados en varios casos por franjas débiles o patrones más extendidos. Esto sugiere una organización menos difusa que cuando no hay regularización, pero todavía menos localizada que en *Dropout*.

El caso de *BoostDropout* es el más llamativo. En esta visualización predominan patrones cercanos al ruido uniforme, con apenas algunos *spots* intensos aislados. Si se juzgara únicamente por esta figura, podría decirse que *BoostDropout* no exhibe la misma firma visual asociada a ruptura de coadaptaciones que sí aparece con claridad en *Dropout*. Sin embargo, conviene ser cautelosos: la presencia de detectores localizados constituye una evidencia visual a favor de una menor coadaptación, pero la ausencia de esa firma no demuestra por sí misma que las coadaptaciones persistan. Esta figura debe entenderse entonces como una herramienta cualitativa de inspección, no como una prueba concluyente.

Aun con esa cautela, el contraste entre arquitecturas resulta sugerente. *Dropout* parece atacar el fenómeno de coadaptación de una manera visualmente muy marcada, mientras que *DropConnect* y, sobre todo, *BoostDropout*, podrían estar regularizando por mecanismos de naturaleza diferente. Una hipótesis razonable es que la ruptura de coadaptaciones en *Dropout* favorezca también una mayor especialización funcional de las unidades y, con ello, una mayor esparsidad representacional. En cambio, si las *features* aprendidas son menos localizadas —como parece ocurrir en *DropConnect* y *BoostDropout*—, podría esperarse que un mismo *input* active un número mayor de unidades. Esta interpretación es todavía especulativa, pero resulta interesante porque anticipa lo que se observa más adelante en la sección dedicada a *sparsity* representacional.

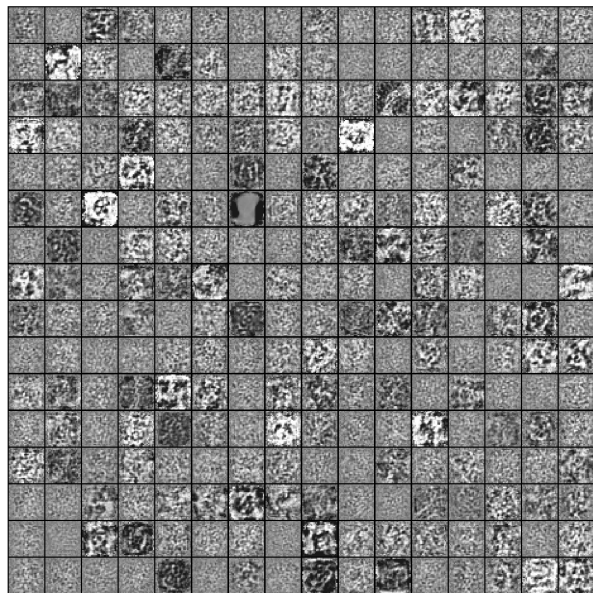
4.2.2 Decodificaciones a partir de neuronas latentes individuales

La figura siguiente muestra, para cada modelo, las reconstrucciones obtenidas al activar individualmente neuronas del espacio latente y propagar luego esa activación a través del decodificador. Esta visualización permite interpretar de manera aproximada la contribución de cada unidad latente a la reconstrucción y ofrece una ventana cualitativa sobre la estructura interna de la representación aprendida.

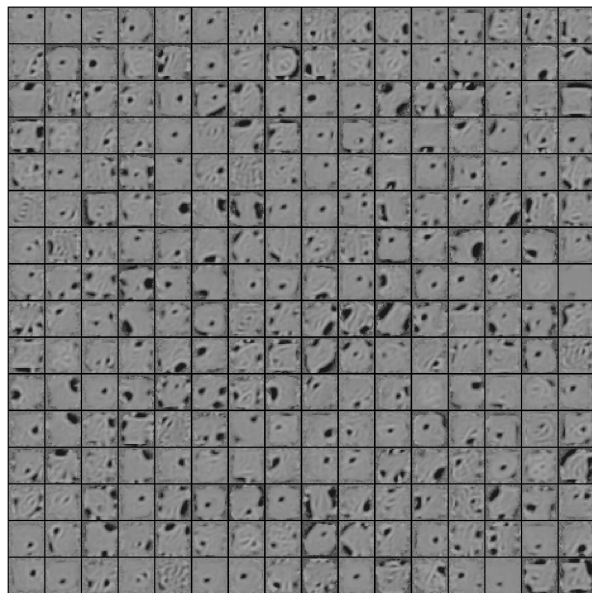
En términos cualitativos, el resultado es muy consistente con la visualización de la sección anterior: las imágenes generadas por neuronas individuales conservan, en buena medida, el mismo tipo de estructura que ya se advertía al inspeccionar las *features* asociadas a los pesos del codificador.

Además, varias imágenes presentan similitudes entre arquitecturas en posiciones equivalentes de la grilla. Esto no debe interpretarse como una identidad exacta entre modelos, pero sí puede estar favorecido por el hecho de que se utilizó la misma semilla de inicialización y entrenamiento, lo cual tiende a alinear parcialmente ciertas direcciones relevantes del aprendizaje y puede hacer que algunas *features* coincidan de manera aproximada entre arquitecturas.

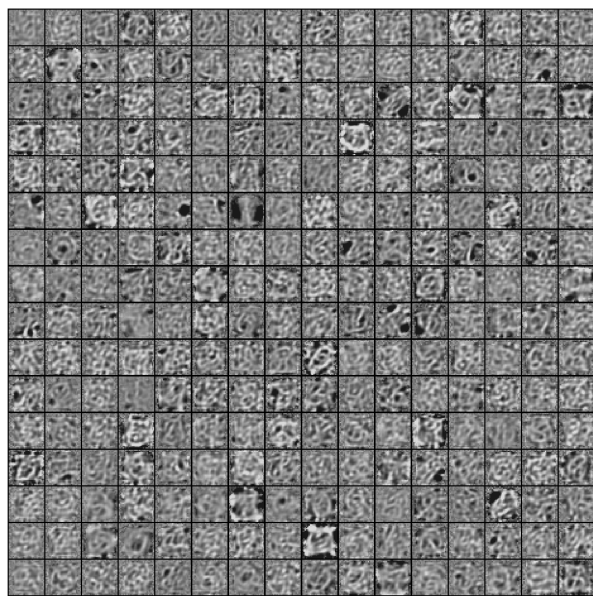
En *Dropout*, la correspondencia con la Figura 27 es particularmente clara. Al inspeccionar una coordenada fija de la grilla, suelen reaparecer prácticamente los mismos *spots* o patrones localizados que se observaban en la visualización de pesos, con diferencias menores. Esto



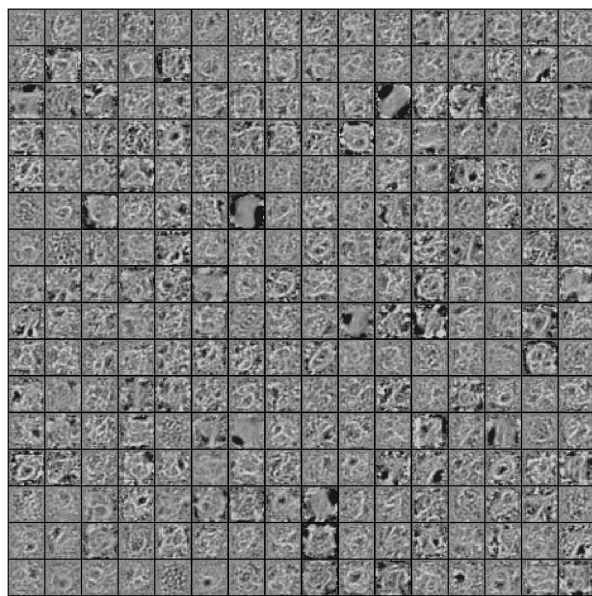
(a) sin regularización.



(b) *Dropout*.



(c) *DropConnect*.



(d) *BoostDropout*.

Figura 28: Reconstrucciones obtenidas al activar individualmente neuronas del espacio latente.

sugiere que, en este caso, la *feature* aprendida por una unidad y el patrón que esa misma unidad tiende a reconstruir están fuertemente alineados. Dicho de otro modo, cada neurona no solo detecta una característica relativamente específica, sino que también la reutiliza como plantilla elemental de reconstrucción, lo cual resulta conceptualmente esperable en un autoencoder.

En la variante sin regularización vuelve a dominar una estructura ruidosa, con menor localización espacial y con artefactos menos interpretables. *DropConnect*, por su parte, mantiene un comportamiento intermedio muy similar al de la figura anterior: siguen apareciendo *spots* localizados, aunque inmersos en patrones más extendidos y menos limpios que en *Dropout*.

El caso de *BoostDropout* vuelve a diferenciarse del resto. Mientras que en la visualización de coadaptaciones predominaba un aspecto cercano al ruido uniforme, aquí aparecen muchas más estructuras localizadas y una organización visual más clara. Esto sugiere que, en *BoostDropout*, la *feature* leída directamente de los pesos del codificador no coincide necesariamente con la *feature* que la misma unidad privilegia al decodificar. En ese sentido, la etapa de decodificación parece estar más ordenada que lo que dejaba entrever la inspección directa de los pesos. Una interpretación posible es que la regularización introducida por *BoostDropout* no favorezca detectores localizados en el mismo sentido que *Dropout*, pero sí induzca una organización más estructurada del espacio latente desde el punto de vista reconstructivo.

4.2.3 Esparsidad de las representaciones

La Figura 29 resume los gráficos de *sparsity* representacional obtenidos para cada modelo. Dado que cada archivo integra dos gráficos complementarios, esta comparación permite evaluar de manera cualitativa cómo cambia la activación del espacio latente y cuán esparsa resulta la representación en cada caso.

Procedimiento de construcción de la figura.

Para construir esta visualización se consideró el autoencoder de referencia introducido en la Sección 3.3.2, es decir, un autoencoder denso de una sola capa oculta con arquitectura $784 \rightarrow 256 \rightarrow 784$ y activaciones ReLU en la capa latente.

Luego de entrenar cada modelo, se tomó un minibatch aleatorio del conjunto de test y se calcularon sus activaciones ocultas $h \in \mathbb{R}^{batch_size \times 256}$. A partir de ese único minibatch se construyeron dos histogramas complementarios: a la izquierda, la distribución de la activación media por unidad oculta; a la derecha, la distribución de todas las activaciones del minibatch.

El resultado más claro de esta figura es que *Dropout* induce una mayor esparsidad representacional que la observada en la red sin regularización. Cualitativamente, los histogramas de *OverFitNet* y *Dropout* conservan una forma general parecida, pero en *Dropout* aumenta de manera muy marcada la masa de activaciones nulas o prácticamente nulas dentro del minibatch analizado. En términos aproximados, se pasa de unas pocas centenas de activaciones cercanas a cero en *OverFitNet* a varios miles en *Dropout*, es decir, a un incremento del orden de magnitud, en línea con la tendencia señalada por Srivastava et al. [1].

DropConnect y *BoostDropout* también muestran un aumento visible de activaciones nulas o casi nulas respecto de *OverFitNet*, pero el efecto es sensiblemente menor que en *Dropout*. En ambos casos, la acumulación cerca de cero parece ubicarse en un régimen intermedio: mayor

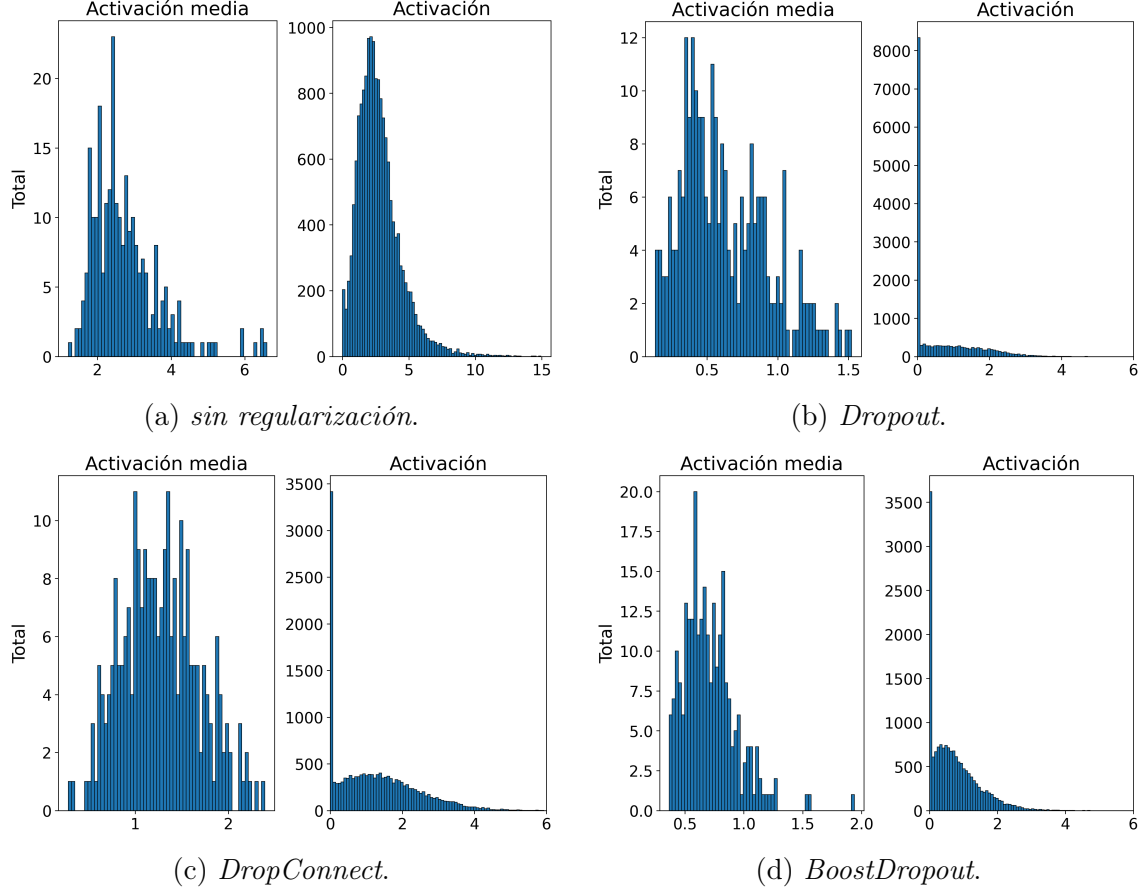


Figura 29: Comparación cualitativa de la esparsidad representacional en el autoencoder.

que en la red sin regularización, pero claramente inferior a la concentración extrema observada bajo *Dropout*. Esto sugiere que estas dos arquitecturas sí promueven cierta esparsidad representacional, aunque no con la misma intensidad.

Este resultado dialoga de manera interesante con las visualizaciones anteriores. En *Dropout*, la aparición de detectores más localizados y menos ruidosos parece venir acompañada por una representación más esparsa: menos unidades participan de manera fuerte ante un mismo ejemplo. En cambio, en *DropConnect* y *BoostDropout*, donde las *features* observadas eran menos localizadas o menos claramente interpretables en la visualización de coadaptaciones, la esparsidad inducida también resulta menor. Esto no demuestra una relación causal estricta, pero sí vuelve plausible la hipótesis de que una menor coadaptación visualmente evidente pueda estar asociada a una mayor especialización funcional de las unidades y, por ende, a una representación más esparsa.

En este sentido, el comportamiento de *BoostDropout* es particularmente interesante. Aunque en las figuras anteriores no mostraba la firma visual típica de *Dropout* para ruptura de coadaptaciones, aquí sí presenta un incremento de esparsidad respecto de *OverFitNet*. Esto sugiere nuevamente que su efecto regularizador podría no pasar por el mismo mecanismo que *Dropout*. Más que reproducir exactamente el patrón clásico de detectores localizados y gran esparsidad, *BoostDropout* parecería inducir una reorganización distinta del espacio latente,

con una esparsidad moderada pero no extrema.

Por último vale la pena mencionar que una mayor *representational sparsity* puede interpretarse como señal de una codificación más selectiva o más compacta en sentido funcional, ya que para cada ejemplo queda involucrado un subconjunto más pequeño de unidades latentes. No obstante, conviene no identificar este efecto con una mayor compresión de datos en sentido estricto, dado que la dimensión del espacio latente permanece fija y lo que cambia aquí no es el tamaño de la compresión de la imagen, sino la fracción de neuronas efectivamente activas en cada representación.

4.2.4 Esparsidad paramétrica en la primera capa

Por último, se examina la esparsidad paramétrica a partir de histogramas de pesos de la primera capa. Aunque las arquitecturas consideradas no imponen explícitamente una poda dura de parámetros, esta visualización permite explorar si algunos mecanismos de regularización inducen distribuciones de pesos más concentradas alrededor de cero o más esparsas.

La inspección de los histogramas de pesos de la primera capa sugiere que las arquitecturas regularizadas tienden, en distinto grado, a concentrar una mayor fracción de parámetros cerca de cero que la red sin regularización. Este efecto es más visible en *Dropout*, donde la distribución aparece más comprimida alrededor del origen y con colas menos extendidas. En términos cualitativos, esto indica una reducción del módulo típico de los pesos y sugiere la posible aparición de una forma débil de esparsidad paramétrica.

Sin embargo, conviene subrayar que esta evidencia no es concluyente. Una mayor concentración de pesos alrededor de cero no equivale, por sí sola, a esparsidad paramétrica en sentido estricto, ya que los histogramas no muestran una poda dura ni una acumulación exacta de masa en cero. Lo que sí puede afirmarse es que *Dropout* parece inducir una contracción más fuerte de la distribución de pesos de la primera capa.

DropConnect y *BoostDropout* también exhiben una tendencia en la misma dirección, aunque menos pronunciada. La diferencia se aprecia sobre todo en la escala vertical: en ambos modelos aumenta la densidad de pesos pequeños respecto de *OverFitNet*, pero el efecto no alcanza la intensidad observada en *Dropout*. Desde este punto de vista, el patrón vuelve a ser coherente con lo encontrado en la esparsidad representacional: *Dropout* aparece como el método que más fuertemente empuja a la red hacia configuraciones más localizadas y esparsas, mientras que *DropConnect* y *BoostDropout* parecen actuar de manera más moderada o por mecanismos parcialmente distintos.

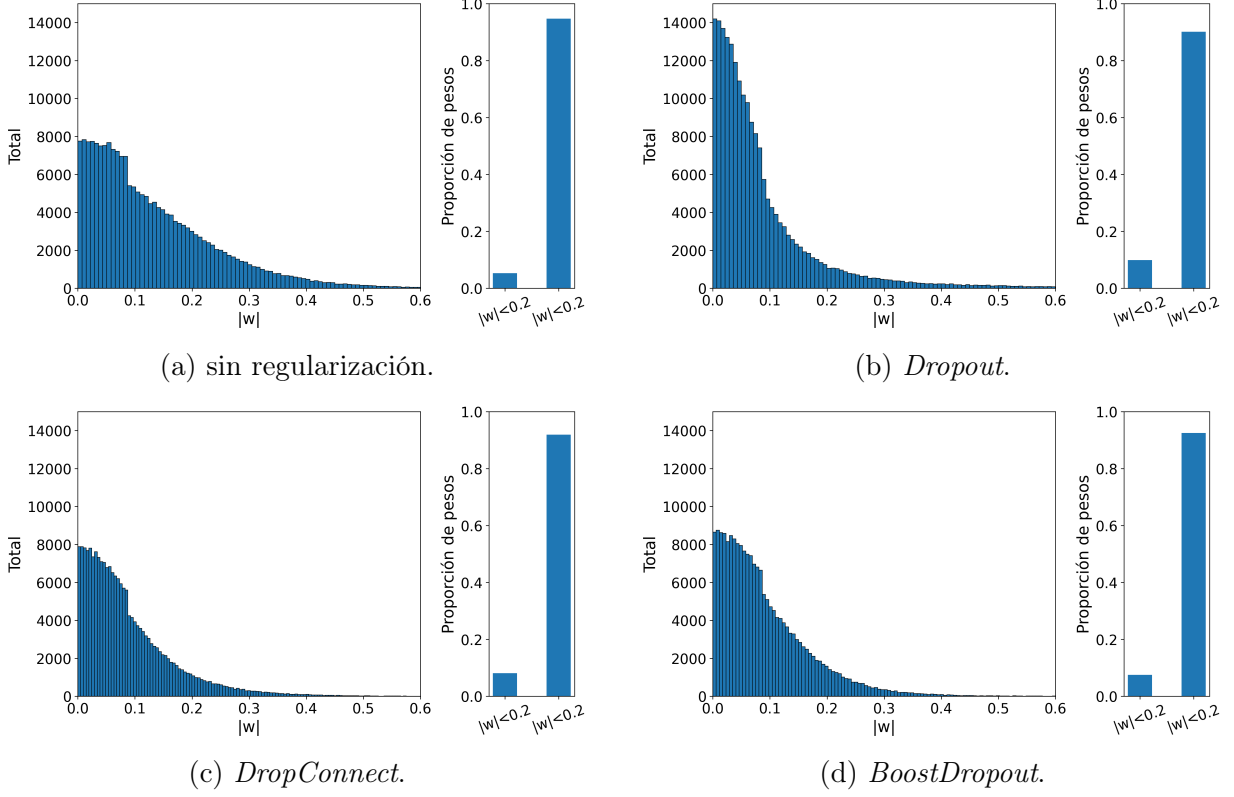


Figura 30: Histogramas de pesos de la primera capa como indicador cualitativo de esparsidad paramétrica.

En conjunto, esta figura debe interpretarse como un indicador ilustrativo y no como una demostración de esparsidad paramétrica estricta. Aun así, aporta una señal consistente con el resto del análisis cualitativo: las tres regularizaciones modifican la organización interna del modelo, pero *Dropout* lo hace de una manera más alineada con la intuición clásica de detectores especializados y pesos efectivos de menor magnitud.

4.2.5 Conclusiones del análisis cualitativo con autoencoder

En conjunto, las visualizaciones cualitativas del autoencoder complementan de manera útil los resultados cuantitativos del protocolo principal de clasificación. Mientras las métricas finales mostraron que *BoostDropout* fue la arquitectura de mejor desempeño en el problema supervisado, el análisis representacional revela que esa ventaja no reproduce simplemente, de manera ampliada, el patrón clásico asociado a *Dropout*, sino que parece responder a una forma distinta de regularización interna.

Desde el punto de vista visual, *Dropout* es la arquitectura que muestra con mayor claridad la firma cualitativa tradicional de ruptura de coadaptaciones: detectores más localizados, menor ruido en las *features* de primera capa y una correspondencia bastante directa entre la característica aprendida por cada unidad y el patrón que esa misma unidad tiende a reconstruir. Además, es también el método que induce la mayor esparsidad representacional en el espacio latente, coherente con la idea de unidades más especializadas y menos simultáneamente activas.

DropConnect ocupa, en general, una posición intermedia. Sus visualizaciones sugieren una reducción parcial del ruido y la emergencia de estructuras reconocibles, aunque menos nítidas y menos localizadas que en *Dropout*. Del mismo modo, la esparsidad representacional y la contracción de pesos aparecen, pero con menor intensidad. Esto sugiere un efecto regularizador real, aunque de naturaleza menos marcada desde el punto de vista visual.

BoostDropout, por su parte, constituye el caso más interesante. En la visualización directa de coadaptaciones no exhibe de manera clara la firma típica de *Dropout*; sin embargo, al activar neuronas latentes individuales aparecen patrones reconstructivos bastante más organizados que los sugeridos por la inspección de pesos. A esto se suma una esparsidad representacional moderada y una cierta contracción de pesos, aunque nuevamente más débil que en *Dropout*. En conjunto, estos resultados sugieren que *BoostDropout* regulariza efectivamente a la red, pero no necesariamente mediante el mismo mecanismo visualmente identificable por el cual *Dropout* parece romper coadaptaciones.

Por lo tanto, el análisis cualitativo refuerza una lectura importante de esta tesis: la mejora de *BoostDropout* en el protocolo principal no debe interpretarse como una mera versión más intensa de *Dropout*. Más bien, los resultados sugieren que distintos mecanismos de regularización pueden conducir a mejoras de generalización por caminos internos diferentes. En ese marco, *Dropout* aparece como el método que más claramente favorece detectores interpretables y representaciones esparsas, mientras que *BoostDropout* parece reorganizar el espacio latente de una manera menos clásica, pero igualmente efectiva desde el punto de vista del desempeño final.

Capítulo 5. Conclusiones generales y próximos pasos

El objetivo de esta tesis fue estudiar el efecto de una estrategia de regularización no binaria sobre redes neuronales *feed-forward*, tomando como punto de partida el problema clásico del sobreajuste y las limitaciones del *dropout* aleatorio tradicional. En ese marco, se propuso y analizó *BoostDropout*, una variante basada en modulación multiplicativa de activaciones, inspirada en la idea de dilución no aleatoria y en su posible vínculo con fenómenos estudiados en física estadística.

Los resultados experimentales obtenidos en la tarea principal de clasificación sobre MNIST muestran, dentro del protocolo adoptado, un comportamiento consistentemente favorable para *BoostDropout*. La arquitectura propuesta alcanzó en general menores valores de pérdida y de error que *OverFitNet*, *Dropout* y *DropConnect*, y además exhibió una dinámica de validación más estable, con menor evidencia visible de sobreajuste en etapas tardías del entrenamiento. Esta ventaja no apareció solo en una corrida particular, sino que se sostuvo al considerar tablas resumen, boxplots, curvas representativas y estudios de sensibilidad respecto de los hiperparámetros p y λ .

El análisis cualitativo con autoencoders permitió complementar esa observación desde otro ángulo. Allí se vio que *Dropout* reproduce con claridad la firma visual clásica asociada a la ruptura de coadaptaciones y a una mayor esparsidad representacional. *BoostDropout*, en cambio, no mostró exactamente ese mismo patrón, aun cuando su desempeño supervisado fue superior en la tarea de clasificación. Esto sugiere que su efecto regularizador no debe interpretarse simplemente como una versión más intensa de *Dropout*, sino como un mecanismo de naturaleza diferente, capaz de mejorar la generalización por una vía menos evidente desde el punto de vista visual.

Al mismo tiempo, el alcance de estas conclusiones debe entenderse dentro de los límites del diseño experimental. Se trabajó sobre un subset estratificado de MNIST, con una semilla de partición fija, una exploración inicial acotada del espacio de hiperparámetros y una evaluación final multisevilla compatible con el presupuesto computacional de una tesis de grado. Por lo tanto, los resultados no constituyen una demostración exhaustiva de superioridad universal de *BoostDropout*, sino una evidencia experimental controlada y metodológicamente consistente dentro del escenario estudiado.

En consecuencia, un paso natural a futuro consiste en ampliar el análisis en dos direcciones. Por un lado, sería deseable explorar con mayor exhaustividad el espacio de hiperparámetros, tanto para clasificación como para autoencoders. Por otro, sería especialmente importante incorporar no solo más semillas experimentales, sino también más semillas de partición del subset, de modo que cambie el muestreo mismo del problema. Esto permitiría verificar con mayor solidez si la ventaja observada para *BoostDropout* se mantiene de forma estable y no depende de una partición particular de los datos.

Más allá de estas extensiones inmediatas, también resultaría natural evaluar el método en datasets más complejos, en arquitecturas más profundas y en contextos donde el costo computacional permita comparaciones más amplias. Desde una perspectiva conceptual, queda abierta además la posibilidad de desarrollar una interpretación teórica más precisa para regularizaciones multiplicativas no binarias, conectando de manera más directa estos resultados con ideas de sistemas complejos, dinámica colectiva y dilución no aleatoria.

En síntesis, esta tesis aporta evidencia a favor de que una modulación multiplicativa

controlada de activaciones puede constituir una alternativa prometedora dentro de la familia de regularizaciones estocásticas para redes neuronales *feed-forward*. Como trabajo de grado, su principal aporte no es clausurar el problema, sino establecer una base experimental y conceptual sólida para seguir investigando el papel de la dilución no aleatoria en aprendizaje profundo.

En ese recorrido, la tesis ofreció además una oportunidad especialmente valiosa para pasar de la comprensión teórica a la construcción efectiva de modelos. El trabajo con *PyTorch* no consistió únicamente en entrenar arquitecturas estándar, sino también en desarrollar herramientas propias, implementar capas personalizadas y enfrentarse de manera directa a los detalles concretos que hacen posible un experimento reproducible. Esto permitió adquirir una visión más madura del proceso de modelado: no alcanza con proponer una idea interesante desde el punto de vista teórico, sino que también es necesario traducirla a una implementación consistente, controlar cuidadosamente sus condiciones de entrenamiento e interpretar sus efectos con criterio experimental.

Desde una perspectiva más amplia, esta tesis también permitió poner en juego, integrar y resignificar aprendizajes construidos a lo largo de la carrera. Conocimientos provenientes de materias diversas encontraron aquí un espacio común de aplicación y diálogo, ya no como contenidos separados, sino como herramientas complementarias para abordar un problema de investigación concreto. En ese sentido, el trabajo realizado no solo aportó resultados sobre una variante específica de regularización, sino que también fortaleció una forma de pensar y de trabajar en la que la teoría, la implementación, la experimentación y la interpretación crítica forman parte de un mismo proceso de construcción de conocimiento.

Anexo A: Nomenclatura

En este anexo se resume la notación utilizada a lo largo del trabajo para mantener consistencia entre secciones teóricas y resultados experimentales.

- $\mathbf{x}$: vector de entrada de una muestra.
- n : dimensión del vector de entrada.
- d : dimensión genérica de un vector de activaciones o de una entrada intermedia cuando no coincide necesariamente con la entrada del modelo.
- d_l : dimensión del vector de activaciones en la capa l .
- y : etiqueta verdadera; en multiclase, $y \in \{1, \dots, C\}$.
- $\hat{y}$: predicción del modelo; en clasificación puede denotar la clase predicha y en regresión el valor estimado.
- c : índice de clase, con $c \in \{1, \dots, C\}$.
- y_{ic} : codificación one-hot de la clase c para la muestra i .
- $\mathbf{z}$ o $\mathbf{z}_i$: vector de logits (pre-activaciones de salida).
- z_c (o z_{ic}): logit asociado a la clase c (o a la muestra i y clase c).
- $\hat{p}_{ic}$: probabilidad predicha para la clase c (salida de softmax).
- L : función de pérdida promedio sobre un conjunto de datos o mini-batch.
- $\boldsymbol{\theta}$: conjunto de parámetros entrenables del modelo.
- η : tasa de aprendizaje.
- $\mathcal{B}_t$: mini-batch en la iteración t .
- m : máscara Bernoulli de dropout; $q = 1 - p$ es la probabilidad de retención.
- λ : parámetro de amplificación/atenuación en la capa *BoostDropout*.
- $m_j^{(l)}$: máscara aleatoria aplicada a la activación $h_j^{(l)}$ en *BoostDropout*.
- $\hat{m}_j^{(l)}$: versión normalizada de la máscara de *BoostDropout*.
- $\odot$: producto elemento a elemento.
- T : temperatura escalar usada en *temperature scaling* para calibración.

Convenciones tipográficas.

Se usa negrita para vectores (por ejemplo, $\mathbf{x}$, $\mathbf{z}$), y negrita griega para el vector de parámetros ($\boldsymbol{\theta}$). Las probabilidades predichas se denotan con sombrero ($\hat{p}$) y las variables calibradas o enmascaradas con tilde cuando corresponde.

Bibliografía

- [1] Nitish Srivastava, Geoffrey Hinton y ... «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». En: *Journal of Machine Learning Research* 15 (2014), págs. 1929-1958.
- [2] Li Wan et al. «Regularization of Neural Networks Using DropConnect». En: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013, págs. 1058-1066.
- [3] Warren S. McCulloch y Walter Pitts. «A Logical Calculus of the Ideas Immanent in Nervous Activity». En: *The Bulletin of Mathematical Biophysics* 5.4 (1943), págs. 115-133. DOI: 10.1007/BF02478259.
- [4] Frank Rosenblatt. «The perceptron: A probabilistic model for information storage and organization in the brain». En: *Psychological Review* 65.6 (1958), págs. 386-408. DOI: 10.1037/h0042519.
- [5] Marvin Minsky y Seymour Papert. *Perceptrons*. MIT Press, 1969.
- [6] David E. Rumelhart, Geoffrey E. Hinton y Ronald J. Williams. «Learning representations by back-propagating errors». En: *Nature* 323.6088 (1986), págs. 533-536. DOI: 10.1038/323533a0.
- [7] George Cybenko. «Approximation by superpositions of a sigmoidal function». En: *Mathematics of Control, Signals, and Systems* 2.4 (1989), págs. 303-314. DOI: 10.1007/BF02551274.
- [8] Kurt Hornik, Maxwell Stinchcombe y Halbert White. «Multilayer feedforward networks are universal approximators». En: *Neural Networks* 2.5 (1989), págs. 359-366. DOI: 10.1016/0893-6080(89)90020-8.
- [9] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [10] Vinod Nair y Geoffrey E. Hinton. «Rectified linear units improve restricted boltzmann machines». En: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. 2010.
- [11] Xavier Glorot, Antoine Bordes y Yoshua Bengio. «Deep sparse rectifier neural networks». En: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, págs. 315-323.
- [12] Djork-Arné Clevert, Thomas Unterthiner y Sepp Hochreiter. «Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)». En: *International Conference on Learning Representations (ICLR)* (2016). arXiv: 1511.07289 [cs.LG].
- [13] Prajit Ramachandran, Barret Zoph y Quoc V. Le. «Searching for Activation Functions». En: *arXiv preprint arXiv:1710.05941* (2017).
- [14] Paul J. Werbos. «Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences». Tesis doct. Harvard University, 1974.

- [15] Xavier Glorot y Yoshua Bengio. «Understanding the difficulty of training deep feed-forward neural networks». En: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2010, págs. 249-256.
- [16] Kaiming He et al. «Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification». En: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2015, págs. 1026-1034. DOI: 10.1109/ICCV.2015.123.
- [17] Sergey Ioffe y Christian Szegedy. «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift». En: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, págs. 448-456.
- [18] Kaiming He et al. «Deep Residual Learning for Image Recognition». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, págs. 770-778. DOI: 10.1109/CVPR.2016.90.
- [19] Chuan Guo et al. «On Calibration of Modern Neural Networks». En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, págs. 1321-1330. URL: <https://arxiv.org/pdf/1706.04599>.
- [20] Geoffrey Hinton, Oriol Vinyals y Jeff Dean. «Distilling the Knowledge in a Neural Network». En: *arXiv preprint arXiv:1503.02531* (2015). NIPS 2014 Deep Learning Workshop. URL: <https://arxiv.org/abs/1503.02531>.
- [21] Léon Bottou. «Large-Scale Machine Learning with Stochastic Gradient Descent». En: *Proceedings of COMPSTAT'2010*. Physica-Verlag HD / Springer, 2010, págs. 177-186. DOI: 10.1007/978-3-7908-2604-3_16.
- [22] John Duchi, Elad Hazan y Yoram Singer. «Adaptive Subgradient Methods for Online Learning and Stochastic Optimization». En: *Journal of Machine Learning Research* 12 (2011), págs. 2121-2159.
- [23] Geoffrey E. Hinton. *Lecture 6e: RMSProp, Coursera Neural Networks for Machine Learning*. Video lecture and notes. 2012.
- [24] Diederik P. Kingma y Jimmy Ba. «Adam: A Method for Stochastic Optimization». En: *International Conference on Learning Representations (ICLR)* (2015). arXiv: 1412.6980 [cs.LG].
- [25] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <https://www.deeplearningbook.org>.
- [26] Priya Goyal et al. «Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour». En: *arXiv preprint arXiv:1706.02677*. 2017.
- [27] Song Han et al. «Learning both Weights and Connections for Efficient Neural Networks». En: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015, págs. 1135-1143.
- [28] Michael Zhu y Suyog Gupta. «To Prune, or Not to Prune: Exploring the Efficacy of Pruning for Model Compression». En: *arXiv preprint arXiv:1710.01878* (2017).
- [29] Zhuang Liu et al. «Learning Efficient Convolutional Networks through Network Slimming». En: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, págs. 2736-2744.

- [30] Geoffrey E. Hinton y Ruslan R. Salakhutdinov. «Reducing the Dimensionality of Data with Neural Networks». En: *Science* 313.5786 (2006), págs. 504-507. DOI: 10.1126/science.1127647.